

Elasticsearch 검색엔진: Politi-Search

남궁명수

프로젝트 개요

이 프로젝트는 검색 시스템 구축부터 성능 평가, 인프라 구조 비교까지의 전 과정을 통합적으로 다루는 것을 목표로 한 실험 중심 프로젝트이다. 실제 서비스 환경에서 활용 가능한 검색 시스템을 구현하고, 시스템 구성 방식에 따라 성능과 동작 특성이 어떻게 달라지는지를 분석하는 데 중점을 두었다.

프로젝트에서는 데이터 저장, 검색 처리, API 서버 구성, 그리고 인프라 환경 설계를 포함한 전체 서비스 흐름을 직접 설계·구현하였다. 이를 바탕으로 동일한 애플리케이션과 데이터셋을 유지한 상태에서, 시스템 구성 방식의 차이가 색인 속도와 검색 응답 성능에 미치는 영향을 비교 분석하였다.

또한 단순한 기능 구현에 그치지 않고, 실험 조건 통제, 테스트 코드 작성, 결과 수집 및 해석의 과정을 체계적으로 수행함으로써 성능 비교의 신뢰성을 확보하고자 하였다. 이를 통해 검색 시스템 설계 시 고려해야 할 구조적 선택 요소와 그에 따른 장단점을 실증적으로 정리하였다.

이 프로젝트는 검색 시스템을 처음 설계하거나 확장하려는 상황에서, 구현·운영·성능 분석 관점을 함께 고려한 설계 접근 방식의 중요성을 보여주며, 향후 유사한 시스템 구축 및 개선 작업을 위한 기초 자료로 활용될 수 있다.

개발 기간

- 2025.11.16 ~ 2025.12.19

연락처

- GitHub: https://github.com/Hwang-Injun34/elasticsearch_notes

- Email: myeongsu.namgung@gmail.com

목차

[1] 프로젝트 개요.....	6
[1-1] 프로젝트 배경	6
[1-2] 문제 정의.....	6
[1-3] 프로젝트 목표.....	6
[2] 전체 시스템 아키텍처	8
[2-1] 시스템 구성 개요	8
[2-2] 데이터 처리 파이프라인	8
[2-3] 의미 분석 및 검색 아키텍처.....	9
[2-4] 클라우드 기반 Elasticsearch 확장 아키텍처 설계 목표.....	9
[3] 하드웨어 구성(HW).....	10
[4] 서버 및 인프라 구조(Docker/Network)	11
[4-1] 개발 환경 및 인프라 구성 개요	11
[4-2] 전체 서버 구성 개요.....	11
[4-3] 컨테이너 기반 서비스 구성.....	12
[4-3-1] Nginx: 단일 진입점 및 리버스 프록시	12
[4-3-2] FastAPI: 백엔드 애플리케이션 서버.....	13
[4-3-3] Elasticsearch: 검색 엔진 인프라	13
[4-3-4] Kibana: 검색 엔진 관리 및 검증 도구.....	14
[4-3-5] MySQL: 관계형 데이터베이스 서버	14
[4-4] 네트워크 구조 및 서비스 간 통신.....	15
[4-5] 데이터 연속성 및 보안 설계.....	15
[5] 제공 서비스 및 애플리케이션	16
[5-1] 서비스 개요 및 사용자 시나리오	16
[5-1-1] 사용자 흐름.....	16
[5-2] 프론트엔드	17
[5-3] 백엔드 서비스 구성 및 데이터 처리 흐름.....	19
[5-3-1] 백엔드 API 구성 개요.....	19
[5-3-2] API 전체 명세.....	20
[5-3-3] 크롤링 처리 과정	20

[5-4] 데이터베이스 구조	23
[5-4-1] ERD(Entity Relationship Diagram)	23
[5-4-2] 테이블 설명	24
[6] 검색 엔진 설계 및 Elasticsearch 이론	25
[6-1] 빅데이터 처리 기술과 검색 엔진의 관계	25
[6-2] Lucene 기반 검색 엔진과 Elasticsearch 의 위치	25
[6-3] Elasticsearch 를 선택한 이유	25
[6-4] Elasticsearch 아키텍처 개요	26
[6-4-1] Elasticsearch 주요 용어 정리	27
[6-5] 색인(Indexing)과 역색인(Inverted Index)구조	28
[6-6] 전문 검색과 텍스트 분석 과정	29
[6-7] 검색 질의(Query)와 관련도 점수(Scoring)	30
[6-8] Elasticsearch 기반 검색 시스템 구현	31
[6-8-1] 환경 설정 및 연결(connection.py)	31
[6-8-2] 인덱스 생성 및 매핑 정의(index_manager.py)	31
[6-8-3] 데이터 색인 과정(indexer.py)	31
[6-8-4] 검색 서비스 구현(search_service.py / query_builder.py)	32
[6-8-5] API 연동(search.py)	32
[6-9] 검색 결과 해석	32
[6-9-1] 검색 정확도(Relevancy)	32
[6-9-2] BM25 기반 검색 결과 분석	33
[6-9-3] 검색 품질 개선 방향	33
[7] 데이터 파이프라인 및 인공지능(AI) 설계	35
[7-1] 데이터 파이프라인	35
[7-1-1] PDF 텍스트 추출(PdfExtraction)	35
[7-1-2] 발화자 단위 세그먼트 분리(PdfTranscriptParser)	37
[7-1-3] 키워드 추출	39
[7-1-4] 문장 임베딩 생성	40
[7-2] AI 적용 범위 및 설계 의도	41
[7-2-1] 규칙 기반 텍스트 처리의 한계	41
[7-2-2] 형태소 분석 기반 접근을 선택한 이유	41
[7-2-3] 임베딩 기반 의미 분석을 도입한 이유	41
[7-2-4] 규칙·언어학·의미 모델의 결합 구조	42
[7-3] 텍스트 정제 단계: Kiwi 형태소 분석기	42

[7-3-1] Kiwi 형태소 분석기의 기본 개념.....	42
[7-3-2] Kiwi 형태소 분석기의 작동 원리.....	43
[7-3-4] 프로젝트에서의 실제 적용 코드.....	45
[7-3-5] 설계 의의.....	45
[7-4] Ko-SBERT 기반 문맥 의미 임베딩.....	46
[7-4-1] 문맥 기반 의미 임베딩의 기본 개념.....	46
[7-4-2] Ko-SBERT의 구조적 정의 및 임베딩 생성 과정.....	47
[7-4-3] 임베딩 공간의 기하학적 해석.....	48
[7-4-4] MMR 기반 의미 다양성 확보.....	48
[7-4-5] jhgan/ko-sbert-sts 모델 선택 이유.....	49
[7-4-6] 정리.....	49
[7-5] 정보 추출 최적화 및 파라미터 튜닝.....	49
[7-5-1] 언어학적 특징 기반 후보 생성.....	49
[7-5-2] 하이퍼파라미터 그리드 서치 설계.....	51
[7-5-3] 추론 단계 계산 최적화.....	52
[7-6] 결과 해석.....	52
[7-6-1] 문서별 키워드 추출 결과 개요.....	52
[7-6-2] N-gram 범위에 따른 영향 분석.....	53
[7-6-3] MMR 다양성 파라미터(Diversity)의 효과.....	54
[7-6-4] 문서 4의 키워드 미추출 결과 해석.....	54
[7-6-5] 종합 해석 및 시사점.....	55
[8] 클라우드.....	56
[8-1] 사용한 플랫폼 서비스.....	56
[8-2] 단일 노드와 클러스터 환경 비교를 위한 시스템 설계.....	56
[8-2-1] 클라우드 VM 공통 구성.....	57
[8-2-2] 단일노드 환경 구축.....	59
[8-2-3] 노드 역할 분리를 적용한 클러스터 환경(1).....	62
[8-2-4] 노드 역할 분리를 적용한 클러스터 환경(2).....	65
[8-3] 비교.....	67
[8-3-1] 테스트 코드 설명.....	67
[8-3-2] 각 환경 테스트 결과.....	69
[8-4] 분석.....	75
[8-4-1] 색인 속도 테스트 분석.....	75
[8-4-2] 검색 속도 테스트 분석.....	75
[8-5] 최종 선택.....	76

[9] Elasticsearch 클러스터 기반 Full-Stack GCP 설계.....	78
[9-1] 시스템 구성	78
[9-2] 웹 서버 및 프론트엔드 설계(VM5)	78
[9-3] 로드밸런싱 및 오토스케일링 아키텍처 설계	80
[9-4] 아키텍처	81
[10] Elasticsearch 클러스터 기반 Full-Stack GCP 구현	82
[10-1] Web Server(Nginx) 구축 및 Reverse Proxy 연결	82
[10-2] 로드밸런싱	85
[10-2-1] FastAPI 백엔드 VM 을 사용자 지정 이미지로 생성	85
[10-2-2] 인스턴스 템플릿 생성	86
[10-2-3] 관리형 인스턴스 그룹(MIG) 생성	87
[10-2-4] 로드밸런서(Load Balancer) 생성	89
[10-2-5] 방화벽 규칙 설정	90
[10-2-6] 테스트	91
[10-3] 결과 화면	91
[10-4] 수정 및 업데이트 방법	93
[10-4-1] 원본(VM1) 수정	93
[10-4-2] 새로운 이미지 생성(v2)	93
[10-4-3] 인스턴스 템플릿 만들기	94
[10-4-4] 운영 서버 무중단 배포(Rolling Update)	96

[1] 프로젝트 개요

[1-1] 프로젝트 배경

국회는 입법과 정책 결정 과정에서 이루어지는 모든 논의를 회의록 형태로 기록하고 있다. 그러나 현재 제공되는 국회 회의록은 대부분 PDF 문서 단위로 구성되어 있으며, 사용자가 특정 의원의 발언이나 관심 있는 정책 논의를 확인하기 위해서는 여러 문서를 직접 열어 검색해야 한다. 이러한 방식은 회의록을 단순한 기록물로 보존하는 데에는 적합할 수 있으나, 실제 국회에서 어떤 논의가 어떻게 이루어졌는지를 탐색하기에는 구조적인 한계를 지닌다.

특히 회의록 내부에는 발언, 안건 설명, 진행 발언, 집단 응답 등이 하나의 연속된 텍스트로 혼재되어 있으며, 발언자와 발언 구간이 명확하게 분리되어 있지 않은 경우도 많다. 그 결과 문서 전체를 읽지 않고서는 개별 발언의 맥락을 이해하기 어렵고, 국회 논의의 흐름을 빠르게 파악하는 데에는 상당한 시간과 노력이 요구된다. Politi-Search 프로젝트는 이러한 문제의식에서 출발하여, 국회 회의록을 단순한 문서가 아닌 분석과 탐색이 가능한 데이터로 재구성하고자 한다.

[1-2] 문제 정의

기존 국회 회의록 검색 환경은 키워드 중심의 문서 검색 방식에 의존하고 있으며, 이로 인해 발언 단위 정보 접근에 여러 제약이 발생한다. 검색 결과는 주로 키워드가 포함된 PDF 파일 목록 형태로 제공되며, 사용자는 해당 문서를 직접 열어 내부 검색을 반복해야만 원하는 발언을 찾을 수 있다. 이 과정에서 검색의 효율성은 크게 저하된다.

또한, 키워드 기반 검색은 발언이 등장한 정치적·정책적 맥락을 충분히 반영하지 못한다. 동일한 단어가 사용되었더라도 찬성 발언인지 반대 발언인지, 또는 어떤 논의 단계에서 언급되었는지를 구분하기 어렵다. 더 나아가 개별 발언이 구조화되어 있지 않기 때문에 의원별 발언 성향이나 시기별 주요 이슈 흐름과 같은 종합적인 분석 역시 제한적이다. 이러한 한계로 인해 국회 활동 전반을 입체적으로 이해하고 인사이트를 도출하는 데 어려움이 발생한다.

[1-3] 프로젝트 목표

Politi-Search 프로젝트의 주요 목표는 대한민국 국회 회의록을 분석 가능한 데이터로 정제하고, 이를 기반으로 키워드 검색과 의미 기반(시멘틱) 검색을 모두 지원하는 검색엔진을 구축하는 것이다. 이를 위해 회의록 문서를 발언 단위로 세분화하고, 각 발언에 대해 발언자, 회의 정보, 시점, 문맥 등 핵심 정보를 구조화하여 관리한다. 사용자는 문서 전체를 열람하지 않더라도 검색 결과만으로 발언의 주체와 맥락을 쉽게 파악할 수 있다.

각 발언에서 핵심 키워드를 추출하기 위해 Kiwi와 KeyBERT를 활용하고, 발언 내용을 벡터 형태로 수치화하기 위해 임베딩 모델을 적용한다. 이러한 과정은 단순 키워드 매칭을 넘어, 발언의 의미와 문맥을 고려한 검색을 가능하게 한다. 이를 통해 사용자는 국회 회의록

속에서 의미적 연관된 발언을 탐색하고, 논의의 흐름과 핵심 이슈를 보다 직관적으로 파악할 수 있는 환경을 제공받게 된다.

[2] 전체 시스템 아키텍처

Politi-Search 프로젝트의 시스템 아키텍처는 국회 회의록 PDF 문서를 출발점으로 하여, 비정형 텍스트 데이터를 발언 단위의 구조화된 데이터로 변환하고 이를 기반으로 의미 검색 및 분석 서비스를 제공하는 구조로 설계되었다. 전체 시스템은 데이터 처리 흐름을 중심으로 단계적으로 구성되었으며, 각 단계는 역할에 따라 명확히 분리되어 있다.

특히, 이 프로젝트는 로컬 환경에서의 기능 구현을 중심으로 개발되었으나, 실제 서비스 운영을 고려한 클라우드 연계 및 확장 구조 또한 함께 설계하였다. 이를 통해 구현 범위와 설계 범위를 명확히 구분하면서도, 시스템의 확장 가능성과 실환경 적용 가능성을 함께 제시하고자 하였다.

[2-1] 시스템 구성 개요

전체 시스템은 크게 데이터 수집 및 전처리 계층, 의미 분석 및 검색 계층, 서비스 제공 계층으로 구성된다. 각 계층은 독립적인 역할을 수행하며, 데이터는 단계적으로 상위 계층으로 전달되는 구조를 가진다.

데이터 수집 및 전처리 계층에서는 국회 회의록 PDF 문서를 입력으로 받아 텍스트를 추출하고, 이를 발언 단위로 분해·정제하는 역할을 담당한다. 이 단계에서 생성된 발언 데이터는 이후 모든 검색과 분석의 최소 단위가 된다.

의미 분석 및 검색 계층에서는 발언 단위로 정제된 데이터를 기반으로 키워드 및 의미 임베딩을 생성하고, 검색 엔진에 인덱싱한다. 이를 통해 단순 키워드 검색뿐만 아니라 발언 간 의미적 유사성을 고려한 검색이 가능하도록 설계하였다.

서비스 제공 계층에서는 사용자로부터 검색의 요청을 처리하고, 발언 단위 검색 결과를 API 형태로 제공한다.

[2-2] 데이터 처리 파이프라인

시스템의 데이터 처리 파이프라인은 국회 회의록 PDF 문서를 발언 단위의 구조화된 데이터로 변환하는 과정을 중심으로 설계되었다. 회의록은 텍스트 기반 PDF 형식으로 제공되지만, 페이지 머리말, 안건 목록, 집단 응답 등 실제 발언과 직접적인 관련이 없는 요소들이 함께 포함되어 있어 단순한 텍스트 추출만으로는 정확한 발언 단위 분리가 어렵다.

이에 따라 PDF에서 추출된 텍스트는 전처리 단계에서 발언자 인식, 발언 구간 분리, 불필요한 메타데이터 제거 과정을 거친다. 이 과정에서 발언과 안건, 진행 문구를 구분하여 발언의 의미가 왜곡되지 않도록 설계하였다. 전처리가 완료되면 각 발언은 발언자 정보, 회의 정보 등의 메타데이터와 함께 하나의 독립적인 데이터 객체로 생성된다.

이렇게 생성된 발언 데이터는 이후 의미 분석 및 검색 인덱싱 단계로 전달되며, 시스템 전반에서 일관된 데이터 흐름을 유지한다. 이러한 파이프라인 구조는 데이터 품질을 확보하는 동시에, 검색 결과의 신뢰성을 높이는 핵심 요소로 작용한다.

[2-3] 의미 분석 및 검색 아키텍처

의미 분석 및 검색 계층에서는 발언 단위 데이터를 대상으로 형태소 분석과 문맥 기반 임베딩을 수행하여 검색에 활용 가능한 형태로 변환한다. 발언의 핵심 키워드를 추출하고, 발언 전체의 의미를 벡터 공간에 매핑함으로써 발언 간 의미적 유사성을 계산할 수 있도록 설계하였다.

검색 엔진은 이러한 키워드 정보와 의미 임베딩을 함께 활용하는 구조로 구성되었으며, 이를 통해 키워드 중심 검색과 의미 기반 검색을 결합한 하이브리드 검색이 가능하다. 사용자는 단순히 특정 단어를 포함한 발언뿐만 아니라, 의미적 유사한 발언까지 함께 탐색할 수 있다.

이 계층은 검색 성능과 확장성을 고려하여 설계되었으며, 데이터 규모 증가에 따라 인덱스 확장이나 검색 노드 분리가 가능하도록 구조적으로 분리되어 있다.

[2-4] 클라우드 기반 Elasticsearch 확장 아키텍처 설계 목표

클라우드 구축 목표는 Elasticsearch 기반 검색엔진을 단일 노드 환경과 역할 분리형 클러스터 환경 두 가지로 구축하고, 각 구성 방식의 성능, 확장성, 안정성을 비교·분석하는 것이다. 단일 서버 기반의 단순 구조와, 마스터·데이터·인게스트 노드를 분리한 클러스터 구조를 각각 구축함으로써, 노드 역할 분리가 실제 검색 서비스 환경에서 제공하는 효율성을 검증한다.

이를 통해, 향후 고가용성과 확장성을 갖춘 안정적인 클라우드 검색 인프라 설계 방향을 제시하는 것이 최종 목표이다.

[3] 하드웨어 구성(HW)

프로젝트는 국회 회의록 PDF의 전처리, 발원 단위 분해, 의미 분석 및 검색 구조 검증 목적으로 하여, 로컬 개발 환경을 기준으로 설계·구현되었다. 실제 서비스 운영 환경보다는 데이터 처리 파이프라인과 검색 아키텍처의 구조적 타당성을 확인하는 데 중점을 두었기 때문에, 단일 로컬 환경에서 모든 시스템 구성 요소를 실행하는 형태로 프로젝트를 구성하였다.

로컬 환경은 PDF 대량 처리, 형태소 분석, 문맥 기반 임베딩, 검색 인덱싱과 같은 연산 집약적인 작업을 수행할 수 있도록 충분한 CPU 및 메모리 자원을 확보한 환경을 기준으로 사용하였다. 이에 따라 본 프로젝트의 하드웨어 구성은 기능 구현과 구조 검증을 수행하기 위한 최소 단위 환경을 기준으로 설계되었으며, 향후 클라우드 환경으로 이전할 경우 동일한 시스템 구조를 기반으로 연산 지원과 검색 노드를 확장할 수 있도록 고려되었다(00 장에서 자세히 설명). 즉, 프로젝트에서의 로컬 환경은 전체 시스템 동작을 검증하기 위한 기준 환경으로 활용되었다.

[4] 서버 및 인프라 구조(Docker/Network)

[4-1] 개발 환경 및 인프라 구성 개요

Politi-Search 프로젝트의 개발 환경과 서버 인프라는 Docker 기반 컨테이너 환경으로 구축하였다. 개발과정에서 로컬 운영체제(OS)나 라이브러리 및 패키지 버전 차이로 인해 발생할 수 있는 환경 불일치를 최소화하기 위해, 모든 서비스는 컨테이너 단위로 구성하였다. Docker 를 활용함으로써 동일한 실행 환경을 보장할 수 있으며, 서비스별 독립적인 실행과 재배포가 가능하도록 설계하였다.

각 서비스는 Dockerfile 를 통해 개별 이미지로 구성되었으며, MySQL 데이터베이스, Elasticsearch 검색 엔진, Nginx 리버스 프록시, FastAPI 백엔드 서버, 그리고 Node.js 기반 프론트엔드 애플리케이션을 포함한 전체 서비스는 docker-compose.yml 을 통해 통합 관리하였다. 이를 통해 개발 및 테스트 과정에서 서비스 간 의존성을 명확히 분리하고, 전체 시스템을 일관된 방식으로 구동할 수 있도록 하였다.

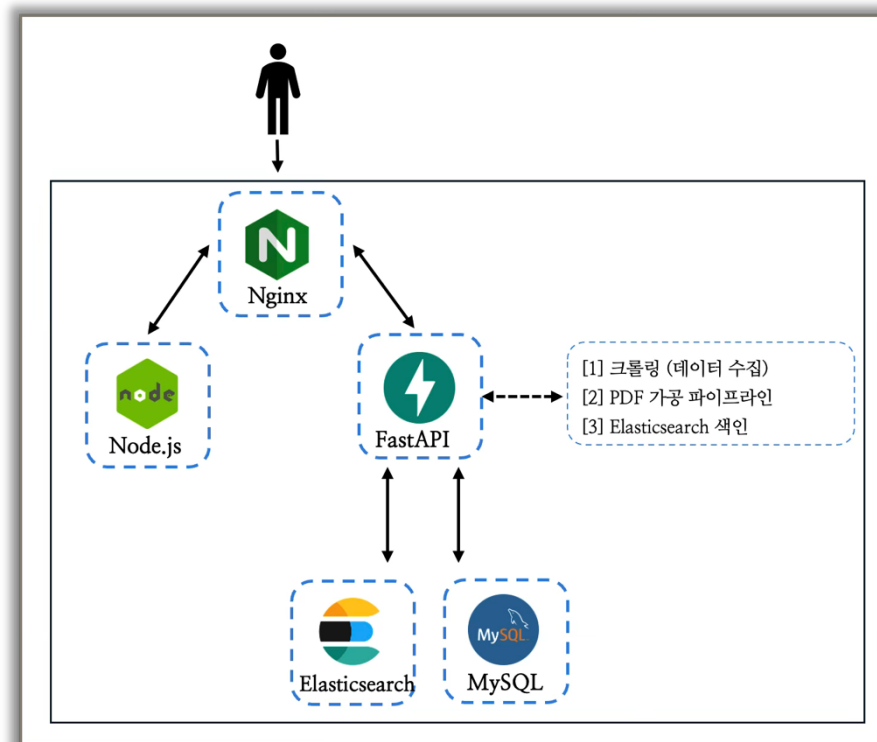
개발 단계에서는 단일 노드 기반의 Elasticsearch 환경을 구성하여 검색 기능 구현과 색인 구조 설계 성능 테스트를 진행하였다. 이후 클라우드 테스트 단계에서는 동일한 구조를 기반으로 노드 역할 분리형 클러스터 환경으로 확장하여, 실제 서비스 환경과 유사한 조건에서 검색 성능과 안정성을 검증할 수 있도록 설계하였다.

[4-2] 전체 서버 구성 개요

프로젝트의 서버 및 인프라 구조는 컨테이너 기반 환경(Docker)을 중심으로 설계되었다. 국회 회의록 PDF 의 전처리, 발언 단위 분해, 검색 색인 및 질의 처리라는 일련의 데이터 처리 파이프라인을 효율적으로 검증하기 위해, 각 기능을 담당하는 구성 요소를 독립적인 서비스 단위로 분리하였다.

전체 시스템은 Nginx 단일 진입점으로 설정하고, 그 뒤에 프론트엔드 애플리케이션과 FastAPI 기반 백엔드, 그리고 검색 기능의 핵심이 되는 Elasticsearch 를 배치하는 구조를 따른다. 사용자는 외부에서 Nginx 에만 접근하며, 내부 서비스들은 Docker 네트워크를 통해서만 상호 통신하도록 구성하였다.

이러한 구조는 로컬 개발 환경임에도 불구하고, 실제 서비스 운영 환경에서 일반적으로 사용되는 리버스 프록시 기반 아키텍처를 반영하고 있으며, 향후 클라우드 환경으로 이전 시에도 동일한 구조를 유지한 채 확장이 가능하도록 설계되었다.



[그림-1]

[4-3]

컨테이너 기반 서비스 구성

[4-3-1] Nginx; 단일 진입점 및 리버스 프록시

Nginx는 본 시스템에서 외부 요청의 단일 진입점 역할을 수행하는 웹 서버이자 리버스 프록시 서버로 사용되었다.

Nginx는 이벤트 기반 비동기 아키텍처를 기반으로 동작하며, 다수의 동시 요청을 효율적으로 처리할 수 있는 구조적 장점을 가진다. 이러한 특성은 프론트엔드와 백엔드가 분리된 시스템 구조에서 요청을 중앙에서 제어하고 라우팅하는 데 적합하다.

Politi-Search 프로젝트에서는 Nginx를 통해 정적 프론트엔드 리소스를 직접 서빙함과 동시에, API 요청을 내부 FastAPI 서버로 전달하는 리버스 프록시 역할을 수행하도록 구성하였다. 이를 통해 브라우저 사용자는 프론트엔드와 백엔드를 구분하지 않고 단일 주소로 시스템에 접근할 수 있으며, 내부 서비스 구조는 외부에 노출되지 않는다.

이와 같은 리버스 프록시 구조는 다음과 같은 설계적 이점을 제공한다.

첫째, 내부 서비스에 대한 직접 접근을 차단함으로써 보안성을 강화할 수 있다.

둘째, 요청 흐름을 중앙에서 통제함으로써 서비스 확장 및 구조 변경 시 유연한 대응이 가능하다.

셋째, 로컬 개발 환경임에도 불구하고 실제 서비스 운영 환경과 유사한 접근 구조를 유지할 수 있다.

[4-3-2] FastAPI: 백엔드 애플리케이션 서버

FastAPI는 Politi-Search 프로젝트의 백엔드 애플리케이션 서버로서, 검색 기능 및 데이터 조회를 위한 API를 제공한다.

FastAPI는 Python 기반의 고성능 웹 프레임워크로, ASGI(Asynchronous Server Gateway Interface)를 기반으로 비동기 요청 처리를 지원한다. 이를 통해 검색 요청과 같이 응답 지연이 발생할 수 있는 작업을 효율적으로 처리할 수 있다.

또한, FastAPI는 Pydantic을 활용한 자동 데이터 검증 기능과 OpenAPI 기반 API 문서 자동 생성 기능을 기본적으로 제공한다. 이러한 특성은 검색 결과, 발언 메타데이터, 필터 조건 등 구조화된 데이터를 다루는 시스템에서 명확한 스키마 정의와 일관된 응답 구조를 유지하는데 효과적이다.

Politi-Search 프로젝트에서는 FastAPI를 통해 Elasticsearch 검색 결과를 REST API 형태로 제공하며, 프론트엔드 애플리케이션은 해당 API를 통해 발언 단위 검색 결과를 조회한다. 이 과정에서 FastAPI는 단순한 요청 전달을 넘어, 검색 파라미터 검증 및 응답 구조 통제를 담당함으로써 시스템 전반의 데이터 일관성을 유지하는 역할을 수행한다.

[4-3-3] Elasticsearch: 검색 엔진 인프라

Elasticsearch는 Politi-Search 프로젝트에서 국회 회의록 검색 기능의 핵심을 담당하는 검색 엔진 인프라이다. 전통적인 관계형 데이터베이스(RDB)가 정형 데이터의 저장과 트랜잭션 처리에 강점을 지니는 반면, Elasticsearch는 역색인(Inverted Index) 구조를 기반으로 비정형 텍스트 데이터에 대한 고속 검색을 지원한다.

회의록 데이터는 자연어 텍스트로 구성된 비정형 데이터이며, 단순한 키워드 포함 여부를 넘어 문맥, 형태소, 단어 빈도와 같은 요소가 검색 품질에 중요한 역할을 미친다.

Elasticsearch는 이러한 요구를 충족하기 위해 형태소 분석, 가중치 기반 검색, 부분 일치 및 복합 조건 검색을 지원하며, 대규모 텍스트 데이터 처리에 적합한 구조를 제공한다.

Politi-Search 프로젝트에서는 회의록을 문서 단위가 아닌 발언 단위로 분해하여 색인을 구성하였다. 이를 통해 발언자, 발언 시점, 주제 키워드 등의 조건을 조합한 검색이 가능하도록 설계하였다. 이러한 색인 구조는 기존의 PDF 파일 단위 검색 방식과 달리, 사용자가 국회 논의의 흐름과 발언 맥락을 보다 세밀하게 탐색할 수 있도록 한다.

Elasticsearch는 단일 노드 구성으로 운영되며, 이는 검색 구조 설계와 성능 검증에 집중하기 위한 선택이다. 향후 다중 노드 클러스터 환경으로의 확장을 고려하여 기본적인 클러스터 설정 구조는 유지하였다.

[4-3-4] Kibana: 검색 엔진 관리 및 검증 도구

Kibana 는 Elasticsearch 와 연동되는 관리 및 시각화 도구로서, 단순한 운영 관리 도구를 넘어 검색 품질 검증 및 실험 도구로 활용하였다.

개발자는 Kibana 의 DevTools 콘솔을 통해 Elasticsearch Query DSL 을 직접 실행함으로써, API 구현 이전 단계에서 검색 쿼리의 동작과 결과를 사전에 검증할 수 있다.

이를 통해 색인 구조 변경이나 분석기 설정 조정에 따른 검색 결과 변화를 즉각적으로 확인할 수 있었으며, 검색 결과의 정확성과 일관성을 반복적으로 개선할 수 있었다. 이러한 검증 과정은 발언 단위 검색이라는 세밀한 검색 구조를 설계하는 데 있어 중요한 역할을 수행하였다.

결과적으로 Kibana 는 검색 엔진의 내부 동작을 시각적으로 확인하고, 데이터 기반 검색 시스템의 신뢰성을 확보하기 위한 핵심 도구로 활용되었다.

[4-3-5] MySQL: 관계형 데이터베이스 서버

Politi-Search 프로젝트에서는 국회 회의록 발언 데이터의 메타정보 및 검색 결과와 연관된 구조화된 데이터를 관리하기 위해 MySQL 기반의 관계형 데이터베이스를 사용하였다.

MySQL 은 정형 데이터의 저장과 관계 관리에 적합한 데이터베이스로, 검색 엔진인 Elasticsearch 의 역할을 분리하여 사용하였다.

실제 개발 환경에서는 MySQL 데이터베이스를 다른 프로젝트와 공용으로 사용하기 위해 별도의 Docker Compose 환경에서 실행하였다. 그러나 시스템 아키텍처 관점에서는 FastAPI 백엔드 서버가 내부 네트워크를 통해 MySQL 컨테이너에 접근하는 구조로 동작하였으며, 이는 단일 인프라 환경 내에서 데이터베이스 서버가 분리 배치된 형태로 볼 수 있다.

FastAPI 애플리케이션은 Docker 네트워크를 통해 MySQL 컨테이너와 연결되며, 발언 단위 데이터의 식별자, 발언자 정보, 회의 메타데이터 등 관계형 구조가 필요한 정보를 MySQL 에 저장하고 관리한다. 반면, 실제 검색 대상이 되는 발언 텍스트 본문은 Elasticsearch 에 색인되어 검색 성능과 유연성을 확보하였다.

이와 같은 구조는 관계형 데이터베이스(RDBMS)와 검색 엔진(Server Engine)의 역할을 명확히 분리한 설계로, 데이터 무결성과 검색 성능을 동시에 고려한 아키텍처이다. 비록 물리적으로는 데이터베이스 컨테이너가 다른 Docker 구성에 포함되어 있었으나, 논리적 시스템 구조상 프로젝트의 서버 및 인프라 구성 요소로 포함하여 설명할 수 있다.

[4-4] 네트워크 구조 및 서비스 간 통신

모든 서비스는 Docker 의 브리지 네트워크를 기반으로 구성되었으며, 컨테이너 이름을 호스트명으로 사용하여 내부 통신을 수행한다.

이를 통해 다음과 같은 통신 흐름이 형성된다.

- 사용자 → Nginx → 프론트엔드 정적 리소스
- 사용자 → Nginx → FastAPI → Elasticsearch
- FastAPI → MySQL

프론트엔드와 Elasticsearch 는 외부에서 직접 접근할 수 없으며, 반드시 Nginx 또는 FastAPI 를 경유하도록 설계되었다. 이는 시스템 내부 구조를 외부로부터 보호하고, 각 서비스의 책임 범위를 명확히 구분하기 위한 설계이다.

[4-5] 데이터 영속성 및 보안 설계

Elasticsearch 의 색인 데이터는 Docker 볼륨을 통해 관리되어, 컨테이너 생명주기와 무관하게 유지된다. 이로 인해 반복적인 색인 실험과 검색 구조 변경 과정에서도 데이터 손실 없이 안정적인 개발이 가능하다.

또한, Elasticsearch 와 Kibana 에는 인증 기반 접근 제어를 적용하여, 검색 엔진에 대한 무단 접근을 방지하였다. 비록 로컬 개발 환경이지만, 보안이 활성화된 구조를 유지함으로써 실제 서비스 환경으로의 이전을 고려한 설계를 반영하였다.

[5] 제공 서비스 및 애플리케이션

Politi-Search 시스템은 사용자가 국회 회의록 정보를 효율적으로 탐색하고, 개별 발언 단위의 상세 내용을 직관적으로 확인할 수 있도록 설계된 웹 기반 검색 서비스이다. 앞서 설명한 아키텍처와 검색 엔진 설계를 바탕으로, 5 장에서는 사용자 관점에서의 서비스 이용 흐름과 각 구성 요소의 역할, 그리고 발언 단위 검색 기능이 어떻게 구현되고 제공되는지를 중심으로 설명한다.

[5-1] 서비스 개요 및 사용자 시나리오

Politi-Search는 국회 회의록 PDF 문서를 발언 단위 데이터로 구조화하여, 사용자가 특정 발언, 발언자, 정책 이슈를 중심으로 국회 논의를 탐색할 수 있는 환경을 제공한다. 사용자는 웹 브라우저를 통해 서비스에 접속한 후, 검색창에 관심 있는 키워드를 입력함으로써 관련된 회의록 발언을 즉시 조회할 수 있다.

시스템은 입력된 키워드를 기반으로 의미적으로 연관된 발언 단위 데이터를 검색하고, 그 결과를 리스트 형태로 제공한다. 검색 결과에는 발언자 정보, 회의명, 발언 시점과 같은 메타데이터가 함께 표시되어, 사용자가 PDF 원문을 직접 열람하지 않더라도 국회 논의의 흐름과 맥락을 파악할 수 있도록 한다.

[5-1-1] 사용자 흐름

Politi-Search 서비스의 기본적인 이용 흐름은 다음과 같다.

1. 사용자는 웹 브라우저를 통해 Politi-Search 서비스에 접속한다.
2. 검색창에 관심 있는 주제와 관련된 키워드를 입력한다(예: “과기부”, “부동산 대책” 등).
3. 시스템은 Elasticsearch를 활용하여 입력된 키워드와 의미적으로 발언 데이터를 검색한다.
4. 검색 결과는 발언 단위 리스트 형태로 화면에 출력한다.
5. 사용자는 특정 발언을 선택하여 상세 페이지로 이동하고, 회의 제목, 회의 일자, 발언자, 발언 내용을 확인한다.

이와 같은 흐름을 통해 사용자는 전체 PDF 문서를 탐색하지 않고도, 관심 있는 발언과 논의 구간에 직접 접근할 수 있다.

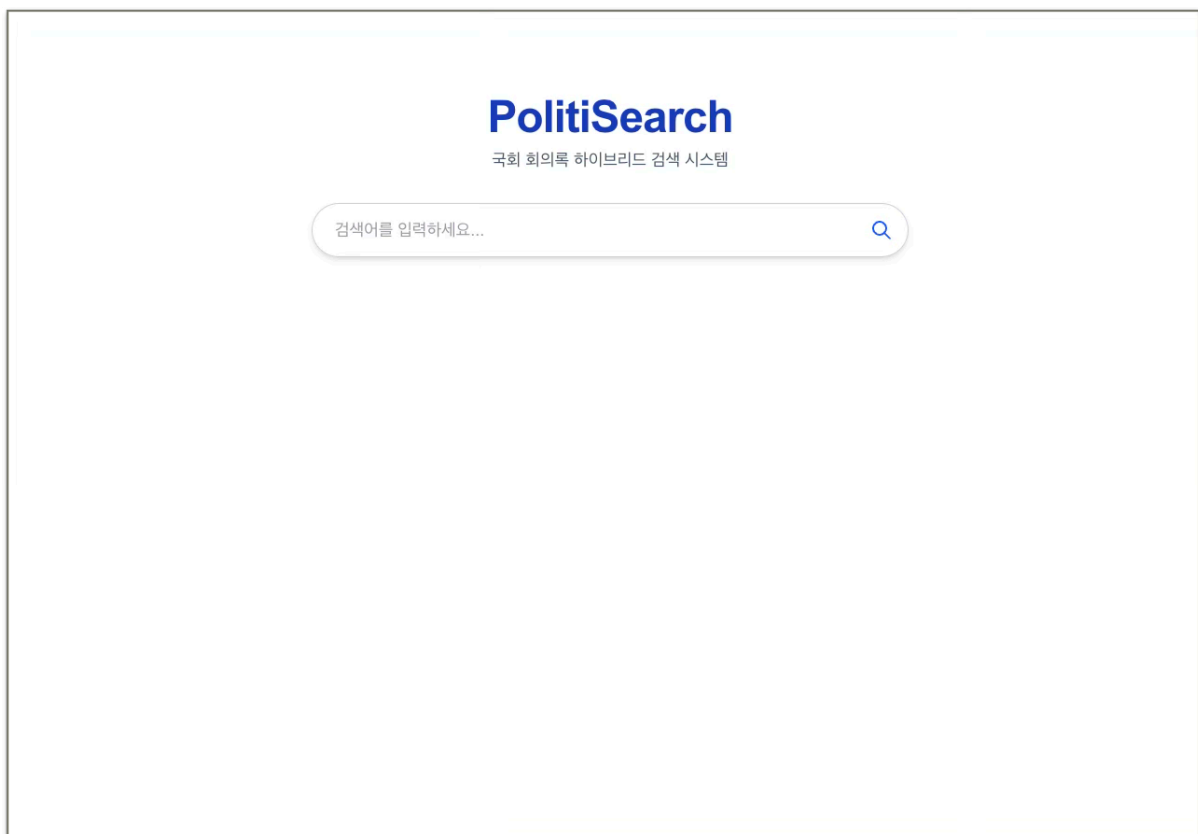
[5-2] 프론트엔드

Politi-Search의 프론트엔드는 Next.js 기반으로 개발되었으며, FastAPI 기반 백엔드에서 제공하는 검색 및 상세 조회 API와 연동하여 동작한다. 프론트엔드의 주요 목적은 사용자가 발언 단위 검색 결과를 직관적으로 탐색하고, 개별 발언의 맥락을 쉽게 이해할 수 있는 사용자 인터페이스를 제공하는 것이다.

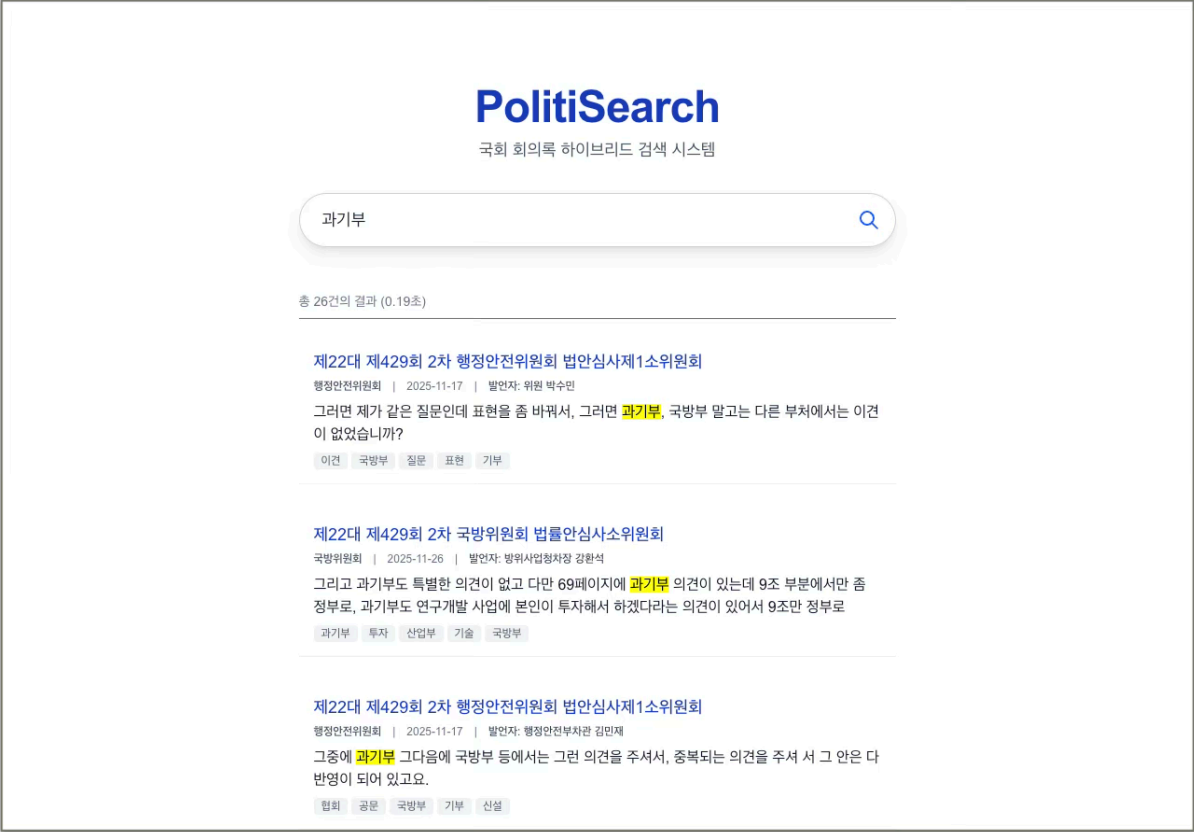
화면 디자인은 AI 기반 도구를 활용하여 설계하였으며, 개발 과정에서는 시각적 완성도보다는 API 통신 구조와 데이터 흐름 구현에 중점을 두었다.

주요 화면 구성요소로는 검색창, 발언 단위 검색 결과 리스트, 발언 상세 보기 화면이 포함된다.

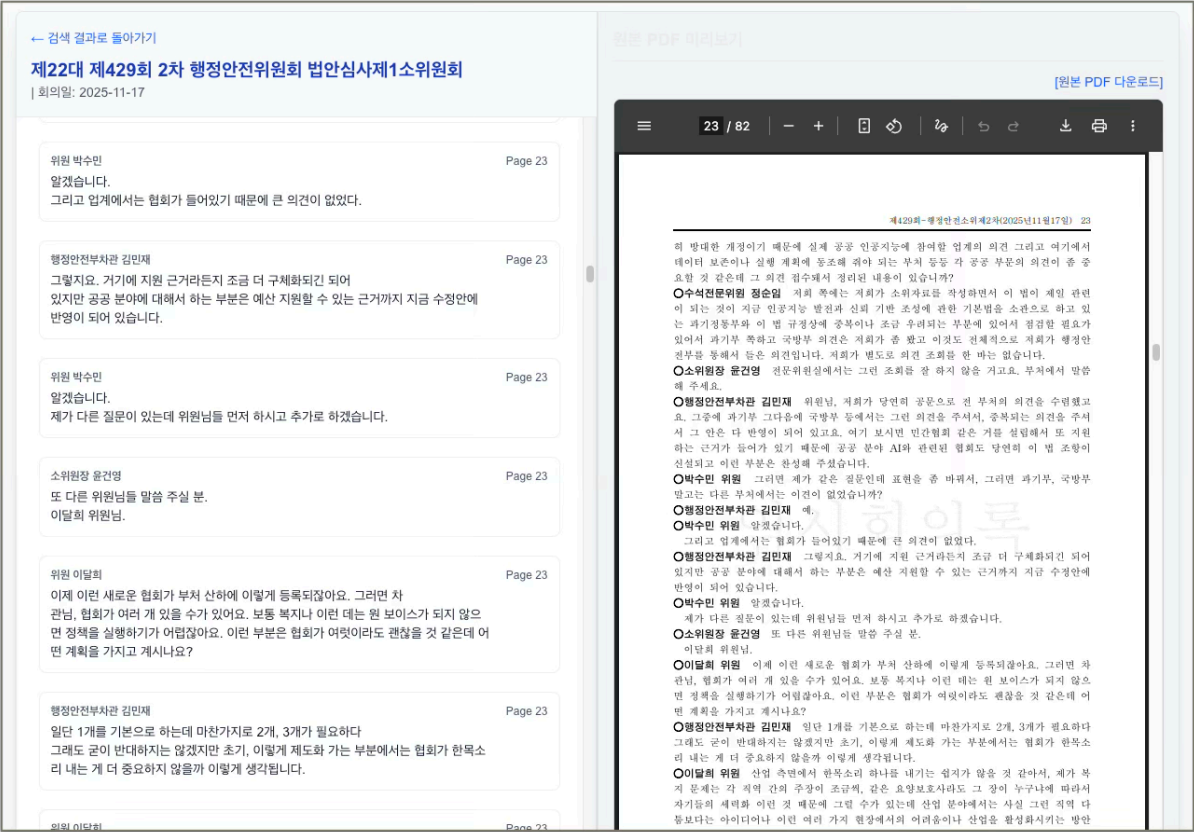
사용자가 검색어를 입력하면 프론트엔드는 이를 HTTP 요청으로 백엔드 서버에 전달하고, 반환된 검색 결과를 즉시 화면에 렌더링한다. 사용자는 검색 결과 중 관심 있는 발언을 선택하여 상세 페이지로 이동할 수 있으며, 이를 통해 발언자 정보와 회의 맥락을 함께 확인할 수 있다. 이러한 구성은 PDF 문서를 직접 열람하지 않고도 국회 논의의 핵심 내용을 파악할 수 있도록 설계되었다.



[그림-2]



[그림-3]



[그림-4]

[5-3] 백엔드 서비스 구성 및 데이터 처리 흐름

Politi-Search의 백엔드는 FastAPI 기반으로 구축되었으며, 사용자 요청 처리, 검색 쿼리 생성, 검색 결과 가공 및 전달을 담당한다. 프론트엔드로부터 검색 요청이 수신되면, 백엔드는 입력 파라미터를 검증한 뒤 Elasticsearch Query DSL을 기반으로 검색 쿼리를 생성하고 발언 단위 검색을 수행한다. 이후 검색 결과 중 서비스 제공에 필요한 필드만을 선별하여 응답 객체로 가공한 후 프론트엔드에 전달한다.

백엔드는 단순한 검색 중계 역할을 넘어, 검색 로그 기록, 필터 처리, 검색 결과 점수(score) 기반 정렬 등 검색 서비스의 안정성과 정확성을 보장하는 핵심 기능을 수행한다. 또한 데이터 수집, 문서 처리, 색인 관리로 이어지는 전체 검색 파이프라인을 API 단위로 분리하여 관리함으로써, 시스템의 확장성과 유지 보수성을 고려한 구조로 설계하였다.

FastAPI는 Starlette 기반의 ASGI 프레임워크로 비동기 I/O 처리에 강점을 가지며, 네트워크 요청, 파일 입출력, 데이터베이스 접근과 같이 I/O 바운드 작업이 많은 검색 서비스에 적합하다. 일부 동기 방식으로 동작하는 라이브러리는 `asyncio.to_thread`를 활용하여 별도 스레드에서 실행하도록 구성함으로써, 비동기 환경에서의 블로킹 문제를 완화하였다.

다만 로컬 및 클라우드 환경 모두 GPU 지원이 없는 상황에서 임베딩 벡터 생성과 같은 CPU 바운드 작업은 비동기 처리만으로 성능 개선에 한계가 있었다. 이러한 지속적인 계산 중심 작업에 대해서는 향후 별도의 처리 전략을 적용할 계획이다.

[5-3-1] 백엔드 API 구성 개요

백엔드 API는 RESTful 구조를 기반으로 하며, 기능별 책임을 명확히 분리하여 설계되었다. 주요 엔드포인트 구조는 다음과 같다.

/admins
/crawlers
/pdf
/search
/segment
/document

각 엔드포인트는 데이터 수집부터 검색 제공까지의 전체 흐름을 단계적으로 지원하며, 실제 서비스 환경에서의 운영 및 확장성을 고려한 구조이다.

[5-3-2] API 전체 명세

Endpoint	Method	설명
/admins/start_indexing	POST	Elasticsearch 전체 색인을 백그라운드 작업으로 실행
/admins/indexing_status	GET	현재 색인 작업의 실행 상태 조회
/crawlers/national-assembly	POST	국회 회의록 PDF 및 메타데이터 수집 실행
/pdf/extract	POST	PDF 페이지 단위 텍스트 추출
/pdf/parse	POST	화자 기준 발언 세그먼트 분리
/pdf/track-a	POST	키워드 추출(Kiwi + KeyBERT)
/pdf/track-b	POST	발언 임베딩 벡터 생성
/pdf/full-pipeline	POST	전체 처리 파이프라인 순차 실행
/search	GET	키워드 검색과 의미 기반 검색을 결합한 하이브리드 검색 제공
/segment/{segment_id}	GET	특정 발언 세그먼트 상세 정보 조회
/document/{doc_id}	GET	문서 전체 문맥 및 발언 구조 조회
/document/{doc_id}/pdf	GET	원본 PDF 파일 다운로드

[5-3-3] 크롤링 처리 과정

Politi-Search 시스템의 크롤링 과정은 국회 회의록 PDF 원문과 해당 문서의 메타데이터를 안정적으로 수집하는 것을 목표로 하며, 총 4 단계로 구성된다. 모든 과정은 대상 사이트(<https://www.assembly.go.kr/portal/main/contents.do?menuNo=600045>)의 서비스 정책을 준수하고 서버 부하를 최소화하는 방향으로 설계되었다.

1 단계: 타겟 사이트 분석 및 robots.txt 확인

크롤링에 앞서 국회 기록보존소 웹 페이지를 대상으로 robots.txt 파일을 사전 분석하여, 크롤링이 허용된 경로와 접근 제한 영역을 확인하였다. 이를 통해 비인가 접근이나 서비스 정책 위반을 사전에 방지하고, 합법적인 데이터 수집이 가능하도록 수집 범위를 설정하였다.

또한 웹 페이지의 구조와 문서 제공 방식을 분석하여, 회의록 목록 페이지의 URL 패턴, PDF 다운로드 링크 형식 등 자동화 수집이 가능한 영역을 선별하였다. 이러한 사전 분석을 통해 크롤링 대상과 범위를 명확히 정의할 수 있었다.

2 단계: 개발자 도구 기반 크롤링 전략 수립

브라우저 개발자 도구의 Network 탭을 활용하여 회의록 목록 조회 과정과 PDF 다운로드 요청 구조를 분석하였다. 그 결과, 단순한 HTML 페이지 파싱 방식이 아닌 JSON API 기반의 POST 요청 구조를 통해 회의록 데이터가 제공됨을 확인하였다.

주요 요청 파라미터 페이지 번호(pageIndex), 위원회 구분(cntsDivCd), 날짜 범위(beginDate, endDate), 그리고 CSRF 토큰(_csrf)으로 구성되어 있었다. 이를 기반으로 서버가 요구하는 요청 형식과 파라미터 구조를 정확히 재현하는 API 요청 방식의 크롤링 전략을 수립하였다.

응답 데이터에는 PDF 파일의 실제 다운로드 경로를 포함하는 pdfLinkUrl 필드가 포함되어 있었으며, 이를 활용하여 회의록 PDF 파일을 직접 다운로드하는 구조로 크롤링을 설계하였다.

3 단계: 수집 코드 구현

분석 결과를 토대로 Python 기반 비동기 크롤러를 구현하였다. HTTP 통신에는 httpx.AsyncClient 를 사용하여 비동기 요청을 처리하였으며, 초기 페이지 로딩 시 BeautifulSoup 을 활용해 CSRF 토큰을 자동으로 추출한 뒤 이후 API 요청에 포함하였다.

수집 과정의 전체 흐름은 다음과 같다.

- 메인 페이지 접속 후 CSRF 토큰 추출
- JSON API 기반 회의록 목록 조회(페이지네이션 반복)
- 각 회의록 항목의 메타데이터 파싱(회의명, 위원회명, 회의일, 회의번호 등)
- PDF 다운로드 링크 요청 및 로컬 파일 저장
- 메타데이터와 파일 경로를 정규화하여 데이터베이스에 저장

중복 수집을 방지하기 위해 회의번호(conferNum)를 기준으로 데이터베이스 중복 검사 로직을 적용하였으며, 서버 부하를 최소화하기 위해 각 요청 사이에 0.5~1.2 초 범위의 랜덤 지연 시간을 삽입하였다. 이를 통해 대량 수집 상황에서도 안정적인 크롤링이 가능하도록 하였다.

4 단계: 수집 정확도 검증 및 예외처리

샘플 기간을 대상으로 반복적인 크롤링 테스트를 수행한 결과, 회의록 메타데이터와 PDF 파일이 정상적으로 저장됨을 확인하였다.

또한 PDF 다운로드 실패, CSRF 토큰 누락, 날짜 파싱 오류 등 다양한 예외 상황에 대비하여 상세 로그 출력과 안전한 continue 처리 로직을 구현하였다. 이를 통해 일부 요청에서 오류가 발생하더라도 전체 크롤링 흐름이 중단되지 않도록 설계하였다.

아래 그림은 실제 크롤링 실행 결과로, 페이지 단위 API 호출과 PDF 파일 저장 성공 로그를 보여준다.

crawlers

POST /api/v1/crawlers/test National Assembly Crawler

Parameters Cancel Reset

No parameters

Request body required application/json

Edit Value | Schema

```
{
  "limit": 1,
  "title": "string"
}
```

Execute Clear

[그림-5]

```
크롤링 시작 | Limit: 1 | 기간: ~
[메인 페이지] 접속 및 보안 토큰 확보 중...
HTTP Request: GET https://www.assembly.go.kr/portal/main/contents.do?menuNo=600045 "HTTP/1.1 200 OK"

[Page 1] API 요청 중...
HTTP Request: POST https://www.assembly.go.kr/portal/cnts/cntsCmmit/listMtgRcord.json "HTTP/1.1 200 OK"
10개의 문서 발견
수집 [시작]: 제 22대 제 429회 10차 농림축산식품해양수산위원회
HTTP Request: GET https://record.assembly.go.kr/assembly/viewer/minutes/download/pdf.do?id=55932 "HTTP/1.1 200 OK"
목표 수량 (1개) 달성으로 [종료]
크롤링 [종료] | 총 1개 수집 완료
INFO: 172.217.161.227:44156 - "POST /api/v1/crawlers/test HTTP/1.1" 200 OK
```

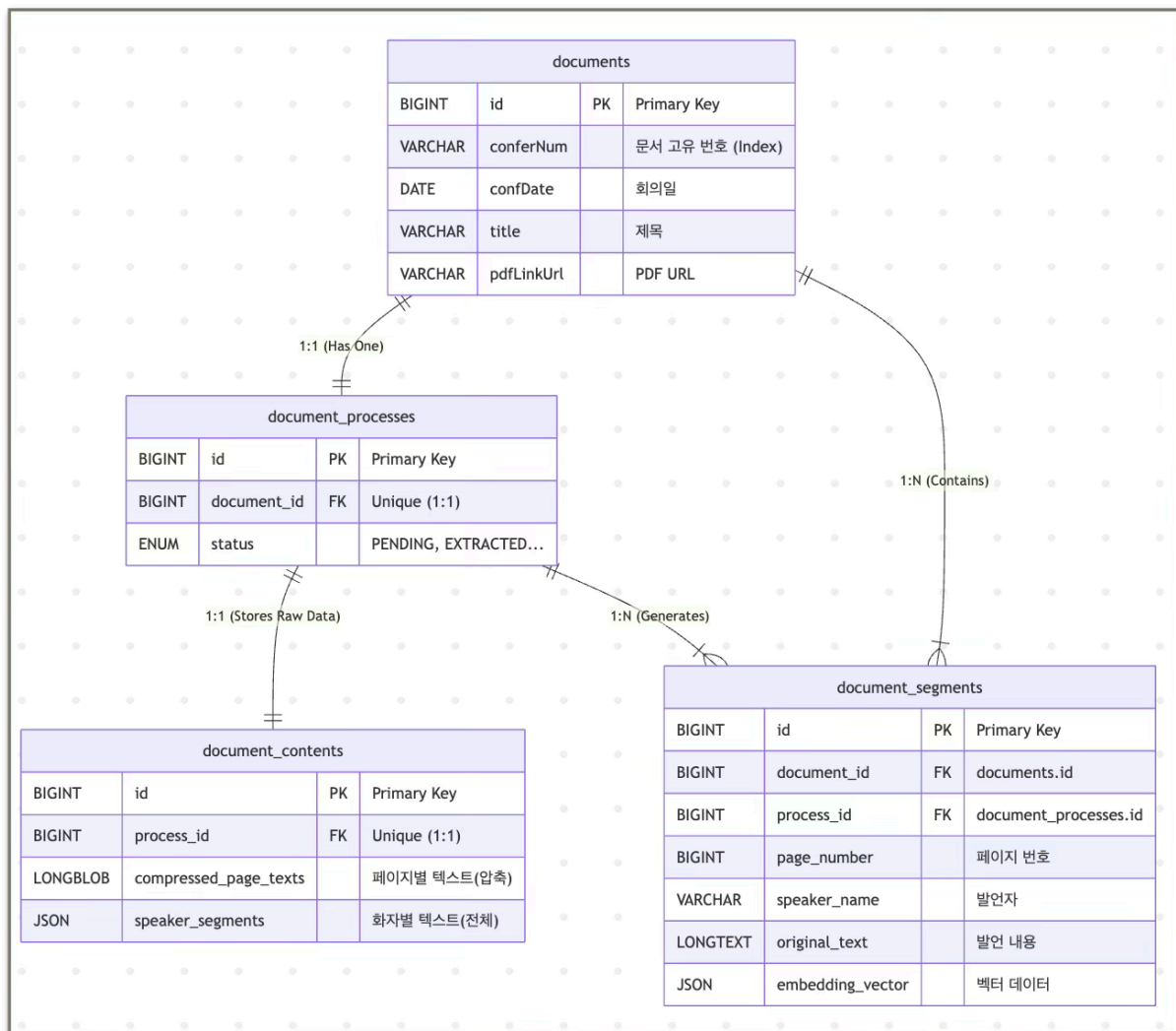
[그림-6]

[5-4] 데이터베이스 구조

국회 회의록 PDF 를 문서 단위로 관리하고, 처리 단계와 발언 세그먼트를 분리하여 저장하는 구조이다. 문서 정보(Document)와 처리 상태(DocumentProcess), 추출 결과(DocumentContent), 발언 단위 데이터(DocumentSegment)가 단계별로 분리된 정규화 구조(1:1, 1:N)로 구성되며, 최종적으로 검색 가능한 세그먼트 데이터가 Elasticsearch 색인의 핵심 데이터가 된다.

[5-4-1] ERD(Entity Relationship Diagram)

데이터베이스 설계는 회의록 PDF 를 문서 기반으로 관리하면서, 이를 처리·분석하고 발언 단위로 분해하기 위한 구조로 다음과 같이 구성되어 있다.



[그림-7]

[5-4-2] 테이블 설명

(1) Document(documents)

회의록 PDF에 대한 기본 정보를 저장하는 중심 엔티티로, 회의 제목, 회의 일자, 위원회명, 원본 링크 등을 포함한다. 각 Document는 반드시 하나의 처리 상태를 가지므로 DocumentProcess와 1:1 관계를 형성하며, 동시에 문서 내부의 여러 발언 단위를 나타내는 DocumentSegment와는 1:N 관계를 가진다.

(2) DocumentProcess(document_processes)

문서의 처리 단계는 DocumentProcess 테이블에서 관리한다. 문서가 현재 어떤 단계 단계에 있는지(텍스트 추출 완료, 파싱 완료, 임베딩 생성 완료 등)를 상태 값으로 저장한다. 처리 단계 정보는 원본 문서와 정확히 한 번 연결되어야 하므로 Document와 1:1 관계를 가진다. 또한 처리 과정에서 생성된 텍스트와 분석 결과는 DocumentContent와 1:1 관계를 이루며, 발언 단위 데이터를 담는 DocumentSegment와는 1:N 관계를 형성한다. 이를 통해 문서 처리 상태, 발화 분리 정보, 최종 가공 결과가 한 프레스 기준으로 관리될 수 있다.

(3) DocumentContent(document_contents)

문서 처리 과정에서 추출된 텍스트와 가공 데이터를 저장하는 역할은 DocumentContent가 담당한다. 이 테이블은 PDF로부터 추출한 페이지별 텍스트, 화자별 병합 텍스트, 압축 데이터 등을 저장하며, 각 데이터는 특정 처리 단계에 종속되므로 DocumentProcess와 1:1 관계로 연결된다. 이로 인해 원본 문서 정보가 변경되더라도, 처리 결과 데이터는 일관성 있게 유지된다.

(4) DocumentSegment(document_segments)

문서의 발언을 세그먼트 단위로 저장하는 테이블이다. 각 발언은 화자명, 역할, 원문 텍스트와 함께 키워드 추출 결과와 임베딩 벡터 등의 분석 정보를 포함한다. 각 세그먼트는 특정 문서(Document)에 속하고, 동시에 문서 처리 단계(DocumentProcess)에 종속되므로 각각 N:1 관계를 가진다. 이를 통해 세그먼트는 문서와 처리 과정을 동시에 참조할 수 있으며, 개별 발언 단위 분석 및 검색이 가능해진다.

[6] 검색 엔진 설계 및 Elasticsearch 이론

[6-1] 빅데이터 처리 기술과 검색 엔진의 관계

Hadoop 은 HDFS(Hadoop Distributed File System)를 기반으로 데이터를 분산 저장하고, MapReduce 와 YARN 을 통해 대규모 병렬 처리를 수행하는 오픈소스 분산 처리 프레임워크이다. 이러한 구조로 인해 Hadoop 은 대용량 데이터의 안정적인 저장과 배치 기반 분석 작업에 강점을 가지는 기술로 평가된다.

그러나 Hadoop 생태계는 대량 데이터 처리에 최적화된 구조를 기반으로 설계되어 있어, 사용자 질의에 대해 즉각적인 응답이 요구되는 실시간 검색 서비스에는 기본적으로 적합하지 않은 한계를 가진다. 특히 MapReduce 기반 처리 방식은 대규모 데이터를 순차적으로 처리하는 배치 작업에 초점을 두고 있어, 검색과 같이 낮은 지연 시간이 요구되는 서비스에는 부적합하다.

Politi-Search 프로젝트는 국회 회의록이라는 대규모 텍스트 데이터를 대상으로 하되, 발언 단위의 의미 기반 검색과 실시간 질의 응답을 핵심 목표로 한다. 이에 따라 전통적인 분산 처리 프레임워크보다는 전문 검색(full-text search)에 특화된 검색 엔진이 필요하였으며, 이러한 요구를 충족하기 위해 Elasticsearch 를 핵심 검색 엔진으로 선택하였다.

[6-2] Lucene 기반 검색 엔진과 Elasticsearch 의 위치

Elasticsearch 는 Apache Lucene 을 내부 검색 엔진으로 사용하는 분산 검색 플랫폼이다. Lucene 은 자바(Java) 기반의 고성능 정보 검색 라이브러리로, 문서를 대상으로 한 역색인(Inverted Index) 구조를 생성하여 매우 빠른 전문 검색을 지원한다.

Lucene 은 단일 서버 환경에서 뛰어난 검색 성능을 제공하지만, 분산 저장, 노드 간 통신, 클러스터와 같은 기능은 제공하지 않는다. Elasticsearch 는 이러한 Lucene 의 격자색 성능을 기반으로 하여, 데이터를 여러 노드에 분산 저장하고 병렬 검색이 가능하도록 확장한 형태의 검색 엔진이다.

즉, Elasticsearch 는 Lucene 의 색인 및 검색 기능을 분산 환경으로 확장한 상위 플랫폼으로 볼 수 있다. Politi-Search 프로젝트는 단일 노드 환경에서 Elasticsearch 를 사용하였으나, 내부적으로는 Lucene 기반의 색인 및 검색 구조를 그대로 활용하므로, 향후 다중 노드 클러스터 환경으로의 확장이 가능한 구조를 유지한다.

[6-3] Elasticsearch 를 선택한 이유

Politi-Search 프로젝트는 국회 회의록이라는 대규모 비정형 텍스트 데이터를 대상으로 하여, 기존의 문서 단위 검색 방식이 가지는 한계를 개선하는 것을 목표로 한다. 회의록 데이터는 단순한 키워드 나열이 아니라 발언의 맥락과 표현의 다양성이 중요한 특성을 가진다.

관계형 데이터베이스(RDBMS)는 무결성과 정형 데이터 관리에는 강점을 가지지만, 대용량 텍스트에 대한 유연한 검색, 형태소 단위 분석, 문맥 기반 탐색에는 구조적인 한계를 가진다. 특히 회의록과 같이 문장이 길고 표현 방식이 다양한 데이터에서는 단순 키워드 매칭 방식으로는 사용자의 검색 의도를 충분히 반영하기 어렵다.

Elasticsearch는 역색인 구조를 기반으로 한 고속 텍스트 검색을 지원하며, 형태소 분석, 가중치 기반 검색, 다양한 조건 조합 기능을 제공한다. 이에 Politi-Search 프로젝트에서는 회의록 데이터를 발언 단위로 세분화하여 Elasticsearch에 색인함으로써, 의미 기반 검색 구조의 타당성과 효율성을 검증하고자 Elasticsearch를 핵심 기술로 선택하였다.

[6-4] Elasticsearch 아키텍처 개요

Elasticsearch는 분산 환경을 전제로 설계된 오픈소스 검색 엔진으로, 대규모 텍스트 데이터를 효율적으로 저장하고 빠른 검색 응답을 제공하기 위한 아키텍처를 가진다. Elasticsearch의 기본 구성 요소는 클러스터(Cluster), 노드(Node), 인덱스(Index), 샤드(Shard), 문서(Document)로 이루어진 계층 구조를 따른다.

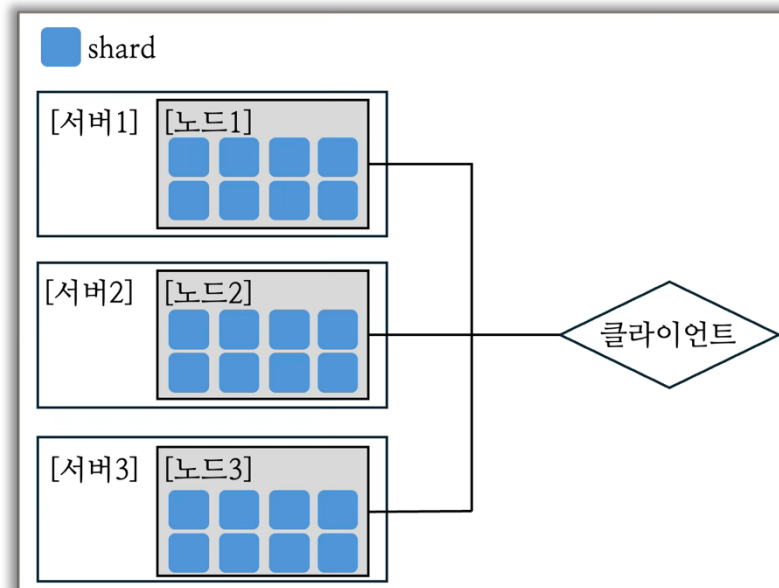
하나의 클러스터는 하나 이상의 노드로 구성되며, 각 노드는 클러스터 상태 관리, 데이터 저장, 검색 요청 처리 등의 역할을 수행한다. 노드는 역할에 따라 마스터 노드(Master Node), 데이터 노드(Data Node), 코디네이팅 노드(Coordinating Node)로 구분될 수 있다. 마스터 노드는 클러스터 메타데이터와 샤드 할당을 관리하며, 데이터 노드는 실제 색인 데이터 저장과 검색 연산을 담당한다. 코디네이팅 노드는 클라이언트 요청을 받아 적절한 노드로 분산 전달하고 결과를 취합하는 역할을 수행한다. 단일 노드 구성에서는 이러한 역할이 하나의 노드에 통합되어 동작한다.

인덱스는 검색 대상이 되는 데이터의 논리적 집합으로, 관계형 데이터베이스의 테이블과 유사한 개념이다. 하나의 인덱스는 내부적으로 여러 개의 샤드(Shard)로 분할되어 저장된다. 샤드는 Elasticsearch에서 데이터 분산과 병렬 처리를 위한 최소 단위이며, 각 샤드는 독립적인 Lucene 인덱스로 구성된다. 이를 통해 검색 요청은 여러 샤드에서 병렬로 처리되어 검색 성능이 향상된다. 또한, 샤드의 복제본인 레플리카(Replica)를 통해 장애 발생 시 데이터 복구와 검색 가용성을 확보할 수 있다.

문서는 인덱스에 저장되는 최소 단위의 데이터로, JSON 형태의 구조화된 데이터를 사용한다. 각 문서는 필드(Field) 단위로 구성되며, 필드별 데이터 타입과 분석 방식은 매핑(Mapping)을 통해 정의된다. 텍스트 필드는 색인 과정에서 분석기(Analyzer)에 의해 토큰화 및 정규화 과정을 거치며, 이 결과는 역색인(Inverted Index)구조로 저장된다. 이러한 구조는 대규모 텍스트 데이터에 대해서도 낮은 지연 시간의 검색을 가능하게 한다.

Politi-Search 프로젝트에서는 개발 및 아키텍처 검증을 목적으로 단일 노드(single-node) 구성을 채택하였다. 이는 클러스터 운영에 따른 복잡성을 최소화하고, 색인 구조 및 검색

동작 원리를 집중적으로 분석하기 위한 선택이다. 단일 노드 환경에서도 Elasticsearch의 핵심 아키텍처 개념은 유지되므로, 향후 다중 노드 기반의 확장 구조로 이전이 가능하다.



[그림-8]

[6-4-1] Elasticsearch 주요 용어 정리

아래는 Elasticsearch 아키텍처 관련 용어에 대한 설명이다.

- Cluster
하나 이상의 노드로 구성된 Elasticsearch의 논리적 집합 단위로, 데이터와 설정 정보를 공유한다.
- Node
Elasticsearch가 실행되는 단일 인스턴스로, 데이터 저장 및 검색 요청 처리를 수행한다. 역할에 따라 마스터 노드, 데이터 노드, 코디네이팅 노드로 구분될 수 있다.
- Index
검색 대상 데이터의 논리적 집합으로, 관계형 데이터베이스의 테이블 대응되는 개념이다.
- Shard
인덱스를 구성하는 물리적 분할 단위로, 데이터 분산 저장과 병렬 검색 처리를 가능하게 한다.
- Replica
프라이머리 샤드의 복제본으로, 장애 복구 및 검색 성능 향상을 위해 사용된다.

- Document

인덱스에 저장되는 최소 단위의 데이터로, JSON 형식의 구조화된 데이터를 사용한다.

- Field

문서를 구성하는 개별 속성으로, 문자열, 숫자, 날짜 등 다양한 데이터 타입을 가질 수 있다.

- Mapping

인덱스 내 필드의 데이터 타입 및 분석 방식을 정의하는 설정 정보이다.

- Analyzer

텍스트 데이터를 색인 및 검색 시 처리하는 구성 요소로, 토큰나이저(Tokenization)와 필터(Filter) 과정을 포함한다.

- Inverted Index(역색인)

단어를 기준으로 해당 단어가 포함된 문서 목록을 저장하는 자료구조로, Elasticsearch의 빠른 검색 성능의 핵심 요소이다.

[6-5] 색인(Indexing)과 역색인(Inverted Index)구조

검색 엔진에서의 색인(Indexing)은 검색 성능을 향상시키기 위해 데이터를 사전에 구조화하여 저장하는 과정이다. 일반적인 관계형 데이터베이스에서는 B-tree나 해시 기반 인덱스를 활용하여 특정 키를 기준으로 데이터를 빠르게 조회한다. 반면, 텍스트 검색은 문서의 전체 내용을 대상으로 질의가 수행되므로, 전문 검색(full-text search)에 적합한 색인 구조가 요구된다.

Elasticsearch는 이러한 요구를 충족하기 위해 역색인(Inverted Index) 구조를 사용한다. 역색인은 기존의 “문서→단어” 중심 구조가 아닌 “단어→ 해당 단어가 포함된 문서 목록”의 형태로 색인을 구성한다. 이를 통해 검색어가 포함된 문서를 즉시 식별할 수 있으며, 모든 문서를 순차적으로 탐색하는 방식에 비해 검색 속도가 크게 향상된다.

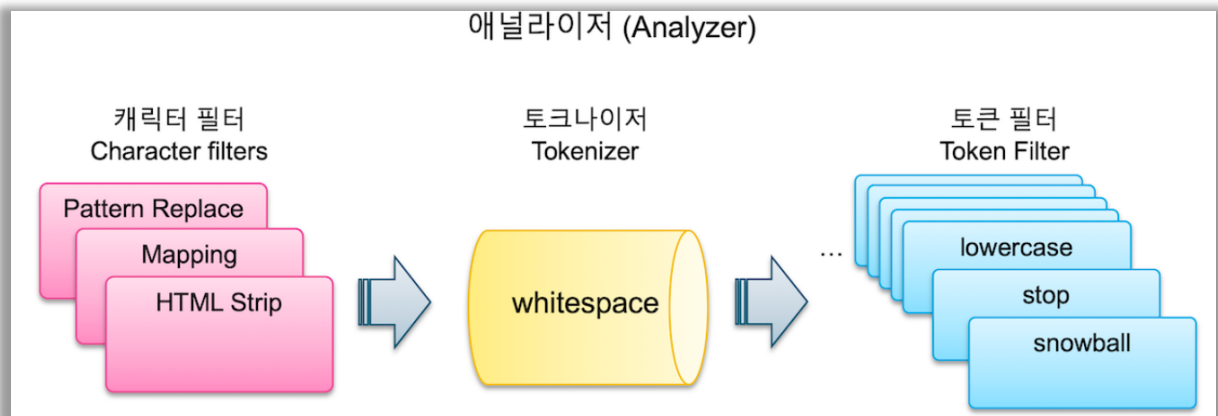
색인 과정에서 Elasticsearch는 문서의 텍스트 필드를 분석기(Analyzer)를 통해 토큰화(Tokenization)하고 정규화한 뒤, 각 토큰을 기준으로 역색인을 생성한다. 이 과정은 Apache Lucene 엔진을 기반으로 수행되며, 대규모 텍스트 데이터에 대해서도 효율적인 검색을 가능하게 한다.

Politi-Search 프로젝트에서는 국회 회의록을 발언 단위의 문서로 분리하여 색인함으로써, 회의록 전체 문서가 아닌 개별 발언 수준에서의 정밀한 검색이 가능하도록 설계하였다. 이를 통해 특정 인물의 발언, 특정 주제에 대한 발언 맥락 등을 보다 정확하게 검색할 수 있다.

[6-6] 전문 검색과 텍스트 분석 과정

전문 검색(full-text search)은 문서의 특정 필드나 키 값이 아닌, 문서 전체의 텍스트 내용을 분석하고 색인하여 검색하는 방식이다. Elasticsearch는 전문 검색을 지원하기 위해 색인 및 검색 과정에서 텍스트 분석(Analysis) 과정을 수행하며, 이 과정은 Analyzer를 중심으로 구성된다.

Analyzer는 텍스트를 검색 가능한 토큰(token)으로 변환하기 위한 처리 파이프라인으로, Character Filter, Tokenizer, Token Filter의 세 단계로 구성된다.



[그림-9]

Character Filter, Tokenizer, Token Filter의 세 단계로 구성된다. Character Filter 단계에서는 HTML 태그 제거, 특수 문자 치환, 불필요한 기호 제거와 같은 전처리 작업이 수행된다. 이후 Tokenizer는 전처리된 문자열을 일정 규칙에 따라 토큰 단위로 분리한다. 마지막으로 Token Filter 단계에서는 소문자 변환, 불용어(stopword) 제거, 형태소 분석 등의 처리가 적용되어 검색에 적합한 형태로 토큰이 정제된다.

Elasticsearch는 이러한 분석 과정을 통해 텍스트를 단순 문자열이 아닌 의미 단위로 해석하고, 이를 기반으로 역색인을 생성한다. 예를 들어, 국회 회의록의 한 발언 문장은 분석 과정을 거쳐 핵심 의미를 가지는 단어 단위로 분해되며, 이 결과를 바탕으로 검색 시 빠른 조회가 가능해진다. 이 과정은 색인 시점과 검색 시점에 동일한 Analyzer를 적용함으로써, 검색어와 저장된 데이터 간의 일관성을 유지한다.

한국어 텍스트는 조사와 어미 변화가 빈번하고 단어 경계가 명확하지 않은 특성을 가지므로, 단순 문자열 기반의 검색만으로는 의미 단위 검색에 한계가 있다. 이에 Politi-Search 프로젝트에서는 형태소 분석을 고려한 분석 방식을 적용하여, 단어의 표면형이 아닌 의미 단위에 기반한 검색이 가능하도록 설계하였다. 이러한 텍스트 분석 전략은 국회 회의록과 같이 문맥 의존성이 높은 텍스트 데이터의 검색 정확도를 향상시키는 핵심 요소이다.

아래 표는 Elasticsearch의 Analyzer 파이프라인에 따라 국회 회의록 문장이 분석되고 역색인으로 저장되는 과정을 단계별로 예시로 나타낸 것이다.

단계	처리 내용	예시 입력	처리 결과
원문 텍스트	분석 이전 원본 텍스트	국회에서 법안을 심의 하였다	국회에서 법안을 심의 하였다
Character Filter	HTML 태그 제거 및 문자 전처리	국회에서 법안을 심의 하였다	국회에서 법안을 심의 하였다
Tokenizer	텍스트를 토큰 단위로 분리	국회에서 법안을 심의 하였다	국회에서, 법안을, 심의하였다
Token Filter	불용어 제거 및 형태소 분석	국회에서, 법안을, 심의하였다	국회, 법안, 심의
역색인 생성	토큰 기준 도큐먼트 매핑	국회, 법안, 심의	각 토큰 → 해당 발언 도큐먼트 ID

[6-7] 검색 질의(Query)와 관련도 점수(Scoring)

검색은 색인된 인덱스 내부에서 사용자가 입력한 질의(Query)에 해당하는 도큐먼트를 찾아가는 과정이다. Elasticsearch는 JSON 기반의 Query DSL(Domain Specific Language)을 제공하며, 이를 통해 다양한 검색 조건과 복합 질의를 조합할 수 있다.

Elasticsearch에서 검색 결과는 단순히 질의 조건을 만족하는 도큐먼트를 나열하는 것이 아니라, 각 도큐먼트의 관련성 점수(score)를 계산하여 높은 점수 순으로 정렬된다. 이 관련도 점수는 Lucene 검색 엔진이 내부적으로 실행하는 스코어링 알고리즘에 의해 산출되며, 기본적으로 BM25(Best Matching 25)모델이 사용된다.

BM25 알고리즘은 다음과 같은 요소를 고려하여 문서와 질의어 간의 관련도를 수치화한다.

- 단어 빈도(Term Frequency, TF): 질의어가 문서 내에 많이 등장할수록 해당 문서의 점수는 증가한다.
- 역문서 빈도(Inverse Document Frequency, IDF): 전체 문서 집합에서 특정 단어가 희귀할수록 해당 단어에 대한 가중치는 커진다.
- 문서 길이 보정: 긴 문서는 동일 단어를 포함할 가능성이 높으므로, 문서 길이를 보정하여 점수를 정규화한다.

이러한 스코어링 모델은 단순 일치 여부가 아니라 질의와 문서의 의미적 관련성을 수치화하여, 사용자에게 더 적합한 결과를 상위에 노출할 수 있도록 한다.

Politi-Search 프로젝트에서는 이러한 스코어링 개념을 활용하여, 발언 텍스트의 내용과 더불어 발언자, 시점, 회의 정보 등 여러 메타데이터를 결합한 구조를 설계하였다. 이를 통해 단순 키워드 일치만이 아니라, 사용자의 검색 의도에 부합하는 검색 결과를 제공하고자 하였다.

[6-8] Elasticsearch 기반 검색 시스템 구현

Politi-Search 프로젝트의 검색 기능은 Elasticsearch 를 기반으로 구현되었으며, FastAPI 백엔드 서버 및 관계형 데이터베이스에 저장된 국회 회의록 데이터를 검색 엔진과 연동하여 활용하는 구조로 설계하였다.

검색 시스템 구축 과정은 연결 설정 → 인덱스 정의 → 데이터 색인 → 검색 서비스 구현 → API 연동의 총 5 단계로 구성된다.

[6-8-1] 환경 설정 및 연결(connection.py)

검색 시스템 구축의 첫 단계로 Elasticsearch 클러스터와의 안정적인 연결을 설정하였다. Elasticsearch Python 공식 클라이언트를 활용하여 싱글톤(Singleton) 방식의 연결 객체를 생성함으로써, 애플리케이션 전반에서 일관된 검색 서버 접근이 가능하도록 하였다. 이를 통해 불필요한 연결 재생성을 방지하고, 검색 요청 처리 시 성능 저하를 최소화하였다.

[6-8-2] 인덱스 생성 및 매핑 정의(index_manager.py)

검색 품질을 좌우하는 핵심 요소는 인덱스 구조이다. Politi-Search 프로젝트에서는 국회 회의록이라는 한국어 장문 텍스트 특성을 고려하여 Nori 형태소 분석기 기반 의미 검색을 동시에 지원하는 인덱스 매핑 구조를 설계하였다.

회의록 전체 문서를 단일 검색 단위로 처리하지 않고, 발언(segment) 단위로 분해하여 색인함으로써 검색 정밀도를 향상시켰다. 각 인덱스 문서는 발언 본문 텍스트, 발언자 정보, 위원회명, 회의 일자 등의 메타데이터와 함께, 의미 기반 검색을 위한 dense_vector 필드를 포함한다. 해당 매핑을 기반으로 minutes_v1 인덱스를 생성하여 회의록 데이터에 최적화된 검색 스키마를 구축하였다.

[6-8-3] 데이터 색인 과정(indexer.py)

인덱스가 생성된 이후에는 데이터베이스에 저장된 회의록 데이터를 Elasticsearch 로 색인한다. 이 과정에서는 Document 및 DocumentSegment 정보를 조회하여, 회의 메타데이터와 발언자 단위 세그먼트를 하나의 Elasticsearch 문서 형태로 변환한다.

변환된 데이터는 Elasticsearch 의 helpers.bulk 기능을 활용하여 대량 색인 방식으로 처리하였으며, 이를 통해 다수의 발언 데이터가 효율적으로 검색 엔진에 반영되도록 구성하였다. 해당 과정은 색인 성능을 확보함과 동시에 대규모 데이터 처리 시 안정성을 보장한다.

[6-8-4] 검색 서비스 구현(search_service.py / query_builder.py)

검색 기능은 키워드 기반 검색(BM25)과 벡터 기반 의미 검색을 결합한 하이브리드 검색 방식으로 구현하였다. 사용자가 입력한 검색어는 형태소 분석을 거친 후, Bool Query 구조를 통해 텍스트 일치도 기반 검색과 벡터 유사도 기반 검색이 함께 수행된다.

이를 통해 사용자가 “과기부”와 같은 단어를 검색할 경우, 단순 문자열 일치에 그치지 않고 의미적 연관성이 높은 회의록 발언 세그먼트가 함께 반환되도록 설계하였다. 이러한 구조는 표현 방식이나 단어 형태 차이로 인한 검색 누락을 줄이는 데 효과적이다.

[6-8-5] API 연동(search.py)

검색 기능은 FastAPI 라우터를 통해 /search 엔드포인트로 제공된다. 프론트엔드는 해당 API를 호출하여 검색 결과를 수신하며, 반환된 발언 단위 검색 결과는 사용자 인터페이스에 즉시 반영된다. 이를 통해 검색 시스템은 외부 서비스와 유기적으로 연동되며, 전체 서비스 흐름 내에서 핵심 기능으로 작동한다.

[6-9] 검색 결과 해석

Elasticsearch는 자연어 기반 검색을 위해 BM25(Best Match25) 알고리즘을 활용하여 검색어와 문서 간의 연관성을 계산한다. 현재 시스템에 색인된 전체 데이터는 총 17,345 건이며, 이는 50 건의 국회 회의록으로부터 추출된 발언, 질의, 답변 등의 세부 발언 단위 데이터로 구성된다.

```
{
  "count": 17345,
  "_shards": {
    "total": 1,
    "successful": 1,
    "skipped": 0,
    "failed": 0
  }
}
```

[그림-10]

[6-9-1] 검색 정확도(Relevancy)

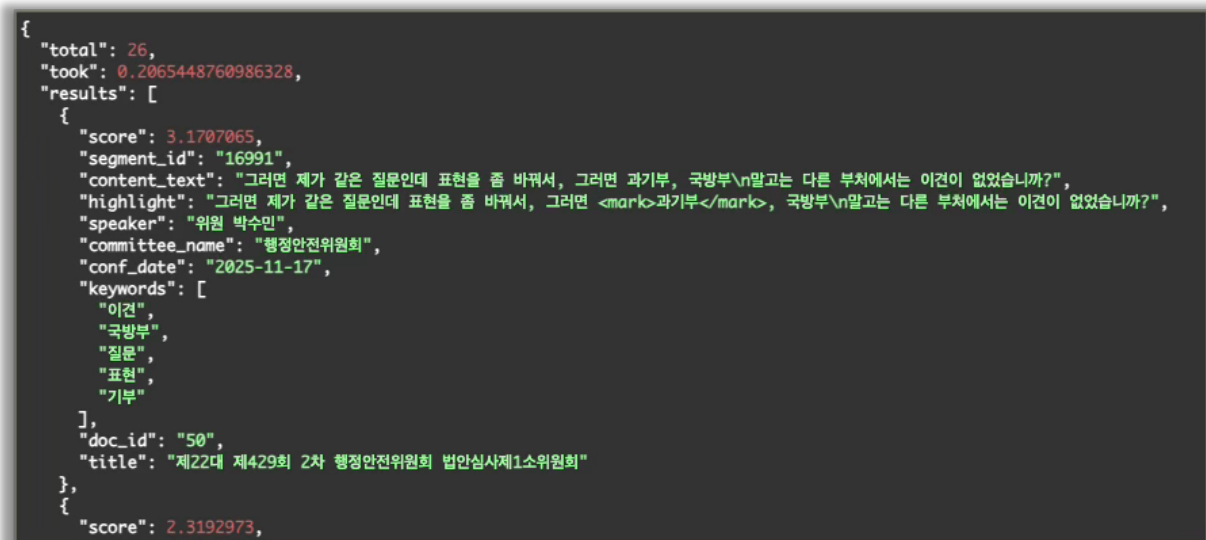
Full-text 검색 엔진에서의 검색 정확도는 사용자가 입력한 검색어와 문서 간의 관련도를 수치화하여 평가된다. Elasticsearch는 이를 기반으로 검색 결과의 순위를 결정하며, 가장 높은 연관성을 가진 문서 상단에 노출되도록 설계되어 있다.

BM25 알고리즘은 검색어의 등장 빈도, 문서 길이, 전체 문서 집합 내에서의 희소성 등을 종합적으로 고려하여 점수를 산출한다. 이는 절대적인 성능 지표라기보다, 동일 검색 결과 내에서의 상대적 비교 지표로 활용된다.

[6-9-2] BM25 기반 검색 결과 분석

예를 들어, 검색 키워드 “과기부”에 대한 조회를 수행한 결과, 해당 키워드와 통계적으로 연관된 문서는 총 26건으로 나타났다. 이 중 가장 높은 BM25 점수를 기록한 문서는 다음 그림과 같이 3.1707 점의 유사도 점수를 보였다. 이는 색인된 전체 데이터 중 “과기부”라는 검색어와 가장 높은 통계적 연관성을 가진 발언임을 의미한다.

그러나 BM25 점수가 높다고 해서 해당 결과가 항상 사용자가 기대하는 의미적 검색 결과를 보장하는 것은 아니다. BM25는 키워드 출현 빈도에 기반한 통계적 모델이므로, 발언의 실제 맥락이나 의미 유사성까지 완전히 반영하는 데에는 한계가 존재한다.



```
{
  "total": 26,
  "took": 0.2065448760986328,
  "results": [
    {
      "score": 3.1707065,
      "segment_id": "16991",
      "content_text": "그러면 제가 같은 질문인데 표현을 좀 바꿔서, 그러면 과기부, 국방부\말고는 다른 부처에서는 이견이 없었습니까?",
      "highlight": "그러면 제가 같은 질문인데 표현을 좀 바꿔서, 그러면 <mark>과기부</mark>, 국방부\말고는 다른 부처에서는 이견이 없었습니까?",
      "speaker": "위원 박수민",
      "committee_name": "행정안전위원회",
      "conf_date": "2025-11-17",
      "keywords": [
        "이견",
        "국방부",
        "질문",
        "표현",
        "기부"
      ],
      "doc_id": "50",
      "title": "제22대 제429회 2차 행정안전위원회 법안심사제1소위원회"
    },
    {
      "score": 2.3192973,
      "segment_id": "16992",
      "content_text": "그러면 제가 같은 질문인데 표현을 좀 바꿔서, 그러면 과기부, 국방부\말고는 다른 부처에서는 이견이 없었습니까?",
      "highlight": "그러면 제가 같은 질문인데 표현을 좀 바꿔서, 그러면 <mark>과기부</mark>, 국방부\말고는 다른 부처에서는 이견이 없었습니까?",
      "speaker": "위원 박수민",
      "committee_name": "행정안전위원회",
      "conf_date": "2025-11-17",
      "keywords": [
        "이견",
        "국방부",
        "질문",
        "표현",
        "기부"
      ],
      "doc_id": "50",
      "title": "제22대 제429회 2차 행정안전위원회 법안심사제1소위원회"
    }
  ]
}
```

[그림-11]

[6-9-3] 검색 품질 개선 방향

현재 검색 결과는 연관도를 수치화하여 제공하고 있으나, 의미 기반 검색 품질을 충분히 확보하기 위해서는 추가적인 개선이 필요하다. 특히 국회 회의록 데이터는 정치·행정 분야의 전문 용어로, 질의·답변 구조, 구어체 문장 특성이 혼재되어 있어 일반적인 키워드 추출 방식만으로는 한계가 있다.

이에 따라 향후에는 도메인 적합성이 높은 형태소 분석기(Kiwi)와 KeyBERT 기반 키워드 추출 모델의 파라미터 튜닝을 통해 발언의 핵심 주제를 보다 정확히 식별할 필요가 있다. 또한 검색 품질 향상이 제한적일 경우, 발언 단위 세분화, OCR 품질 저하 구간 제거, 문장 의미 기반 색인 구조로의 전환 등 색인 파이프라인 전반에 대한 재설계가 요구된다.

결론적으로 검색 엔진의 성능 향상은 알고리즘 선택 이전에 데이터 품질과 키워드 추출 정확도에 의해 크게 좌우되며, 도메인 특화 전처리 및 색인 구조의 정교화가 검색 품질 개선의 핵심 요소임을 확인하였다.

[7] 데이터 파이프라인 및 인공지능(AI) 설계

Politi-Search 프로젝트는 국회 회의록이라는 비정형 PDF 문서를 대상으로, 검색과 분석에 적합한 구조화 데이터로 변환하고, 의미 기반 검색을 가능하게 하는 데이터 파이프라인과 AI 처리 구조를 설계하였다. 기존 국회 회의록 시스템은 문서 단위 검색 또는 단순 키워드 기반 색인에 의존하여, 발언 맥락이나 정책 주제 흐름을 정밀하게 탐색하는 데 한계가 있었다. 이에 Politi-Search 프로젝트는 발언 단위 세분화, 형태소 기반 키워드 추출, 의미 임베딩 기반 검색을 결합한 파이프라인 구조를 통해 이러한 한계를 극복하고자 하였다.

특히 회의록 데이터가 가지는 형식적·언어적 특수성을 고려하여, 단순 텍스트 처리나 통계 기반 접근이 아닌 형태소 분석과 문장 임베딩을 결합한 의미 중심 처리 구조를 구성하였다. 이를 통해 사용자는 특정 키워드뿐 아니라 정책 주제와 발언 맥락에 기반한 탐색이 가능하도록 시스템을 설계하였다.

[7-1] 데이터 파이프라인

데이터 파이프라인은 PDF 원문 수집부터 검색 인덱싱에 이르기까지 총 4 단계로 구성되며, PDF 텍스트 추출 → 발언 단위 세그먼트 분리 → 키워드 추출 → 문장 임베딩 생성의 4 단계로 구성된다.

각 단계의 결과는 데이터베이스에 저장되고 문서 처리 상태는 단계별로 관리된다. 이를 통해 대용량 회의록 데이터에 대해서도 안정적인 비동기 처리와 재처리가 가능하도록 설계하였다.

[7-1-1] PDF 텍스트 추출(PdfExtraction)

크롤링을 통해 수집된 국회 회의록 PDF 파일을 대상으로, pdfplumber 라이브러리를 활용하여 페이지 단위 텍스트 추출을 수행하였다. PDF 파일은 본질적으로 시각적 레이아웃 기반 문서 포맷이기 때문에, 단순 텍스트 추출 과정에서는 발언 본문과 구조적 메타데이터가 동일한 텍스트로 인식되는 한계가 존재한다. 추출된 텍스트는 페이지 번호 정보를 함께 포함한 JSON 형태로 변환되어 DocumentContent 테이블에 저장되며, 해당 단계가 완료되면 문서의 처리 상태는 EXTRACTED 로 갱신된다.

그러나 국회 회의록 PDF의 경우, 각 페이지 상단에는 쪽수, 회차 정보, 회의명, 개최 일자 등 발언 내용과 직접적인 관련이 없는 머리말(Header) 정보가 반복적으로 삽입되어 있다. 이러한 머리말은 페이지의 홀수·짝수 여부에 따라 위치와 형식이 달라지며, 예를 들어

“6 제 429 회-농림축산식품해양수산제 10 차(2025 년 12 월 1 일)” 또는
“제 429 회-농림축산식품해양수산제 10 차(2025 년 12 월 1 일) 5”

와 같은 다양한 형태로 나타난다.

[그림-12]

6 제429회-농림축산식품해양수산제10차(2025년12월1일)

법안 의결에 앞서 지난 금요일 11월 28일 자로 우리 위원회에 국민의힘 강명구 위원이 사임하시고 박준태 위원이 보임되었음을 알려 드립니다.

[그림-13]

문제는 이러한 머리말 정보가 텍스트 추출 단계에서는 일반 발언 문장과 동일하게 처리된다는 점이다. 이로 인해 페이지 전한 시 머리말 텍스트가 이전 또는 이후 발언과 하나의 연속된 문장으로 변합되는 현상이 발생하였으며, 결과적으로는 발언 단위 분리 과정에서 화자 발언 내용이 왜곡되거나 불필요한 텍스트가 포함되는 문제가 확인되었다. 특히 이러한 오류는 발언 단위 세그먼트의 의미적 정확도를 저하시켜, 이후 검색 인덱싱 및 검색 품질에도 부정적인 영향을 미칠 수 있다.

이를 해결하기 위해 머리말(Header) 전용 필터링 로직을 설계하여 PDF 텍스트 전처리 단계에 적용하였다. 회차 정보, 회의명, 개최 일자, 페이지 번호를 포함하는 머리말 패턴을 정규표현식으로 정의하고, 각 페이지의 텍스트를 라인 단위로 처리하는 과정에서 해당 패턴에 일치하는 라인은 즉시 제거하도록 구성하였다. 이 Header Filter는 텍스트 파싱 로직의 최우선 단계에서 적용되어, 머리말 정보가 발언 텍스트 버퍼에 포함되지 않도록 차단한다.

그 결과, 페이지 경계에서 발생하던 발언 병합 문제가 실질적으로 제거되었으며, 발언 단위 분리 정확도가 크게 향상되었다. 이는 이후 키워드 추출 및 임베딩 생성 단계에서 의미 왜곡을 최소화하여, 검색 결과의 정밀도와 문맥 일관성을 동시에 향상시키는 기반이 되었다.

```
[API 요청] PDF 추출 시작 (Limit: 1)
[1단계] PDF 추출 시작 (Limit: 1)
[Start] Document ID 51: /app/static/pdfs/2025-12-01_55932_제 22대 제 429회 10차 농림축산식품해양수산위원회.pdf
제 429회-농림축산식품해양수산제 10차 (2025년 12월 1일) 1
농림축산식품해양수산위원회회의록
제 429회국회 제 10 회
(정기회)
(임시회의록)
국회사무처
일시 2025년 12월 1일 (월)
장소 농림축산식품해양수산위원회회의실
의사일정
1. 소위원회 위원 개선의 건
2. 농지법 일부개정법률안(정희용 의원 대표발의)(의안번호 2209497)
3. 농지법 일부개정법률안(문대림 의원 대표발의)(의안번호 2201938)
4. 농지법 일부개정법률안(이병진 의원 대표발의)(의안번호 2201145)
5. 농지법 일부개정법률안(윤준병 의원 대표발의)(의안번호 2202465)
6. 농지법 일부개정법률안(대안)
7. 농어촌 빈집정비에 관한 특별법안(이만희 의원 대표발의)(의안번호 2208561)
8. 농어촌 빈집 정비 및 활용에 관한 특별법안(윤준병 의원 대표발의)(의안번호 2209124)
9. 농어촌정비법 일부개정법률안(윤준병 의원 대표발의)(의안번호 2209128)
10. 농어촌 빈집정비 촉진을 위한 특별법안(정희용 의원 대표발의)(의안번호 2209765)
```

[그림-14]

[7-1-2] 발화자 단위 세그먼트 분리(PdfTranscriptParser)

발언 단위 세그먼트는 국회 회의록의 의미 구조를 가장 자연스럽게 반영하는 최소 단위로, 화자별 정책 입자야 분석, 질의·응답 흐름 추적, 의미 기반 검색 정밀도를 향상시키는 핵심 요소이다. 이에 따라 Politi-Search 프로젝트에서는 페이지 단위로 추출된 텍스트를 화자 단위 발언 세그먼트로 재구성하였다.

국회 회의록의 발언은 일반적으로 “○ 직책 이름” 형태로 시작되므로, 이를 기반으로 화자 인식 정규식을 설계하였다. 동일 화자의 발언이 페이지를 넘어 이어질 경우 하나의 세그먼트로 병합하여 발언 단위의 의미적 연속성을 유지하도록 하였다.

그러나 실제 회의록에서는 안건 제목이나 의사일정 목록이 화자의 발언 종료 이후 바로 이어지는 경우가 많다. 해당 문구가 이전 화자의 발언으로 잘못 인식되는 문제가 발생하였다.

1. 소위원회 위원 개선의 건

(14시12분)

○위원장 어기국 그러면 먼저 의사일정 제1항 소위원회 위원 개선의 건을 상정합니다.

이 안건은 위원 사보임에 따라 농림축산식품법안심사소위원회와 예산결산심사소위원회에 국민의힘 강명구 위원을 국민의힘 박준태 위원으로 각각 개선하고자 하는데 이의 없으십니까?

(「예」 하는 위원 있음)

가결되었음을 선포합니다.

2. 농지법 일부개정법률안(정희용 의원 대표발의)(의안번호 2209497)

3. 농지법 일부개정법률안(문대림 의원 대표발의)(의안번호 2201938)

4. 농지법 일부개정법률안(이병진 의원 대표발의)(의안번호 2201145)

5. 농지법 일부개정법률안(윤준병 의원 대표발의)(의안번호 2202465)

6. 농지법 일부개정법률안(대안)

7. 농어촌 빈집정비에 관한 특별법안(이만희 의원 대표발의)(의안번호 2208561)

8. 농어촌 빈집 정비 및 활용에 관한 특별법안(윤준병 의원 대표발의)(의안번호 2209124)

9. 농어촌정비법 일부개정법률안(윤준병 의원 대표발의)(의안번호 2209128)

10. 농어촌 빈집정비 촉진을 위한 특별법안(정희용 의원 대표발의)(의안번호 2209765)

11. 농어촌정비법 일부개정법률안(곽규택 의원 대표발의)(의안번호 2210387)

12. 농어촌정비법 일부개정법률안(서천호 의원 대표발의)(의안번호 2211302)

13. 농어촌정비법 일부개정법률안(김선교 의원 대표발의)(의안번호 2211508)

14. 농어촌 빈집정비 특별법안(서삼석 의원 대표발의)(의안번호 2212742)

15. 농어촌 빈집 정비 및 관리에 관한 특별법안(대안)

16. 농어촌구조개선 특별회계법 일부개정법률안(윤준병 의원 대표발의)(의안번호 2209126)

17. 자유무역협정 체결에 따른 농어업인 등의 지원에 관한 특별법 일부개정법률안(윤준병 의원 대표발의)(의안번호 2209127)

18. 농수산물 유통 및 가격안정에 관한 법률 일부개정법률안(박덕흠 의원 대표발의)(의안번호 2206171)

19. 농수산물 유통 및 가격안정에 관한 법률 일부개정법률안(김선교 의원 대표발의)(의안번호 2209127)

[그림-15]

이를 해결하기 위해 국회 회의록 특유의 안건 나열 형식(숫자 + 안건명 + 의안번호)을 정규식으로 정의하고, 해당 패턴이 탐지될 경우 현재 화자를 명시적으로 해제하는 로직을 추가하였다. 이를 통해 발언 텍스트와 회의 진행 정보가 명확히 분리되었으며, 발언 단위 데이터의 구조적 정확도가 크게 향상되었다.

또한 발언 중간에 삽입되는 집단 응답 표현(예: “(「예」 하는 위원 있음)”)은 실제 발언 의미에 기여하지 않는 노이즈로 판단하여, 별도의 패턴(COLLECTIVE_RE)으로 탐지 후 제거하였다. 이 과정은 화자를 유지한 채 불필요한 텍스트 정제함으로써, 발언 세그먼트의 의미적 일관성과 검색 적합도를 동시에 개선하는 효과를 가져왔다.

```
INFO:      172.217.161.227:40859 - "POST /api/v1/pdf/run-extraction?limit=1 HTTP/1.1" 200 OK
[API 요청] 세그먼트 파싱 시작 (Limit: 1)
[2단계] 파싱 및 세그먼트 분리 시작 (Limit: 1)
[Start] Process ID 51 세그먼트이션 시작 (총 21페이지)
[성공] Process ID 51 파싱 완료 (매칭: 0건)
[성공] Process ID 51 파싱 완료 (매칭: 0건)
[성공] Process ID 51 파싱 완료 (매칭: 0건)
[성공] Process ID 51 파싱 완료 (매칭: 0건)
MATCH: 어기구(위원장) -> 의석을 정돈해 주시기 바랍니다....
[성공] Process ID 51 파싱 완료 (매칭: 1건)
MATCH: 어기구(위원장) -> 그러면 먼저 의사일정 제1항 소위원회...
[성공] Process ID 51 파싱 완료 (매칭: 2건)
[성공] Process ID 51 파싱 완료 (매칭: 2건)
MATCH: 어기구(위원장) -> 다음은 의사일정 제2항부터 제59항까...
[성공] Process ID 51 파싱 완료 (매칭: 4건)
[성공] Process ID 51 파싱 완료 (매칭: 6건)
[성공] Process ID 51 파싱 완료 (매칭: 6건)
[성공] Process ID 51 파싱 완료 (매칭: 11건)
[성공] Process ID 51 파싱 완료 (매칭: 11건)
[성공] Process ID 51 파싱 완료 (매칭: 11건)
[성공] Process ID 51 파싱 완료 (매칭: 11건)
[성공] Process ID 51 파싱 완료 (매칭: 13건)
[성공] Process ID 51 파싱 완료 (매칭: 15건)
[성공] Process ID 51 파싱 완료 (매칭: 16건)
[성공] Process ID 51 파싱 완료 (매칭: 16건)
[성공] Process ID 51 파싱 완료 (매칭: 17건)
[성공] Process ID 51 파싱 완료 (매칭: 19건)
[성공] Process ID 51 파싱 완료 (매칭: 19건)
총 21건 처리 완료 및 커밋됨.
INFO:      172.217.161.227:34800 - "POST /api/v1/pdf/run-parser?limit=1 HTTP/1.1" 200 OK
```

[그림-16]

[7-1-3] 키워드 추출

세그먼트 단위로 분리된 발언은 키워드 추출 단계로 전달된다. 국회 회의록은 의례적 표현과 절차적 발언이 고빈도로 등장하는 구어체 데이터이기 때문에, 단순 TF-IDF 기반 키워드 추출은 ‘위원장’, ‘발언’, ‘질의’와 같은 형식적인 단어를 과대 평가하는 한계를 가진다.

이를 해결하기 위해 Kiwi 형태소 분석기를 활용하여 조사, 어미, 접속 표현 등 의미 기여도가 낮은 요소를 제거하고 명사 중심의 후보 키워드를 생성하였다. Kiwi는 한국어의 복합 명사 구조를 비교적 안정적으로 유지할 수 있어, 법안명이나 정책 용어의 의미 훼손을 최소화하는 데 적합하다.

이후 KeyBERT 모델을 적용하여 세그먼트 텍스트와 후보 키워드 간의 의미적 유사도를 계산함으로써 핵심 키워드를 산출하였다. 이 과정에서는 MMR(Maximal Marginal Relevance)을 적용하여, 문서 전체 의미와의 유사도는 유지하면서 이미 선택된 키워드와의 중복성은 최소화하였다. 이를 통해 특정 주제에 편중되지 않고 회의 발언의 주요 정책 의제를 균형 있게 반영하는 키워드 세트가 생성되었다.

키워드 추출 과정은 CPU 연산량이 크기 때문에 별도의 비동기 스레드에서 수행되며, 결과는 DocumentSegment 테이블에 저장되어 이후 검색 인덱싱 및 분석 단계에서 활용된다.

```
INFO: 172.217.161.227:34800 - "POST /api/v1/pdf/run-parser?limit=1 HTTP/1.1" 200 OK
[API 요청] 키워드 추출 시작 (Limit: 1)
[3-a단계] 키워드 추출 시작 (Limit: 1)
----- [Track A] 키워드 추출 결과 -----
[DocumentSegmentSaveSchema(document_id=51, process_id=51, page_number=5, speaker_name='어기구', speaker_role='위원장', original_text='
의석을 정돈해 주시기 바랍니다.\n성원이 되었으므로 제 429회 국회(정기회) 제 10차 농림축산식품해양수산위원회를 개최\n하겠습니다.\n보고사항
은 배부해 드린 유인물을 참고해 주시기 바랍니다.\n(보고사항은 끝에 실음)\n오늘 회의에서는 그동안 농림축산식품법안심사소위원회 및 해양수
산법안심사소위원회\에서 심사 완료한 법안 등을 의결하도록 하겠습니다.', keywords=['법안', '식품', '수산', '의석', '정기회']), DocumentS
egmentSaveSchema(document_id=51, process_id=51, page_number=6, speaker_name='어기구', speaker_role='위원장', original_text='6 제 429회-
농림축산식품해양수산제 10차 (2025년 12월 1일)\n법안 의결에 앞서 지난 금요일 11월 28일자로 우리 위원회에 국민의힘 강명구 위원님이\n사임하시고
박준태 위원님이 보임되었음을 알려드립니다.\n1. 소위원회 위원 개선의 건\n(14시 12분)', keywords=['박준태', '사임', '법안', '수산', '강명
구']), DocumentSegmentSaveSchema(document_id=51, process_id=51, page_number=6, speaker_name='어기구', speaker_role='위원장', original_t
ext='그러면 먼저 의사일정 제1항 소위원회 위원 개선의 건을 상정합니다.\n이 안건은 위원 사임에 따라 농림축산식품법안심사소위원회와 예산
결산심사소위원회\에 국민의힘 강명구위원을 국민의힘 박준태 위원으로 각각 개선하고자 하는데 이의 없\n습니까?\n(「예」 하는 위원 있음)
\n가결되었음을 선포합니다.\n2. 농지법 일부개정법률안(정희용 의원 대표발의)(의안번호 2209497)\n3. 농지법 일부개정법률안(문대림 의원 대표
발의)(의안번호 2201938)\n4. 농지법 일부개정법률안(이병진 의원 대표발의)(의안번호 2201145)\n5. 농지법 일부개정법률안(윤준병 의원 대표발
의)(의안번호 2202465)\n6. 농지법 일부개정법률안(대안)\n7. 농어촌 빈집정비에 관한 특별법안(이만희 의원 대표발의)(의안번호 2208561)\n8.
농어촌 빈집 정비 및 활용에 관한 특별법안(윤준병 의원 대표발의)(의안번호 2209124)\n9. 농어촌정비법 일부개정법률안(윤준병 의원 대표발의)(
의안번호 2209128)\n10. 농어촌 빈집정비 추진을 위한 특별법안(정희용 의원 대표발의)(의안번호 2209765)\n11. 농어촌정비법 일부개정법률안(곽
규택 의원 대표발의)(의안번호 2210387)\n12. 농어촌정비법 일부개정법률안(서천호 의원 대표발의)(의안번호 2211302)\n13. 농어촌정비법 일부개
정법률안(김선교 의원 대표발의)(의안번호 2211508)\n14. 농어촌 빈집정비 특별법안(서삼석 의원 대표발의)(의안번호 2212742)\n15. 농어촌 빈집
정비 및 관리에 관한 특별법안(대안)\n16. 농어촌구조개선 특별법안(윤준병 의원 대표발의)(의안번호 2209126)\n17. 자유무역
협정 체결에 따른 농어업인 등의 지원에 관한 특별법 일부개정법률안(윤준병\의원 대표발의)(의안번호 2209127)\n18. 농수산물 유통 및 가격안
정에 관한 법률 일부개정법률안(박덕홍 의원 대표발의)(의안번호\2206171)\n19. 농수산물 유통 및 가격안정에 관한 법률 일부개정법률안(김선
교 의원 대표발의)(의안번호\2206203)\n20. 농수산물 유통 및 가격안정에 관한 법률 일부개정법률안(임미애 의원 대표발의)(의안번호\220675
8)\n21. 농수산물 유통 및 가격안정에 관한 법률 일부개정법률안(윤준병 의원 대표발의)(의안번호\2206783)\n22. 농수산물 유통 및 가격안정에
관한 법률 일부개정법률안(문대림 의원 대표발의)(의안번호\2207683)', keywords=['법률안', '법안', '정비법', '농지법', '윤준병']), Docum
entSegmentSaveSchema(document_id=51, process_id=51, page_number=7, speaker_name='어기구', speaker_role='위원장', original_text='제 429회
```

[그림-17]

[7-1-4] 문장 임베딩 생성

키워드 생성이 완료된 세그먼트를 대상으로 문장 임베딩을 생성한다. Politi-Search 프로젝트에서는 jhgan/ko-sbert 계열의 Ko-SBERT 모델을 사용하여 발언 텍스트를 고정 차원의 벡터로 변환하였다. 이 임베딩 벡터는 발언의 문맥적 의미를 반영하므로, 단어 수준 검색을 넘어 의미 기반 검색과 유사도 계산에 활용할 수 있다.

임베딩 생성 또한 비동기 처리로 수행되며, 계산된 벡터는 Elasticsearch의 dense_vector 필드에 저장되어 cosine similarity 기반 벡터 검색 및 하이브리드(키워드 + 의미) 검색 구조 구현의 핵심 데이터로 활용된다. 이를 통해 사용자는 동일한 정책 주제가 서로 다른 표현으로 기술된 경우에도 의미적으로 유사한 발언을 안정적으로 탐색할 수 있다.

```
INFO: 172.217.161.227:28074 - "POST /api/v1/pdf/run-track-a?limit=1 HTTP/1.1" 200 OK
[API 요청] 임베딩 벡터 생성 시작
[3-b단계] 임베딩 생성 시작 (Limit: 3)
----- [Track B] 51개 세그먼트 임베딩 시작 -----
▶ [Start] Process ID 1 (세그먼트 141개)
Batches: 100%|██████████| 5/5 [00:06<00:00, 1.20s/it]
총 141건 임베딩 및 저장 완료
[성공]] Process ID 1 완료
▶ [Start] Process ID 2 (세그먼트 66개)
Batches: 100%|██████████| 3/3 [00:04<00:00, 1.41s/it]
총 66건 임베딩 및 저장 완료
[성공]] Process ID 2 완료
▶ [Start] Process ID 3 (세그먼트 148개)
Batches: 100%|██████████| 5/5 [00:05<00:00, 1.16s/it]
총 148건 임베딩 및 저장 완료
[성공]] Process ID 3 완료
▶ [Start] Process ID 4 (세그먼트 254개)
Batches: 100%|██████████| 8/8 [00:10<00:00, 1.33s/it]
총 254건 임베딩 및 저장 완료
[성공]] Process ID 4 완료
▶ [Start] Process ID 5 (세그먼트 25개)
Batches: 100%|██████████| 1/1 [00:02<00:00, 2.08s/it]
총 25건 임베딩 및 저장 완료
[성공]] Process ID 5 완료
```

[그림-18]

[7-2] AI 적용 범위 및 설계 의도

Politi-Search 프로젝트에서 인공지능(AI)은 단순 자동화 도구가 아니라, 국회 회의록이라는 고복잡도 정책·법률 문서를 의미 단위로 재구성하고 탐색 가능하게 만드는 핵심 해석 계층으로 설계되었다. 본 절에서는 기존 규칙 기반·통계 기반 접근의 한계를 분석하고, 형태소 분석과 문맥 기반 임베딩을 결합한 AI 처리 구조를 선택한 이유를 설명한다.

[7-2-1] 규칙 기반 텍스트 처리의 한계

국회 회의록은 형식적으로는 일정한 패턴(화자 표시, 안전 구분, 절차 발언 등)을 가지지만, 발언 내용 자체는 정책 설명, 질의응답, 법률해석, 의견 표명 등이 혼재된 고도로 비정형적인 텍스트이다. 기존의 정규표현식이나 규칙 기반 처리 방식은 화자 인식이나 구조 분리에는 효과적이지만, 발언의 의미나 정책적 맥락을 해석하는 데에는 근본적인 한계를 가진다.

특히 동일한 정책 개념이 서로 다른 표현으로 반복 등장하는 경우(예: “지원 확대”, “재정적 뒷받침”, “예산 반영”) 단순 문자열 매칭이나 키워드 기반 검색은 의미적 연관성을 포착하지 못한다. 이러한 한계는 국회 회의록 검색 시스템에서 맥락 기반 탐색과 정책 흐름 분석을 어렵게 만드는 주요 원인으로 작용한다.

[7-2-2] 형태소 분석 기반 접근을 선택한 이유

한국어 조사와 어미가 명사 및 용어에 결합되는 교착어적 특성을 가지며, 국회 회의록은 여기에 구어체 발화와 절차적 관용 표현이 결합된 데이터이다. 이로 인해 단순 토큰화된 문자열 분해 방식은 의미 단위와 문법적 기능어를 구분하지 못하고, 검색 노이즈를 과도하게 생성하는 문제가 발생한다.

이로 인해 한국어 형태소 분석기인 Kiwi를 도입하여, 발언 텍스트를 의미를 구성하는 최소 단위 중심으로 정제하였다. 형태소 분석을 통해 조사, 어미, 보조 표현을 제거하고 명사 중심의 후보 공간을 구성함으로써, 이후 키워드 추출 및 임베딩 단계에서 의미 왜곡을 최소화하도록 설계하였다.

즉, 형태소 분석은 단순 전처리 단계가 아니라, AI 기반 의미 분석의 입력 품질을 보장하기 위한 구조적 필터링 장치로 활용되었다.

[7-2-3] 임베딩 기반 의미 분석을 도입한 이유

형태소 분석만으로는 발언 간 의미적 유사성이나 정책 맥락의 연속성을 충분히 표현하기 어렵다. 동일한 정책 주제가 서로 다른 문장 구조나 어휘로 표현되는 경우, 단어 단위 분석은 의미 연결을 보장하지 못한다.

이를 해결하기 위해 Politi-Search 프로젝트에서는 문장 단위 문맥 정보를 반영할 수 있는 Ko-SBERT 기반 의미 임베딩 접근을 선택하였다. Ko-SBERT는 문장을 고차원 벡터

공간으로 사상하여, 표면적으로 다른 표현이라도 의미적으로 유사한 발언들을 가까운 위치에 배치할 수 있다.

이를 통해 사용자는 특정 키워드가 포함되지 않더라도, 동일한 정책·법률 맥락에 속하는 발언을 의미 기반으로 탐색할 수 있으며, 이는 기존 키워드 검색 방식이 가지는 한계를 근본적으로 보완한다.

[7-2-4] 규칙·언어학·의미 모델의 결합 구조

프로젝트에서의 AI 설계는 단일 기법에 의존하지 않고, 각 접근 방식의 장점을 결합하는 구조로 구성되었다. 정규표현식과 파서 기반 규칙 처리의 회의록의 구조적 안정성을 확보하는 데 사용되었고, 형태소 분석은 언어학적 노이즈 제거와 의미 단위 정제를 담당하였다. 이후 Ko-SBERT 임베딩은 발언의 문맥적 의미를 고차원 공간에서 표현하며, MMR 기반 재랭킹은 의미 중복 문제를 기하학적으로 제어하는 역할을 수행한다.

이와 같이 다층적 설계는 국회 회의록과 같은 복합 문서에서 구조적 정확성, 의미 표현력, 검색 실효성을 동시에 확보하기 위한 필연적인 선택이며, 이후에 설명하는 각 기술 요소의 결합을 통해 그 효과를 실험적으로 검증한다.

[7-3] 텍스트 정제 단계: Kiwi 형태소 분석기

Politi-Search 프로젝트에서는 회의록 발언 텍스트의 의미 기반 분석을 위해 형태소 분석 기반 텍스트 정제 단계를 설계하였다. 국회 회의록은 조사, 어미, 접사 등이 빈번하게 결합되는 한국어의 교착어적 특성과 구어체 발화 특성이 결합된 데이터로, 단순 문자열 기반 분석이나 토큰화 방식만으로는 검색 품질과 의미 해석의 정확도를 확보하기 어렵다.

그렇기에 한국어 형태소 분석기인 Kiwi(Korean Intelligent Word Identifier)를 활용하여, 발언 텍스트에서 의미를 구성하는 핵심 어휘를 식별하고 이후 키워드 추출 및 문장 임베딩 단계의 입력 품질을 향상시키고자 하였다.

[7-3-1] Kiwi 형태소 분석기의 기본 개념

Kiwi는 한국어 문장을 형태소 단위로 분석하고 각 형태소에 품사 정보를 부여하는 형태소 분석기이다. 한국어는 조사와 어미가 명사 및 용어에 결합되는 구조를 가지므로, 동일한 의미의 문장이라도 다양한 표면 형태로 표현된다. 이러한 특성은 검색 시스템에서는 단순 문자열 매칭을 적용할 경우 검색 누락이나 의미 왜곡을 초래하는 주요 원인이 된다.

예를 들어, “국가재법”라는 표현은 이미 표면적으로 하나의 문자열처럼 보이지만, 실제로는 명사 “국가재정법”과 조사 “에”가 결합된 형태이다. Kiwi는 이러한 구조를 인식하여, 의미를 담당하는 핵심 형태소와 문법적 기능을 수행하는 요소로 분리한다.

Kiwi의 핵심 목적은 문장에서 의미를 결정하는 형태소를 식별하고, 검색이나 의미 분석에 불필요한 요소를 제거하거나 분리할 수 있는 정보를 제공하는 데 있다. Kiwi를 텍스트 정제 단계에 배치함으로써, 이후 단계에서 수행되는 의미 임베딩과 키워드 추출이 보다 정확한 입력을 기반으로 수행되도록 설계하였다.

[7-3-2] Kiwi 형태소 분석기의 작동 원리

[1]문제를 인간 언어로 재정의

사람은 “먹었다”라는 문자열을 보면 즉시 다음과 같이 해석한다.

- 먹/였/다
- 동사/과거/종결

그러나 컴퓨터에게 이 문제는 전혀 다른 방식으로 정의된다. 컴퓨터의 관점에서 형태소 분석이란, 주어진 문자열을 생성할 수 있는 모든 가능한 형태소 시퀀스 중에서 확률적으로 가장 그럴듯한 하나를 선택하는 문제이다.

[2] 수학적 문제 정의

관측값은 문자열 X 이다.

$X = \text{"먹었다"}$

숨겨진 값은 형태소 시퀀스 Y 이다.

$Y = y_1, y_2, \dots, y_k$

각 y_i 는 형태소 + 품사

예를 들면,

$y_1 = \text{먹/VV}$

$y_2 = \text{였/EP}$

$y_3 = \text{다/EF}$

[3] 목표 정의

우리가 찾고자 하는 것은 다음을 만족하는 형태소 시퀀스이다.

$$\hat{Y} = \arg \max_y P(Y | X)$$

즉, “이 문자열이 주어졌을 때, 해당 형태소 시퀀스가 맞을 확률이 가장 큰 것”이다.

[4] 베이즈 정리를 이용한 문제 분해

베이즈 정리를 적용하면 다음과 같이 표현된다.

$$P(Y|X) = \frac{P(X|Y)P(Y)}{P(X)}$$

여기서 $P(X)$ 는 모든 Y 에 대해 동일한 상수이므로 생략할 수 있다. 따라서 최적화 문제는 다음과 같이 단순화된다.

$$\hat{Y} = \arg \max_y P(X|Y)P(Y)$$

[5] 두 확률의 의미

[5-1] $P(Y)$: 형태소 시퀀스의 자연스러움

$$P(Y) = P(y_1) \cdot P(y_2 | y_1) \cdot P(y_3 | y_2) \dots$$

이는 형태소 단어 언어 모델에 해당하며, 문법적으로 자연스러운 연결일수록 높은 확률을 가진다.

[5-2] $P(X|Y)$: 관측 문자열 설명 가능성

$$P(X|Y) = \prod_i P(\text{표면 문자열} | y_i)$$

해당 형태소들이 실제로 문자열을 생성할 수 있는지를 나타내며, 불가능한 결합의 경우 확률은 0이 된다.

[6] 로그 공간에서의 계산

확률을 그대로 곱할 경우 언더플로우 문제가 발생하므로, 실제 계산에서는 로그를 사용한다.

$$\log P(X|Y) + \log P(Y)$$

이는 곱셈 기반 최적화 문제를 덧셈 기반 최적화 문제로 변환한다.

[7] 형태소 후보 폭발 문제와 DAG 구성

문장이 길어질수록 가능한 형태소 시퀀스의 수는 기하급수적으로 증가한다. 이를 해결하기 위해 Kiwi는 문자열 위에 형태소 후보를 노드로 배치하고, 가능한 전이만을 간선으로 연결한 방향 비순환 그래프(DAG)를 구성한다.

[8] 비터비(Viterbi) 알고리즘

DAG 상에서 다음 점화식을 사용하여 최적 경로를 탐색한다.

$$V(s) = \max[V(s') + \log P(s | s') + \log P(x | s)]$$

각 단계에서는 가장 점수가 높은 경로 하나만 유지하여 계산 복잡도를 줄인다.

[9] 수학적 정당성: Markov 가정

Kiwi의 내부 모델은 다음 Markov 가정을 따른다.

$$P(y_t | y_1, \dots, y_{t-1}) = P(y_t | y_{t-1})$$

이로 인해 부분 경로가 최적이면 전체 경로에서도 최적이라는 성질이 성립하며, 동적 계획법 적용이 가능해진다.

[7-3-4] 프로젝트에서의 실제 적용 코드

```
def get_candidates(text):
    tokens = kiwi.analyze(text)
    return [
        t.form
        for t in tokens[0][0]
        if t.tag in ['NNG', 'NNP', 'SL'] and len(t.form) > 1
    ]
```

[1] 코드 역할

위 함수는 발언 텍스트에서 의미 분석에 활용할 명사 중심의 후보 단어 집합을 생성한다. Kiwi의 형태소 분석 결과 중 가장 확률이 높은 형태소 시퀀스를 활용하여, 이후 키워드 추출 단계의 입력으로 사용된다.

[2] tokens[0][0]

- 첫 번째[0]: 입력 문장
- 두 번째[0]: Kiwi가 확률적으로 가장 자연스럽다고 판단한 1-best 형태소 시퀀스

이는 다음 수식을 통해 선택된 결과이다.

$$\hat{Y} = \arg \max_y P(X | Y) P(Y)$$

[7-3-5] 설계 의의

프로젝트에서 Kiwi 형태소 분석기는 단순한 토큰 분해 도구가 아니라, 의미 기반 분석을 위한 정제 장치로 활용되었다. 조사와 어미를 제거한 명사 중심 후보 생성은 키워드 추출의 노이즈를 줄이고, Ko-SBERT 기반 문장 임베딩 의미 일관성을 높이는 데 기여하였다.

이를 통해 국회 회의록과 같은 고복잡도 구어체·법률 텍스트에 대해, 형태소 분석 기반 전처리가 의미 검색 품질 향상의 핵심 요소임을 확인하였다.

[7-4] Ko-SBERT 기반 문맥 의미 임베딩

국회 회의록 발언 및 정책 문서를 의미 기반으로 표현하기 위해 사용한 Ko-SBERT 모델의 개념적 배경, 구조적 정의, 기하학적 해석 및 키워드 추출 단계와 결합 방식을 설명한다. Ko-SBERT는 형태소 기반 접근이 가지는 한계를 보완하고, 문맥 단위 의미 비교 및 검색 품질을 향상시키기 위한 핵심 구성 요소이다.

[7-4-1] 문맥 기반 의미 임베딩의 기본 개념

[1] 문맥 기반 의미 임베딩이란 무엇인가

문맥 기반 의미 임베딩(Contextual Semantic Embedding)이란, 단어 또는 문장을 고정된 의미로 취급하지 않고, 문장 내 위치와 주변 단어들과의 관계를 고려하여 벡터로 표현하는 방식을 의미한다.

기존의 형태소 분석이나 TF-IDF 기반 기법은 단어를 독립적으로 토큰 단위로 처리하므로, 동일한 단어가 서로 다른 정책·법률 맥락에서 사용되더라도 동일한 의미로 인식되는 한계를 가진다. 반면 Ko-SBERT는 문장 전체를 하나의 의미 단위로 인식함으로써, “어떤 표현이 어떤 맥락에서 사용되었는가”를 벡터 공간에 직접 반영한다.

예를 들어, ‘지원’이라는 단어는 재정 지원, 제도적 지원, 행정적 지원 등 다양한 의미로 사용될 수 있으며, Ko-SBERT는 이러한 차이를 문맥 정보로부터 학습하여 상이한 벡터로 표현한다.

[2] 고차원 의미 공간으로의 매핑

Ko-SBERT는 입력된 문장 또는 구문을 고정 차원의 실수 벡터(일반적으로 768 차원)으로 변환한다. 이 과정은 텍스트를 고차원 의미 공간 상의 하나의 점으로 사상(mapping)하는 과정으로 이해할 수 있다.

이 의미 공간에서 의미적으로 유사한 문장들은 가까운 위치에 배치되고, 맥락이나 주제가 상이한 문장들은 멀리 배치된다. 이를 통해 형태소 단위로 분절된 복합 명사 표현들도 동일한 정책·법률 맥락에 속할 경우 의미적으로 가까운 벡터로 표현될 수 있다.

[3] 형태소 기반 접근의 한계 보완

프로젝트에서는 Kiwi 형태소 분석기를 통해 명사 중심의 후보 키워드를 생성하되, Ko-SBERT를 통해 해당 후보들이 동일한 정책·법률 맥락에서 사용되는지를 의미적으로 판단한다.

예를 들어, ‘문화관광축제’, ‘지역축제’, ‘글로벌 축제’와 같이 표면적으로 상이한 표현들은 Ko-SBERT 임베딩 공간에서 가까운 위치에 배치되며, 이는 복합 명사 파편화 문제를 완화하는 역할을 수행한다.

[7-4-2] Ko-SBERT의 구조적 정의 및 임베딩 생성 과정

[1] Ko-SBERT의 구조적 정의

Ko-SBERT(Korean Sentence-BERT)는 사전 학습된 BERT 인코더 위에 문장 임베딩 생성을 위한 풀링 계층을 추가하고, 시암 네트워크(Siamese Network) 구조를 통해 의미 유사도 학습을 수행한 모델이다.

이를 함수 합성 형태로 표현하면 다음과 같다.

$$f_{Ko-SBERT}(x) = g_{pool}(f_{BERT}(x))$$

[2] 입력 단계: 토큰화 및 초기 임베딩

입력 문장 x 는 WordPiece 기반 토큰라이저를 통해 토큰 시퀀스로 변환된다.

$$x \rightarrow (t_1, t_2, \dots, t_n), \quad t_i \rightarrow e_i \in \mathbb{R}^{768}$$

각 토큰 t_i 는 다음 세 임베딩의 합으로 표현된다.

$$x_i = e_i^{token} + e_i^{position} + e_i^{segment}$$

[3] BERT 인코딩: 문맥화된 토큰 표현

BERT 인코더는 self-Attention 메커니즘을 통해 각 토큰을 문맥이 반영된 표현으로 변환한다.

$$(e_1, \dots, e_n) \xrightarrow{\text{Transformer Encoder(BERT)}} \{h_1, h_2, \dots, h_n\}, \quad h_i \in \mathbb{R}^{768}$$

이 단계의 출력은 토큰 단위의 문맥 표현 집합이며, 아직 문장 전체를 대표하는 벡터는 아니다.

[4] 풀링 계층: 문장 벡터 생성

Ko-SBERT는 Mean Pooling 방식을 사용하여 토큰 임베딩 집합을 하나의 문장 벡터로 응축한다.

$$H = g_{mean}(H) = \frac{1}{n} \sum_{i=1}^n h_i \in \mathbb{R}^{768}$$

이를 통해 문장은 고차원 의미 공간 상의 하나의 점으로 표현된다.

[5] Siamese Network를 통한 의미 공간 학습

학습 단계에서는 동일한 파라미터(θ)를 공유하는 두 개의 인코더를 사용한다.

$$v_A = f_{\theta}(x_A), \quad v_B = f_{\theta}(x_B)$$

의미적으로 유사한 문장 쌍은 임베딩 공간에서 가까워지도록, 비유사한 문장 쌍은 멀어지도록 학습이 진행된다. 이로 인해 Ko-SBERT는 단순 문맥 표현을 넘어 의미저거 좌표계를 형성하게 된다.

[7-4-3] 임베딩 공간의 기하학적 해석

[1] 768 차원의 의미 공간

768 차원은 BERT-base 의 hidden size 로, 12 개의 attention head(각 64 차원)의 병렬 구조에 의해 결정된 값이다. 이는 문장이 가질 수 있는 다수의 잠재 의미 인자(latent factors)를 표현하기 위한 충분한 자유도를 제공한다.

[2] 단위 초구 상의 의미 비교

코사인 유사도를 사용할 경우, 모든 문장 벡터는 정규화되어 767 차원 단위 초구 $\mathbb{S}^{767}$ 위의 점으로 해석된다.

$$\hat{v} = \frac{v}{\|v\|} \rightarrow \hat{v} \in \mathbb{S}^{767}$$

이 공간에서 두 벡터가 가깝다는 것은 모델이 학습을 통해 도출한 강한 의미적 연관성이 있음을 의미한다.

[3] 비등방성 문제

실제 Ko-SBERT 임베딩 공간은 이상적인 균등 분포가 아닌, 특정 방향으로 밀집되는 비등방성(anisotropy) 현상을 보인다. 이로 인해 단순 유사도 기반 키워드 추출 시 의미적으로 중복된 표현들이 상위에 집중되는 문제가 발생한다.

[7-4-4] MMR 기반 의미 다양성 확보

이러한 비등방성 문제를 해결하기 위해 MMR(Maximal Marginal Relevance) 알고리즘을 적용하였다. MMR 은 문서와의 유사도를 유지하면서, 이미 선택된 후보들과의 중복을 최소화하는 방향으로 키워드를 선택한다.

$$MMR = \arg \max_{D_i \in K \setminus S} [\lambda \cdot \text{sim}(D_i, \text{Doc}) - (1 - \lambda) \cdot \max_{D_j \in S} \text{sim}(D_i, D_j)]$$

이는 기하학적으로 문서 벡터에 대한 인력(attractive force)과 기존 선택 벡터들로부터 척력(repulsive force)의 균형으로 해석할 수 있다.

[7-4-5] jhgan/ko-sbert-sts 모델 선택 이유

프로젝트에서는 Ko-SBERT 모델 중 jhgan/ko-sbert-sts 를 사용하였다. 해당 모델을 선택한 이유는 다음과 같다.

1. 한국어 STS(Semantic Textual Similarity) 태스크에 특화
 - 국회 회의록과 같은 문장 간 의미 비교가 핵심인 데이터 특성에 적합
2. 문장 단위 임베딩 품질
 - 키워드, 발언, 문서 요약 벡터 간의 직접 비교가 가능
3. Mean Pooling 기반 구조
 - 문장 길이 및 발화 스타일이 다양한 회의록 데이터에 대해 안정적인 표현 제공
4. 실무적 효율성
 - 문장을 독립적으로 임베딩할 수 있어 대규모 문서 저장 및 검색 시 계산 효율이 높음
5. Ko-SBERT + MMR + KeyBERT 구조와의 높은 정합성
 - 의미 유사도 기반 키워드 추출 파이프라인에 자연스럽게 결합 가능

[7-4-6] 정리

Ko-SBERT 는 프로젝트에서 단순한 임베딩 생성기가 아니라, 형태소 기반 전처리와 의미 기반 검색을 연결하는 핵심 의미 표현 계층으로 작동한다. 이를 통해 국회 회의록과 같은 고복잡도 정책 텍스트에 대해 문맥 중심의 의미 비교, 키워드 추출, 검색 품질 향상을 안정적으로 달성할 수 있었다.

[7-5] 정보 추출 최적화 및 파라미터 튜닝

국회 회의록을 대상으로 한 의미 기반 키워드 추출 과정에서 정보 손실을 최소화하고, 문서별 핵심 의제를 안정적으로 포착하기 위해 적용한 최적화 전략과 파라미터 튜닝 과정을 설명한다. 본 실험은 언어학적 제약 조건과 고차원 의미 공간에서의 수학적 최적화를 결합하는 방향으로 설계되었다.

[7-5-1] 언어학적 특징 기반 후보 생성

한국어 법률·행정 문서는 조사, 어미, 직위명, 절차적 관용 표현이 빈번히 등장하는 형태론적 고밀도 텍스트이다. 이러한 특성으로 인해 단순 토큰화나 빈도 기반 후보 생성 방식은 의미적 중요도가 낮은 표현을 과도하게 포함하는 문제가 발생한다.

이를 해결하기 위해 본 탐구에서는 Kiwi 형태소 분석기를 활용한 형태·의미론적 필터링(Morpho-Semantic Filtering) 기반 후보 생성 전략을 적용하였다. 정보 검색 관점에서 실질적인 의미 단위를 구성하는 일반명사(NNG)와 고유명사(NNP)만을 분석 대상으로

제한하였으며, 조사·어미·보조 용언과 같은 기능어는 사전에 제거하였다. 이는 Ko-SBERT 임베딩 공간에서 기능어가 문서 중심 벡터와의 코사인 유사도 계산 시 노이즈로 작용하는 문제를 방지하기 위함이다.

또한, 두 글자 미만의 토큰과 숫자 단독 토큰을 제거하여 우연적 매칭 가능성을 차단하였으며, 국회 회의록 도메인 특성을 반영해 의원 직위명, 발언 관용구, 법률 형식어, 기관명 등 빈도는 높으나 지시 대상이 불분명한 표현을 도메인 특화 불용어로 정의하였다. 이는 임베딩 공간에서 특정 표현군으로 의미가 쏠리는 의미론적 드리프트(Semantic Drift) 현상을 사전에 차단하기 위한 조치이다.

이러한 언어학적 제약은 실제 구현 단계에서 다음과 같은 사용자 정의 토크나이저로 구체화되었다.

```
def kiwi_tokenizer(text):
    tokens = kiwi.analyze(text)
    result = []

    if not tokens:
        return result

    for t in tokens[0][0]:
        if t.tag not in ["NNG", "NNP"]:
            continue
        if len(t.form) < 2:
            continue
        if t.form in CUSTOM_STOPWORDS:
            continue
        if t.form.isdigit():
            continue

        result.append(t.form)

    return result
```

이 토크나이저는 단순한 전처리 단계를 넘어, 임베딩 단계로 전달되는 후보 공간 자체를 제약함으로써 의미 벡터 분포를 구조적으로 통제하는 역할을 수행한다.

[7-5-2] 하이퍼파라미터 그리드 서치 설계

의미 기반 키워드 추출의 성능은 후보 단위의 구성 방식과 MMR 재랭킹 파라미터 설정에 따라 크게 달라진다. 이에 따라 본 탐구에서는 다음 두 가지 핵심 변수를 독립적으로 설정하여 그리드 서치를 수행하였다.

첫째, n-gram 범위 설정이다. unigram(1,1)은 단일 개념 포착에는 유리하지만, ‘문화상품’과 ‘제작’처럼 의미적으로 결합된 정책 용어가 분리되는 한계가 있다. 반면 bigram 을 포함한 (1,2) 설정은 ‘문화상품 제작’, ‘지방자치단체 지원’과 같은 복합 명사구를 하나의 의미 단위로 유지할 수 있어 정책 맥락 보존에 유리하다.

후보 생성은 Kiwi 기반 토큰라이저와 결합된 CountVectorizer 를 통해 수행되었다.

```
vectorizer = CountVectorizer(  
    tokenizer=kiwi_tokenizer,  
    ngram_range=nr,  
    token_pattern=None  
)
```

둘째, MMR(Maximal Marginal Relevance) 다양성 파라미터 λ 이다. λ 값은 문서 중심 벡터와의 유사도(Relevance)와 키워드 간 중복 제거(Diversity) 간의 트레이드오프를 조절한다. 낮은 λ 값은 문서 중심 의미에 과도하게 밀착된 키워드만을 선택하여 유의어 중복 문제가 발생할 수 있으며 높은 λ 값은 문서 내 상대적으로 덜 강조된 세부 의제까지 포착할 수 있다.

본 탐구에서 λ 값을 0.2 부터 0.7 까지 단계적으로 변화시키며 키워드 분포의 변화를 관찰하였다. MMR 기반 재랭킹은 다음과 같이 적용되었다.

```
keywords = kw_model.extract_keywords(  
    docs=doc,  
    vectorizer=vectorizer,  
    use_mmr=True,  
    diversity=div,  
    top_n=top_n  
)
```

[7-5-3] 추론 단계 계산 최적화

국회 회의록과 같이 분량이 큰 문서를 대량 처리하는 환경을 고려하여, 추론 단계에서는 계산 효율성 도한 주요 고려 요소로 설정하였다. Ko-SBERT 임베딩 생성 및 키워드 추출 과정은 GPU(CUDA) 환경에서 수행되며, 연산 지원을 효율적으로 활용하기 위해 다음과 같이 최적화를 적용하였다.

```
device = "cuda" if torch.cuda.is_available() else "cpu"
model = SentenceTransformer(model_path, device=device)
```

또한 키워드 추출 전체 과정을 `torch.no_grad()` 컨텍스트 내에서 수행하여, 역전파를 위한 연산 그래프 저장을 방지함으로써 메모리 사용량을 유의미하게 절감하였다.

```
with torch.no_grad():
    keywords = kw_model.extract_keywords(...)
```

이러한 최적화를 통해 고차원 행렬 연산이 반복되는 임베딩 및 유사도 계산 과정에서도 안정적인 처리 속도를 확보할 수 있었다.

[7-6] 결과 해석

[7-5]에서 수행한 하이퍼파라미터 그리드 서치 결과를 바탕으로, n-gram 범위와 MMR 다양성 파라미터(diversity)가 키워드 추출 결과에 미치는 영향을 분석한다. 이를 통해 국회 회의록이라는 정책·법률 도메인에서 의미 기반 키워드 추출 방식의 실효성을 종합적으로 해석한다.

[7-6-1] 문서별 키워드 추출 결과 개요

실험은 총 4 개의 회의록 발언 문서를 대상으로 수행되었으며, 각 문서에 대해 `ngram_range` $\in \{(1, 1), (1, 2)\}$ 와 `diversity` $\in [0.2, 0.7]$ 조합을 적용하였다.

그 결과, 문서의 내용 밀도와 구조적 특성에 따라 키워드 분포 양상이 뚜렷하게 구분되는 것이 확인되었다.

문서 1~3 의 경우 정책·제도·법률 해석과 관련된 실질적 논의가 포함되어 있어 의미 있는 키워드가 안정적으로 추출되었다. 문서 4 는 의례적 발언으로만 구성되어 모든 설정에서 키워드가 생성되지 않았다. 이는 정보 밀도가 낮은 발언을 자동으로 배제하는 방향으로 시스템이 정상적으로 작동함을 보여준다.

[7-6-2] N-gram 범위에 따른 영향 분석

[1] Unigram(ngram_range(1,1))

Unigram 설정에서는 문서의 중심 개념을 구성하는 단일 명사가 주로 추출되었다. 예를 들어, 문서 1에서는 “세종”, “법률”, “예술”, 문서 2에서는 “올림픽”, “장애인”, 문서 3에서는 “법령”, “의무”, “재량”과 같은 핵심 개념어가 안정적으로 포착되었다.

이는 Unigram 방식의 문서의 주제 중심 키워드 탐지에는 효과적임을 보여주지만, 법률 문서 특유의 복합 명사 구조를 충분히 반영하지 못하는 한계를 동시에 드러낸다. 단일 단어 수준에서는 정책 대상이나 제도 명칭의 구체성이 약화되며, 실제 검색이나 요약 활용 시 의미 해석의 모호성이 발생할 수 있다.

[2] Bigram 포함(ngram_range=(1,2))

Bigram 을 포함한 설정에서는 “국어 기본법”, “패럴림픽 기념사업”, “산업 통상부”, “법령 의무”와 같이 정책·제도 단위의 의미를 직접적으로 표현하는 키워드가 다수 추출되었다.

특히 문서 1 과 문서 2 에서 확인되듯, 법률명·사업명·행정 단위가 하나의 의미 단위로 유지되면서 문서의 정책적 맥락이 보다 명확하게 드러났다. 이는 국회 회의록과 같이 복합 명사 중심으로 의미가 구성되는 문서에서는 (1,2) n-gram 설정이 정보 손실을 최소화하는 데 유리함을 시사한다.

[표] N-gram 범위에 따른 대표 키워드 비교(diversity=0.7)

문서	ngram_range	상위 키워드
문서 1	(1,1)	세종, 예술, 한국콘텐츠진흥원, 평가, 대출
	(1,2)	국어 기본법, 문화 예술, 제안 결과, 청소년 대상, 지원 대출
문서 2	(1,1)	올림픽, 진흥법, 서울, 사용처, 기부
	(1,2)	패럴림픽 기념사업, 사용처 서울, 일부 개정, 행정 재정, 경쟁력 제고
문서 3	(1,1)	법령, 통상부, 산업, 부여, 당구장
	(1,2)	법령 의무, 추후 판단, 산업 통상부, 부여, 당구장 표시

[7-6-3] MMR 다양성 파라미터(Diversity)의 효과

MMR(Maximal Marginal Relevance) 기반 재랭킹에서 diversity(λ)는 키워드 간 중복 제거 수준을 조절하는 핵심 요소로 작용하였다.

[1] 낮은 diversity(0.2~0.3)

낮은 λ 값에서는 문서 중심 벡터와의 유사도가 매우 높은 키워드들이 반복적으로 선택되었다. 예를 들어, 문서 3에서는 “법령”, “의무”, “재량”과 같이 의미적으로 유사한 개념이 집중적으로 추출되어 문서의 단일 논점을 강하게 강조하는 결과를 보였다.

이는 핵심 주제 요약에는 유리하지만, 문서가 포함하는 부차적 논의나 조건적 해석을 충분히 반영하지 못하는 한계를 가진다.

[2] 중간 diversity(0.5~0.6)

중간 수준의 값에서는 문서 내 주요 주제와 함께 행정적·절차적 맥락이 균형 있게 드러나기 시작했다. 문서 2의 경우 “올림픽” 중심 키워드 외에도 “수립”, “문화”, “기부”, “행정 재정” 등의 키워드가 함께 등장하며, 단순 제도 명칭을 넘어 정책 운용 구조까지 포착하는 양상을 보였다.

이는 회의록 분석 목적상 가장 실용적인 구간으로 해석된다.

[3] 높은 diversity(0.7)

$\lambda = 0.7$ 에서는 문서 중심에서 상대적으로 떨어진 키워드까지 포함되며 문서의 세부 의제 탐색 능력이 강화되었다. 다만 문서 1에서 확인되듯 일부 키워드는 의미 유사도 점수가 매우 낮아(예: “대출(-0.01)”) 해석 신뢰도가 저하되는 사례도 발생하였다.

따라서 높은 다양성 설정은 보조 의제 탐색에는 유효하지만, 검색 결과 품질 관리를 위해 추가적인 임계값 조정이 필요함을 시사한다.

[7-6-4] 문서 4의 키워드 미추출 결과 해석

문서 4는 “검토의견과 수정의견에 대해서 동의합니다.”라는 단일 문장으로 구성되어 있으며, 해당 문장에서 핵심 단어들이 모두 도메인 특화 불용어(CUSTOM_STOPWORDS)에 포함되어 있다. 또한 Kiwi 형태소 분석 결과 의미 단위로 남는 명사가 존재하지 않아 후보 키워드 자체가 생성되지 않았다.

이는 시스템의 오류가 아닌, 의미적 정보가 없는 의례적 발언을 자동으로 배제하는 정상적인 동작 결과로 해석된다. 모든 문장에서 강제로 키워드를 생성하지 않고 정보 밀도가 낮은 발언을 구분해낸다는 점에서 본 접근 방식의 정밀성을 확인할 수 있다.

[7-6-5] 종합 해석 및 시사점

실험 결과를 종합하면, 국회 회의록과 같은 정책·법률 문서에서는 `ngram_range=(1,2)`와 `diversity=0.7` 조합이 가장 높은 의미 표현력과 주제 다양성을 동시에 확보하는 것으로 나타났다. 이는 복합 명사 중심의 언어 구조를 유지하면서도 단일 중심 주제에 과도하게 수렴하지 않는 키워드 집합을 구성할 수 있기 때문이다.

결과는 국회 회의록 검색·요약 시스템에서 의미 기반 키워드 추출이 단순 빈도 분석을 넘어, 문서 구조와 논의 맥락을 반영하는 핵심 기술 요소로 활용될 수 있음을 시사한다.

[8] 클라우드

[8-1] 사용한 플랫폼 서비스

Elasticsearch의 단일 노드 환경과 노드 역할을 분리한 클러스터 구성을 모두 지원해야 했기 때문에 서버 인프라를 직접 제어할 수 있는 IaaS(Infrastructure-as-a-Service) 모델을 선택하였다. IaaS는 VM, 네트워크, 스토리지 등을 사용자가 세부적으로 구성할 수 있어, 검색 엔진인 Elasticsearch의 환경 구축에 가장 적합한 방식이라고 판단하였다.

클라우드 서비스 후보군 평가 과정

초기 검토 대상은 Oracle Cloud, AWS, GCP 세 가지였다.

서비스	평가 내용	결론
Oracle Cloud(OCI)	무료 크레딧 제공 및 상대적으로 높은 성능이 장점이었으나, 회원가입 시 본인 인증 문제로 여러 해결 방법을 시도했음에도 계정 생성 실패가 지속됨	사용 불가
AWS	AWS Free Tier 경험이 있었으나, 이전 프로젝트에서 단기 트래픽 증가 시 인스턴스가 즉시 종료되는 문제를 경험했고, 비용 초과 시 요금 예측이 어려움	안정성 측면에서 제외
GCP	안정적인 무료 크레딧 정책, VM 사양 대비 비교적 낮은 비용을 지원하기에 적합하다고 판단	최종 선택

이들 IaaS 서비스 중에서 프로젝트의 목적과 예산 범위 내에서 확장성과 안정성을 가장 균형있게 확보할 수 있다고 판단하여, 최종적으로 GCP(Google Cloud Platform)를 도입하였다.

[8-2] 단일 노드와 클러스터 환경 비교를 위한 시스템 설계

Elasticsearch 단일 노드 환경과 클러스터 환경의 색인 및 검색 성능을 비교하기 위해, 핵심 구성 요소인 FastAPI 백엔드 서버와 Elasticsearch만을 대상으로 테스트를 수행하였다. 검색 성능 및 안정성을 평가하는 것이 목적이었기 때문에, 애플리케이션 서버와 인프라 요소들은 동일한 조건으로 통제하고, 오직 Elasticsearch 구성 방식만 다르게 설계하였다.

두 환경 모두 동일한 VM 사양, 동일한 네트워크 조건, 동일한 FastAPI 애플리케이션 코드, 동일한 Python 라이브러리 및 동일한 색인·검색 API를 사용하였다. 또한 외부 변수의 영향을 줄이기 위해 GCP의 구역(Region)과 가용 영역(Zone)을 us-central1(아이오와)과 A 존으로 고정하여 구성하였다. 리전 또는 존이 달라질 경우 예기치 않은 추가 비용이나 네트워크 지연이 발생할 수 있으므로, 비용 및 성능 왜곡을 방지하기 위한 조치이다.

[8-2-1] 클라우드 VM 공통 구성

두 테스트 환경은 모두 GCP 에서 제공하는 무료 크레딧을 활용하여 구축하였다. Elasticsearch 구성 방식 이외의 변수를 배제하기 위해 완전히 동일한 VM 인스턴스 사양, 동일한 리전, 동일한 네트워크 환경을 적용하였으며, 이를 통해 인프라 차이로 인한 성능 편차를 최소화하였다.

[1] FastAPI 전용 VM 구성

FastAPI 전용 VM 은 애플리케이션 서버로만 활용되도록 구성하였다. 해당 VM 에서는 FastAPI 백엔드 애플리케이션과 Cloud SQL 에 접근하기 위한 cloud_sql_proxy 를 함께 실행된다. 데이터베이스 연결은 외부 IP 방식이 아닌 SQL Proxy 를 통해서만 이루어지도록 제한하여, 접근 제어 강화와 연결 안정성 확보라는 두 가지 목적을 달성하였다.

항목	사양
플랫폼	GCP
인스턴스 유형	e2-medium
CPU	vCPU 2개
메모리	4GB
디스크	30GB
OS	Ubuntu 22.04 LTS

[2] cloud SQL 구성

[2-1] mysql 인스턴스 생성

Cloud SQL 콘솔에서 MySQL 을 선택하고, 기본 가이드라인을 기반으로 인스턴스를 생성하였다.

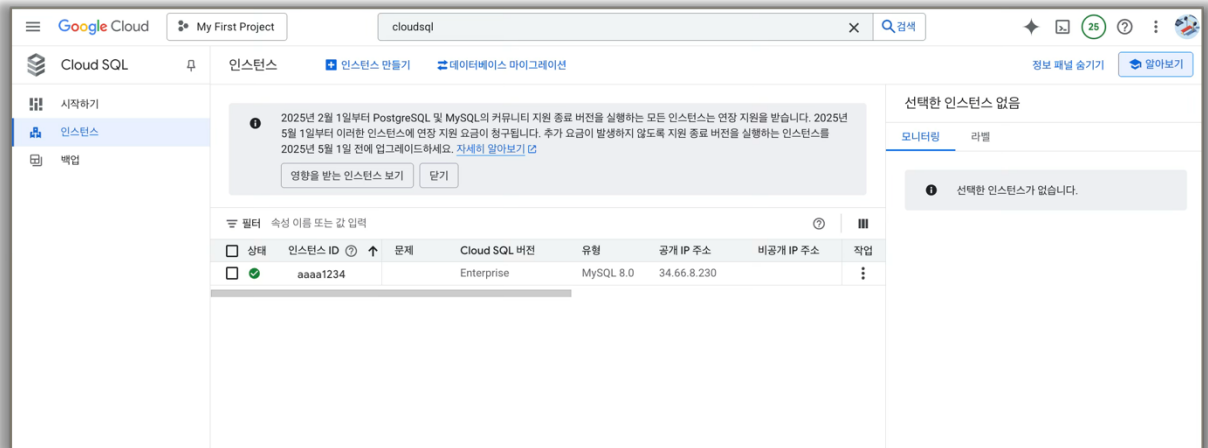
[2-2] 로컬 백업 데이터 이관

VM 에 접속하여 기존 로컬에 보관된 MySQL 백업 파일을 Cloud SQL 로 복원하였다. 다음 과정을 통해 DB 를 마이그레이션 하였다.

- VM 내부에서 백업 파일 업로드
- Cloud SQL MySQL 인스턴스에 연결 후 스키마 및 데이터 복구 수행
- 복구 완료 후 테이블 관계, 인덱스 생성 여부 검증

[2-3] Cloud SQL Proxy 연결 설정

FastAPI 애플리케이션은 외부 IP 로 직접 연결하지 않고, cloud-sql-proxy 를 통해 Cloud SQL 인스턴스에 연결하도록 구성하였다.



[그림-19]

[3] Elasticsearch 환경 설계 차이와 VM 구성

Elasticsearch 환경 설계

구분	단일 노드 환경	클러스터 환경
노드 수	1개	3개(마스터1, 데이터2)
노드 역할	단일 노드에 모두 역할 할당	역할 분리
내결함성	장애 발생 시 즉시 중단	노드 장애 시 서비스 지속 가능
메모리 사용량	상대적으로 낮음	JVM Heap 증가로 메모리 부담
기대 효과	저비용, 구조 단순	고성능 색인/검색, 안정성 향상

Elasticsearch 전용 VM 구성

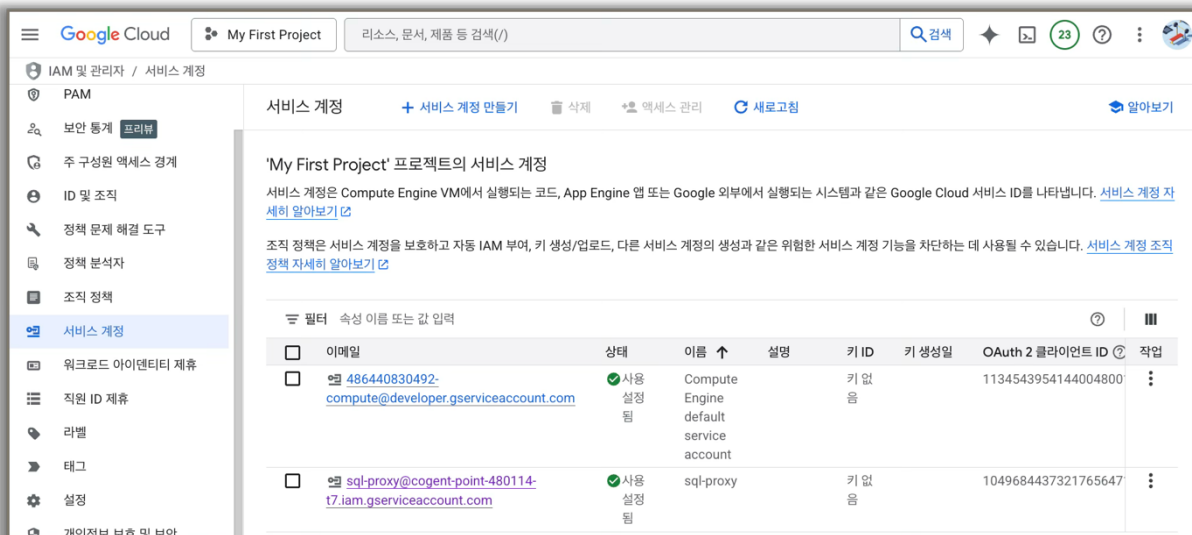
항목	사양
플랫폼	GCP
인스턴스 유형	e2-medium
CPU	vCPU 2개
메모리	4GB
디스크	10GB
OS	Ubuntu 22.04 LTS

[8-2-2] 단일노드 환경 구축

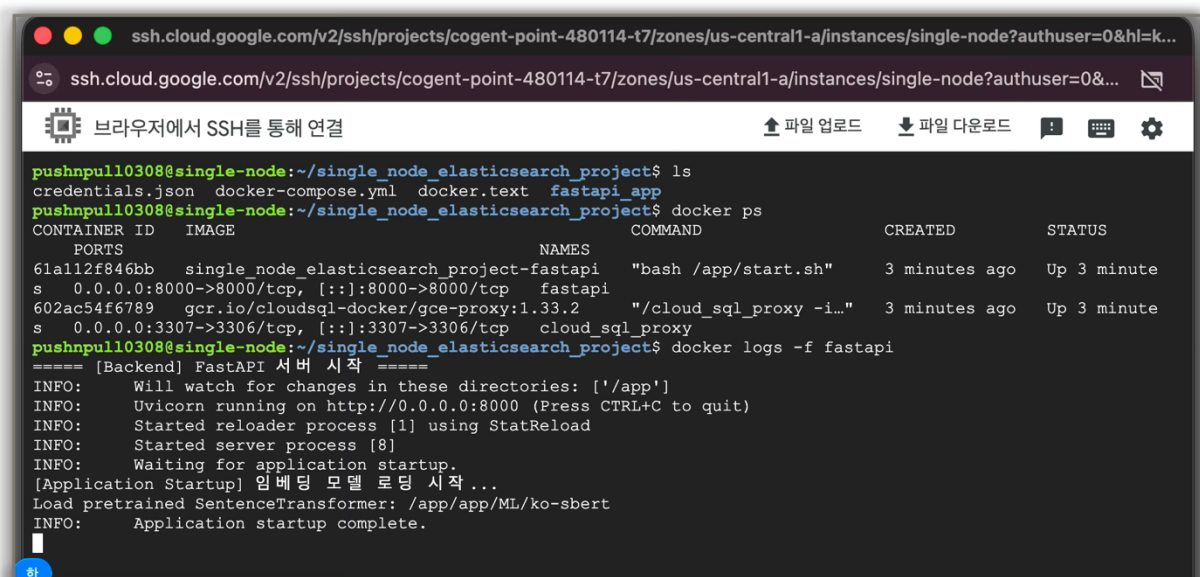
[1] 구성 요소

구성 요소	역할
VM1(FastAPI 전용)	API 서버 + Cloud SQL Proxy
VM2(Elasticsearch 단일 노드)	색인 및 검색 엔진
Cloud SQL(MySQL)	원본 회의록 및 메타데이터 저장

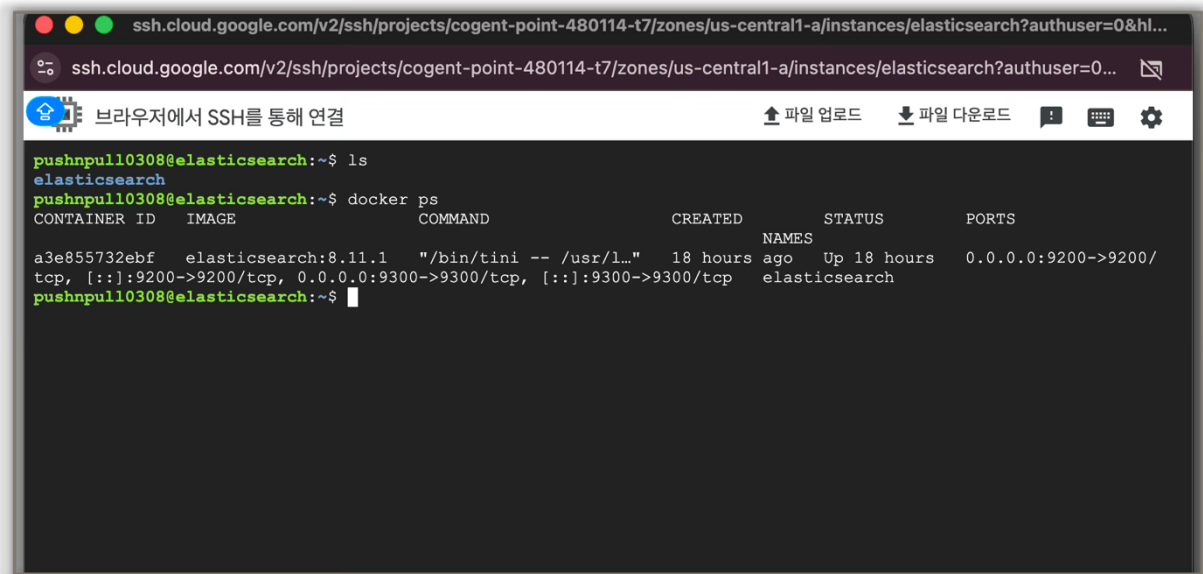
단일 노드 환경에서는 Elasticsearch를 전용 VM에 단독 배치하고, FastAPI는 별도의 VM에서 실행하도록 구성하였다. 두 서버 간 통신은 Google Cloud 내부 네트워크(Internal IP)를 사용하여 외부 트래픽 비용을 방지하고, 통신 지연을 최소화하였다.



[그림-20]

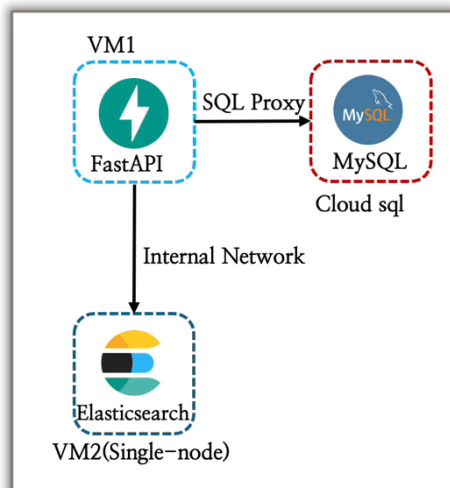


[그림-21]



[그림-22]

[2] 아키텍처



[그림-23]

[3] Elasticsearch 단일 노드

단일노드 환경에서는 node.roles를 별도로 지정하지 않아도, 하나의 노드가 마스터(Master), 데이터(Data), 인제스터(Ingest), 코디네이팅(Coordinating) 역할을 모두 동시에 수행한다. 또한 설정 파일에 discovery.type=single-node를 적용하여 클러스터 탐색 기능을 비활성화하고, 단일 노드만으로 동작하는 구조를 구성하였다.

이러한 방식은 구조가 단순하고 색인(인덱싱) 성능이 빠르다는 장점이 있다. 그러나 노드가 하나뿐이기 때문에 복제본을 저장할 공간이 없어 데이터 손실 위험이 매우 크다는 단점이 존재한다. 즉, VM 디스크가 손상되거나 장애가 발생하면 데이터를 복구할 방법이 없다.


```

pushnpull0308@elasticsearch:~/elasticsearch$ cat docker-compose.yml
version: "3.9"

services:
  elasticsearch:
    image: elasticsearch:8.11.1 # 빌드보다 공식 이미지 사용 권장
    container_name: elasticsearch
    restart: always
    environment:
      - node.name=single-node
      - cluster.name=gcp-cluster
      - discovery.type=single-node
      - ES_JAVA_OPTS=-Xms1g -Xmx1g
      # ★중요 : 외부 (VM 2)에서 접속 허용
      - network.host=0.0.0.0
      # 개발용 보안 해제
      - xpack.security.enabled=false
    ulimits:
      memlock:
        soft: -1
        hard: -1
    ports:
      - "9200:9200"
      - "9300:9300"
    volumes:
      - esdata:/usr/share/elasticsearch/data

volumes:
  esdata:

```

[그림-24]

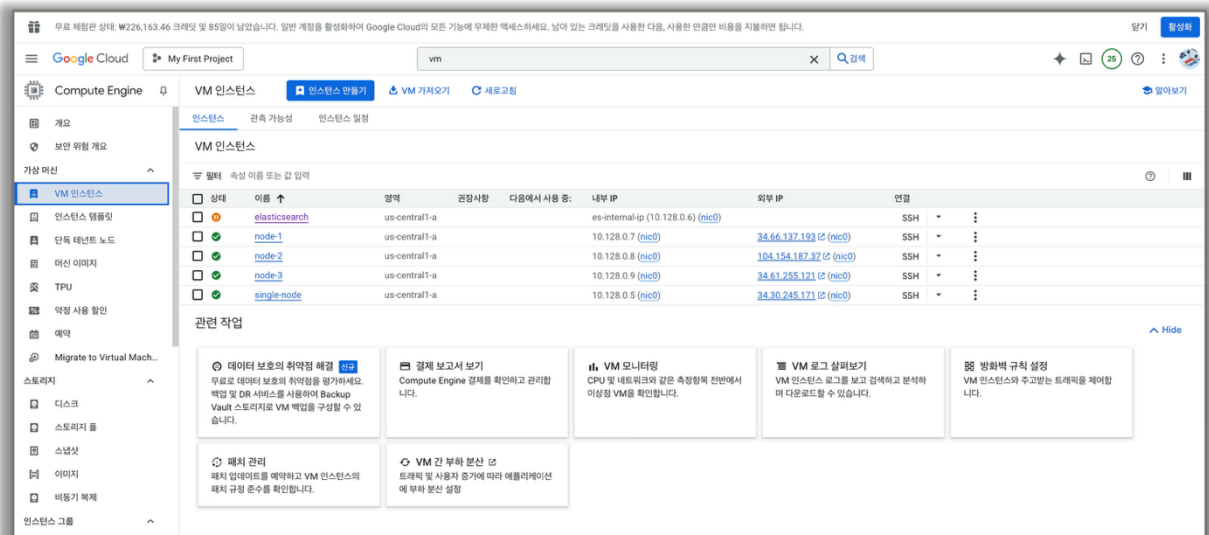
[8-2-3] 노드 역할 분리를 적용한 클러스터 환경(1)

[1] 구성요소

구성 요소	역할
VM1(FastAPI 전용)	API 서버 + Cloud SQL Proxy
VM2(메인/코디네이팅): node-1	Master(클러스터 관리) + Data(데이터 저장) + Ingest/Coord(요청 접수)
VM3(데이터 전용): node-2	Master(후보) + Data(데이터 저장 위주)
VM4(데이터 전용): node-3	Master(후보) + Data(데이터 저장 위주)
Cloud SQL(MySQL)	원본 회의록 및 메타데이터 저장

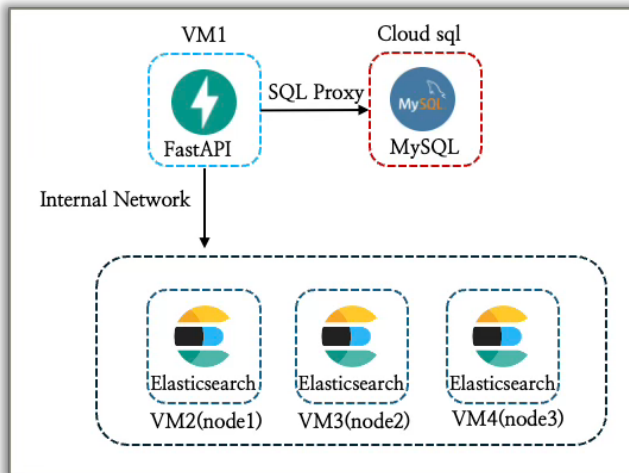
다중 노드 환경에서는 Elasticsearch를 3개의 전용 VM에 분산 배치하여 클러스터 구조를 구축하였다. FastAPI 서버는 별도의 VM에서 단독으로 실행되며, 검색 요청 처리 및 데이터 전송 역할만 담당한다. 애플리케이션 서버(FastAPI)와 Elasticsearch 클러스터 간 통신은 Google Cloud 내부 네트워크(Internal IP)를 통해 이루어지며, 이를 통해 외부 트래픽 비용을 방지하고 네트워크 지연을 최소화하였다.

Elasticsearch는 노드별 역할을 분리하여 구성하였다. node-1은 Master + Coordinator + Data 역할을 수행하여 클러스터 상태 관리와 요청 조정 기능을 담당하며, node-2와 node-3은 Data 노드이면서 Master 후보(Eligible Master)로 설정하였다. 이러한 구성은 Master 장애 발생 시 자동 Failover가 가능하며, 모든 검색 데이터는 여러 노드에 분산 저장되어 안정적인 검색 서비스를 제공한다.



[그림-25]

[2] 간단 아키텍처



[그림-26]

```

sshnpul10308@node-1:~$ curl -X GET "http://localhost:9200/_cat/nodes?v"
ip          heap.percent ram.percent cpu load_1m load_5m load_15m node.role master name
10.128.0.7  37          78      1    0.35   1.43   1.37  dimr      *    node-1
10.128.0.8  21          76      1    0.13   0.92   1.16  dm        -    node-2
10.128.0.9  24          74      1    0.09   0.57   0.97  dm        -    node-3
sshnpul10308@node-1:~$

```

[그림-27]

[3] Elasticsearch

node-1 을 메인 노드로 구성하여 Master, Data, Ingest/Coordinating 역할을 모두 수행하도록 설계하였다. 이 노드는 클러스터 상태 관리뿐 아니라 실제 인덱스 데이터 저장과 검색 요청 처리까지 담당하는 핵심 노드이다. 반면, node-2 와 node-3 은 데이터 전용 노드(Data Node)이면서 Master 후보(Eligible Master)로 설정하였다

```
# [Node-1]
version: "3.9"

services:
  elasticsearch:
    image: elasticsearch:8.11.1
    container_name: es-node1
    restart: always
    environment:
      - node.name=node-1
      - cluster.name=gcp-cluster
      # 3대 모두 Master 후보 + Data 저장 가능하게 설정 (안정성 최우선)
      - node.roles=master,data,ingest,remote_cluster_client

      # [네트워크 설정]
      - network.host=0.0.0.0
      - network.publish_host=10.128.0.7 # * Node-1의 내부 IP

      # [친구 찾기] 3대의 내부 IP를 모두 적어줍니다.
      - discovery.seed_hosts=10.128.0.7,10.128.0.8,10.128.0.9
      - cluster.initial_master_nodes=node-1,node-2,node-3

      # [메모리 및 보안]
      - ES_JAVA_OPTS=-Xms1g -Xmx1g
      - xpack.security.enabled=false
    ulimits:
      memlock:
        soft: -1
        hard: -1
    ports:
      - "9200:9200"
      - "9300:9300"
    volumes:
      - esdata:/usr/share/elasticsearch/data

volumes:
  esdata:
```

[그림-28]

```
# [Node-2]
version: "3.9"

services:
  elasticsearch:
    image: elasticsearch:8.11.1
    container_name: es-node2
    restart: always
    environment:
      - node.name=node-2
      - cluster.name=gcp-cluster
      - node.roles=master,data #["master"]

      # [네트워크 설정]
      - network.host=0.0.0.0
      - network.publish_host=10.128.0.8 # * Node-2의 내부 IP

      # [친구 찾기]
      - discovery.seed_hosts=10.128.0.7,10.128.0.8,10.128.0.9
      - cluster.initial_master_nodes=node-1,node-2,node-3

      - ES_JAVA_OPTS=-Xms1g -Xmx1g
      - xpack.security.enabled=false
    ulimits:
      memlock:
        soft: -1
        hard: -1
    ports:
      - "9200:9200"
      - "9300:9300"
    volumes:
      - esdata:/usr/share/elasticsearch/data

volumes:
  esdata:
```

[그림-29]

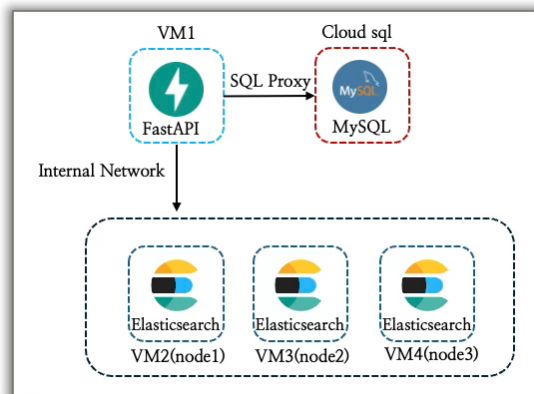
[8-2-4] 노드 역할 분리를 적용한 클러스터 환경(2)

[1] 구성요소

구성 요소	역할
VM1(FastAPI 전용)	API 서버 + Cloud SQL Proxy
VM2(메인/코디네이팅): node-1	Master(클러스터 관리)
VM3(데이터 전용): node-2	Master(후보) + Data(데이터 저장 위주) + Ingest
VM4(데이터 전용): node-3	Master(후보) + Data(데이터 저장 위주) + Ingest
Cloud SQL(MySQL)	원본 회의록 및 메타데이터 저장

8-2-3.(1)에서 VM1, VM3, VM4는 각각 FastAPI 전용 서버와 데이터 저장을 담당하는 Elasticsearch 노드로 구성된다. 이번 구조에서 가장 큰 변화는 VM2의 역할이 완전히 Master 전용 노드로 분리되었다는 점이다. 기존 구성에서는 VM2가 데이터 저장 기능까지 수행했지만, 이번 설정에서는 해당 기능을 제거하여 클러스터 관리와 요청 분배만 전담하는 Master 노드로 운영된다. 데이터 저장과 검색 처리는 오직 데이터 전용 노드(VM3, VM4)가 맡는 형태로 구조가 재편되었다.

[2] 간단 아키텍처



[그림-30]

[3] Elasticsearch

이번 구성에서 node-1(VM1)은 Master 전용 노드로 동작하며, 클러스터 상태 관리, 메타데이터 조정 샤드 배치 결정 등의 핵심적인 클러스터 관리 기능만 수행한다. 반면 node-2(VM3)와 node-3(VM4)는 Data·Ingest 노드로서 실제 데이터 저장, 검색 처리, 인제스터 파이프라인 실행을 담당한다. 또한 두 노드는 Master 후보 역할도 함께 수행한다.

```
# [Node-1]
version: "3.9"

services:
  elasticsearch:
    image: elasticsearch:8.11.1
    container_name: es-node1
    restart: always
    environment:
      - node.name=node-1
      - cluster.name=gcp-cluster
      # 3대 모두 Master 후보 + Data 저장 가능하게 설정 (안정성 최우선)
      - node.roles=master
      # [네트워크 설정]
      - network.host=0.0.0.0
      - network.publish_host=10.128.0.7 # * Node-1의 내부 IP

      # [친구 찾기] 3대의 내부 IP를 모두 적어줍니다.
      - discovery.seed_hosts=10.128.0.7,10.128.0.8,10.128.0.9
      - cluster.initial_master_nodes=node-1,node-2,node-3

      # [메모리 및 보안]
      - ES_JAVA_OPTS=-Xms1g -Xmx1g
      - xpack.security.enabled=false
    ulimits:
      memlock:
        soft: -1
        hard: -1
    ports:
      - "9200:9200"
      - "9300:9300"
    volumes:
      - esdata:/usr/share/elasticsearch/data

volumes:
  esdata:
```

[그림-31]

```
# [Node-3]
version: "3.9"

services:
  elasticsearch:
    image: elasticsearch:8.11.1
    container_name: es-node3
    restart: always
    environment:
      - node.name=node-3
      - cluster.name=gcp-cluster
      - node.roles=["master","data","ingest"]
      # [네트워크 설정]
      - network.host=0.0.0.0
      - network.publish_host=10.128.0.9 # * Node-3의 내부 IP

      # [친구 찾기]
      - discovery.seed_hosts=10.128.0.7,10.128.0.8,10.128.0.9
      - cluster.initial_master_nodes=node-1,node-2,node-3

      - ES_JAVA_OPTS=-Xms1g -Xmx1g
      - xpack.security.enabled=false
    ulimits:
      memlock:
        soft: -1
        hard: -1
    ports:
      - "9200:9200"
      - "9300:9300"
    volumes:
      - esdata:/usr/share/elasticsearch/data

volumes:
  esdata:
```

[그림-32]

[8-3] 비교

비교 대상은 단일 노드 환경, 클러스터 환경(1), 클러스터 환경(2) 총 3가지이며, 평가 항목은 색인 속도와 검색 속도이다. 현재 색인 대상 데이터는 PDF 50건에서 추출한 총 17,345건으로, 이 데이터를 기반으로 각 환경의 성능 차이를 추정할 수 있다.

색인 속도

문서 데이터를 인덱스에 반영하는 데 걸리는 시간을 의미하며, 데이터 규모와 클러스터 구성에 따라 차이가 발생할 수 있다.

검색 속도

사용자가 쿼리를 요청했을 때 결과를 반환하는 속도를 의미하며, 데이터 노드 분산 구조, 요청 분배 방식, 네트워크 왕복 횟수 등의 영향을 받는다.

[8-3-1] 테스트 코드 설명

[1] 색인 속도 테스트 코드

FastAPI 에서 색인 작업을 활용하여 Elasticsearch 색인 속도를 평가한다. 테스트 코드는 MinutesIndexer 를 이용하여 PDF 50 건에서 추출한 총 17,345 건의 문서를 Elasticsearch 에 색인하며, batch_size 단위로 나누어 처리함으로써 대량 데이터 색인 시 메모리 부담을 줄이고 안정적인 수행이 가능하도록 설계되어 있다. 또한, 작업 시작과 종료 시간을 기록하여 색인에 걸린 총 시간과 처리량(docs/sec)을 계산하도록 구성되어 있어, 각 환경에서의 색인 성능을 정량적으로 평가하는 데 적합하다.

```
# [1] 시작 시간 기록
start_time = time.time()
print(f"\n [Start] 색인 작업을 시작합니다. (Batch Size: {batch_size})")

try:
    # 실제 색인 작업 실행
    # indexer가 처리한 문서 수(int)를 반환한다고 가정
    result = await indexer.index_all_documents(batch_size=batch_size)

    # result가 숫자가 아닐 경우를 대비한 안전장치
    count = result if isinstance(result, int) else 0

    # [2] 종료 시간 기록 및 계산
    end_time = time.time()
    duration = end_time - start_time
    throughput = count / duration if duration > 0 else 0

    # [3] 결과 로그 출력 (터미널에서 바로 확인 가능)
    print(f"-----")
    print(f"✅ [Complete] 색인 작업 완료")
    print(f" - 총 문서 수 : {count} 건")
    print(f" - 소요 시간   : {duration:.2f} 초")
    print(f" - 처리량      : {throughput:.2f} docs/sec")
    print(f"-----\n")
```

[그림-33]

[2] 검색 속도 테스트 코드

FastAPI 에서 구현한 검색 기능을 활용하여 Elasticsearch 의 검색 속도를 평가한다. 테스트 코드는 사용자 입력어 검색어를 SentenceTransformer 모델로 벡터화한 뒤, build_hybrid_query 를 통해 하이브리드 검색 쿼리를 생성하고, es_async.search 를 통해 Elasticsearch 에서 실제 검색을 수행한다. 검색 작업과 종료 시간을 기록하여 전체 수행 시간과 Elasticsearch 내부 처리 시간(took)을 모두 측정할 수 있으며, 검색 결과 건수와 하이라이트 텍스트를 함께 확인할 수 있도록 구성되어 있다.

```
# [1] 시작 시간 기록
start_time = time.time()

# 서비스 로직 실행
result = await service.search_minutes(q, committee, limit)

# [2] 종료 시간 기록 및 계산
end_time = time.time()
duration = end_time - start_time

if isinstance(result, dict):
    hit_count = result.get('total_hits', 0)
else:
    hit_count = getattr(result, 'total_hits', 'Unknown')

print(f"\n [Total Latency] 전체 소요 시간")
print(f" - 검색어      : '{q}'")
print(f" - 소요 시간   : {duration:.4f} sec")
print(f" - 결과 건수   : {hit_count}")
return result
```

[그림-34]

```
#####
사용자 검색어 -> 벡터 변환 -> ES 하이브리드 검색 -> 결과 변환
#####

start_time = time.time()

# 1. 검색어 벡터화 (여기가 시간이 꽤 걸릴 수 있음!)
query_vector = self.model.encode(keyword).tolist()

# 2. ES 쿼리 빌드
es_query = build_hybrid_query(keyword, query_vector, committee, limit)

# 3. ES 검색 요청
response = await es_async.search(index=INDEX_NAME, body=es_query)

# [추가] ES 내부 연산 시간 (단위: ms)
es_took_ms = response["took"]

# 4. 결과 가공
hits = response["hits"]["hits"]
results = []

for hit in hits:
    source = hit["_source"]
    score = hit["_score"]
    # highlight 처리 안전하게 (없을 경우 대비)
    highlight_field = hit.get("highlight", {}).get("content_text", [])
    highlight_text = highlight_field[0] if highlight_field else source["content_text"][:200]

    results.append({
        "score": score,
        "segment_id": source["segment_id"],
        "doc_id": source["doc_id"],
        "title": source["title"],
        "content_text": source["content_text"],
        "highlight": highlight_text,
        "speaker": f"({source['speaker_role']}) {source['speaker_name']}",
        "committee_name": source["committee_name"],
        "conf_date": source["conf_date"],
        "keywords": source["keywords"]
    })

# Python 총 소요 시간 (초 단위)
total_duration_sec = time.time() - start_time

# [로그 출력] 이렇게 찍어두면 디버깅할 때 범인 색출 가능
print(f"\n [Perf] Total: {total_duration_sec:.4f}s | ES Took: {es_took_ms}ms")

return {
    "total_hits": response["hits"]["total"]["value"],
    "execution_time": total_duration_sec, # 전체 수행 시간 (초)
    "es_took": es_took_ms, # ES 내부 시간 (밀리초)
    "results": results
}
```

[그림-35]

[8-3-2] 각 환경 테스트 결과

[1] 색인 속도 테스트

각 환경별로 총 3 회 반복 수행하였다. 테스트 결과는 [그림-36]과 같이 로그를 통해 확인할 수 있으며, 각 환경에서 얻은 결과를 비교하기 위해 표로 정리하였다. 이를 통해 단일 노드 환경과 클러스터 환경(1, 2)에서 색인 속도와 처리량의 차이를 직관적으로 파악할 수 있으며, 반복 수행으로 인해 발생할 수 있는 변동성을 최소화하여 비교하였다.

```

ssh.cloud.google.com/v2/ssh/projects/cogent-point-480114-t7/zones/us-central1-a/instances/single-node?authuser=0&hl=k...
ssh.cloud.google.com/v2/ssh/projects/cogent-point-480114-t7/zones/us-central1-a/instances/single-node?authuser=0&hl=k...
브라우저에서 SSH를 통해 연결
파일 업로드 파일 다운로드
500건 색인 완료 (누적 : 10500)
PUT http://10.128.0.6:9200/_bulk [status:200 duration:1.111s]
500건 색인 완료 (누적 : 11000)
PUT http://10.128.0.6:9200/_bulk [status:200 duration:1.132s]
500건 색인 완료 (누적 : 11500)
PUT http://10.128.0.6:9200/_bulk [status:200 duration:0.992s]
500건 색인 완료 (누적 : 12000)
PUT http://10.128.0.6:9200/_bulk [status:200 duration:0.931s]
500건 색인 완료 (누적 : 12500)
PUT http://10.128.0.6:9200/_bulk [status:200 duration:0.960s]
500건 색인 완료 (누적 : 13000)
PUT http://10.128.0.6:9200/_bulk [status:200 duration:1.105s]
500건 색인 완료 (누적 : 13500)
PUT http://10.128.0.6:9200/_bulk [status:200 duration:0.817s]
500건 색인 완료 (누적 : 14000)
PUT http://10.128.0.6:9200/_bulk [status:200 duration:0.693s]
500건 색인 완료 (누적 : 14500)
PUT http://10.128.0.6:9200/_bulk [status:200 duration:0.746s]
500건 색인 완료 (누적 : 15000)
PUT http://10.128.0.6:9200/_bulk [status:200 duration:0.750s]
500건 색인 완료 (누적 : 15500)
PUT http://10.128.0.6:9200/_bulk [status:200 duration:0.831s]
500건 색인 완료 (누적 : 16000)
PUT http://10.128.0.6:9200/_bulk [status:200 duration:0.949s]
500건 색인 완료 (누적 : 16500)
PUT http://10.128.0.6:9200/_bulk [status:200 duration:1.072s]
500건 색인 완료 (누적 : 17000)
PUT http://10.128.0.6:9200/_bulk [status:200 duration:0.667s]
345건 색인 완료 (누적 : 17345)
[Indexer] 전체 완료. 총 17345건 색인됨.
[Complete] 색인 작업 완료
- 총 문서 수 : 17345 건
- 소요 시간 : 71.57 초
- 처리량 : 242.37 docs/sec
INFO: 223.39.206.130:56218 - "GET /api/v1/users/indexing_status HTTP/1.1" 200 OK

```

[그림-36]

[1-1] 단일 노드 색인 테스트 결과

단일 노드 환경에서 PDF 50 건에서 추출한 총 17,345 건을 색인하는 테스트를 각 환경별로 3 회 반복 수행하였다. 각 회차의 결과는 다음과 같다.(batch_size=500)

회차	총 문서 수	소요 시간 (초)	처리량 (docs/sec)
1	17,345	71.57	242.37
2	17,345	74.28	233.52
3	17,345	74.56	232.62

[1-2] 클러스터(1) 색인 테스트 결과

클러스터 환경(1)에서도 동일하게 PDF 50 건에서 추출한 총 17,345 건을 색인하는 작업을 3 회 반복 수행하였다. 각 회차의 결과는 다음과 같다.(batch_size=500)

회차	총 문서 수	소요 시간 (초)	처리량 (docs/sec)
1	17,345	117.01	148.24
2	17,345	96.44	179.86
3	17,345	92.70	187.10

[1-3] 클러스터(2) 색인 테스트 결과

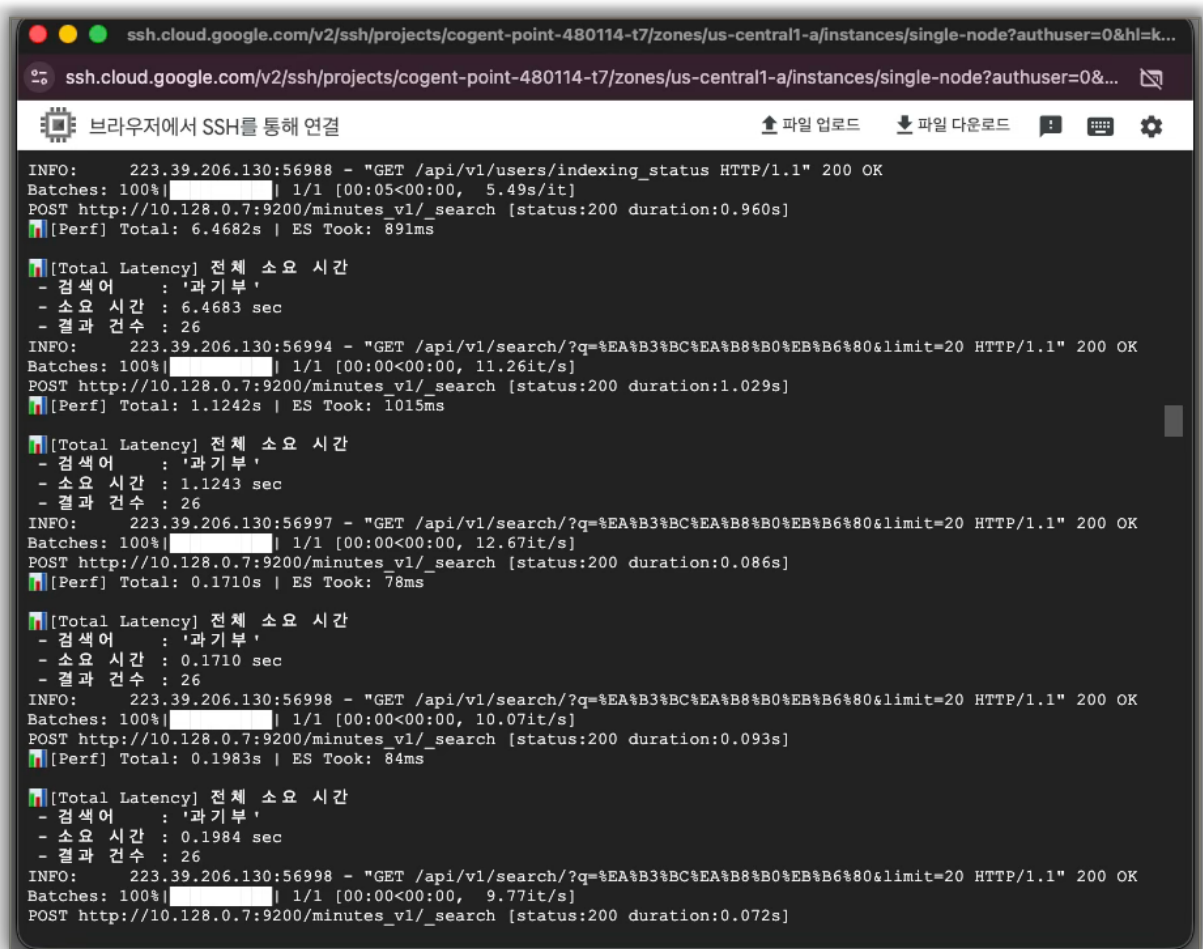
클러스터 환경(1)에서도 동일하게 PDF 50 건에서 추출한 총 17,345 건을 색인하는 작업을 3 회 반복 수행하였다. 각 회차의 결과는 다음과 같다.(batch_size=500)

회차	총 문서 수	소요 시간 (초)	처리량 (docs/sec)
1	17,345	123.27	140.71
2	17,345	115.06	150.74
3	17,345	101.96	170.12

[2] 검색 속도 테스트

색 속도 테스트는 크게 두 가지 반복 조건으로 수행되었다. 첫 번째는 동일 검색어 반복 실험으로, 특정 검색어(예: “과기부”)를 사용하여 1회차 검색 후 2~11회차까지 반복 수행하였다. 이를 통해 검색어 캐싱, 벡터 변환, Elasticsearch 내부 검색 처리 등의 반복 수행 성능 변화를 평가할 수 있다.

두 번째는 다른 검색어 반복 실험으로, 서로 다른 검색어를 순차적으로 입력하여 검색 속도를 측정하였다. 사용된 검색어 순서는 “과기부 → 국방부 → 환경부 → 가방 → 대학생 → 쓰레기 → 건물 → 인권 → 사랑 → 졸업”이며, 이를 통해 검색어 특성에 따른 검색 성능과 전체 검색 처리 시간의 변화를 확인할 수 있다.



```
ssh.cloud.google.com/v2/ssh/projects/cogent-point-480114-t7/zones/us-central1-a/instances/single-node?authuser=0&hl=k...
ssh.cloud.google.com/v2/ssh/projects/cogent-point-480114-t7/zones/us-central1-a/instances/single-node?authuser=0&...
브라우저에서 SSH를 통해 연결
파일 업로드
파일 다운로드
[Perf] Total: 6.4682s | ES Took: 891ms
[Total Latency] 전체 소요 시간
- 검색어 : '과기부'
- 소요 시간 : 6.4683 sec
- 결과 건수 : 26
INFO: 223.39.206.130:56994 - "GET /api/v1/users/indexing_status HTTP/1.1" 200 OK
Batches: 100% | 1/1 [00:05<00:00, 5.49s/it]
POST http://10.128.0.7:9200/minutes_v1/_search [status:200 duration:0.960s]
[Perf] Total: 1.1242s | ES Took: 1015ms
[Total Latency] 전체 소요 시간
- 검색어 : '과기부'
- 소요 시간 : 1.1243 sec
- 결과 건수 : 26
INFO: 223.39.206.130:56997 - "GET /api/v1/search/?q=%EA%B3%BC%EA%B8%B0%EB%B6%80&limit=20 HTTP/1.1" 200 OK
Batches: 100% | 1/1 [00:00<00:00, 12.67it/s]
POST http://10.128.0.7:9200/minutes_v1/_search [status:200 duration:0.086s]
[Perf] Total: 0.1710s | ES Took: 78ms
[Total Latency] 전체 소요 시간
- 검색어 : '과기부'
- 소요 시간 : 0.1710 sec
- 결과 건수 : 26
INFO: 223.39.206.130:56998 - "GET /api/v1/search/?q=%EA%B3%BC%EA%B8%B0%EB%B6%80&limit=20 HTTP/1.1" 200 OK
Batches: 100% | 1/1 [00:00<00:00, 10.07it/s]
POST http://10.128.0.7:9200/minutes_v1/_search [status:200 duration:0.093s]
[Perf] Total: 0.1983s | ES Took: 84ms
[Total Latency] 전체 소요 시간
- 검색어 : '과기부'
- 소요 시간 : 0.1984 sec
- 결과 건수 : 26
INFO: 223.39.206.130:56998 - "GET /api/v1/search/?q=%EA%B3%BC%EA%B8%B0%EB%B6%80&limit=20 HTTP/1.1" 200 OK
Batches: 100% | 1/1 [00:00<00:00, 9.77it/s]
POST http://10.128.0.7:9200/minutes_v1/_search [status:200 duration:0.072s]
```

[그림-37]

[2-1] 단일 노드 검색 속도 테스트 결과(단위: 초)

시나리오	검색어	회차	총 소요 시간 (Total)	ES Took	결과 건수 (hits)	비고
Cold Start	과기부	1	1.7387	114	26	초기 검색, 모델/캐시 로딩 포함
반복 검색	과기부	2	0.2150	52	26	캐시 적중
반복 검색	과기부	3	0.1729	45	26	캐시 적중
반복 검색	과기부	4	0.1459	52	26	캐시 적중
반복 검색	과기부	5	0.1713	45	26	캐시 적중
반복 검색	과기부	6	0.1668	48	26	캐시 적중
반복 검색	과기부	7	0.1770	73	26	캐시 적중
반복 검색	과기부	8	0.1506	51	26	캐시 적중
반복 검색	과기부	9	0.1608	48	26	캐시 적중
반복 검색	과기부	10	0.2045	64	26	캐시 적중
반복 검색	과기부	11	0.1623	50	26	캐시 적중
다른 검색어	국방부	1	0.3365	96	92	새로운 검색어
다른 검색어	환경부	1	0.2811	160	23	새로운 검색어
다른 검색어	가방	1	0.2098	98	20	새로운 검색어
다른 검색어	대학생	1	0.2776	90	20	새로운 검색어
다른 검색어	쓰레기	1	0.1970	79	20	새로운 검색어
다른 검색어	건물	1	0.1815	64	27	새로운 검색어
다른 검색어	인권	1	0.2297	106	42	새로운 검색어
다른 검색어	사랑	1	0.1979	74	21	새로운 검색어
다른 검색어	졸업	1	0.1616	57	24	새로운 검색어

[2-2] 클러스터(1) 검색 속도 테스트 결과 (단위: 초)

시나리오	검색어	회차	총 소요 시간 (Total)	ES Took	결과 건수 (hits)	비고
Cold Start	과기부	1	6.4683	891	26	초기 검색, 모델/캐시 로딩 포함
반복 검색	과기부	2	1.1243	1015	26	
반복 검색	과기부	3	0.1710	78	26	캐시 적중
반복 검색	과기부	4	0.1984	84	26	캐시 적중
반복 검색	과기부	5	0.1811	65	26	캐시 적중
반복 검색	과기부	6	0.1529	67	26	캐시 적중
반복 검색	과기부	7	0.1474	53	26	캐시 적중
반복 검색	과기부	8	0.1782	67	26	캐시 적중
반복 검색	과기부	9	0.1858	62	26	캐시 적중
반복 검색	과기부	10	0.1600	65	26	캐시 적중
반복 검색	과기부	11	0.1342	47	26	캐시 적중
다른 검색어	국방부	1	0.1671	66	93	새로운 검색어
다른 검색어	환경부	1	0.1659	48	23	새로운 검색어
다른 검색어	가방	1	0.1913	80	20	새로운 검색어
다른 검색어	대학생	1	0.1420	45	20	새로운 검색어
다른 검색어	쓰레기	1	0.1381	48	20	새로운 검색어
다른 검색어	건물	1	0.1339	44	27	새로운 검색어
다른 검색어	인권	1	0.1508	61	42	새로운 검색어
다른 검색어	사랑	1	0.1462	55	21	새로운 검색어
다른 검색어	졸업	1	0.1513	55	24	새로운 검색어

[2-3] 클러스터(2) 검색 속도 테스트 결과 (단위: 초)

시나리오	검색어	회차	총 소요 시간 (Total)	ES Took	결과 건수 (hits)	비고
Cold Start	과기부	1	5.5442	1751	26	초기 검색, 모델/캐시 로딩 포함
반복 검색	과기부	2	1.5113	1395	26	
반복 검색	과기부	3	0.2116	85	26	캐시 적중
반복 검색	과기부	4	0.2277	103	26	캐시 적중
반복 검색	과기부	5	0.1953	77	26	캐시 적중
반복 검색	과기부	6	0.1993	78	26	캐시 적중
반복 검색	과기부	7	0.2245	72	26	캐시 적중
반복 검색	과기부	8	0.2002	72	26	캐시 적중
반복 검색	과기부	9	0.1956	78	26	캐시 적중
반복 검색	과기부	10	0.1964	75	26	캐시 적중
반복 검색	과기부	11	0.1673	52	26	캐시 적중
다른 검색어	국방부	1	0.2127	88	92	새로운 검색어
다른 검색어	환경부	1	0.2069	68	24	새로운 검색어
다른 검색어	가방	1	0.1850	63	20	새로운 검색어
다른 검색어	대학생	1	0.1806	49	20	새로운 검색어
다른 검색어	쓰레기	1	0.1849	63	20	새로운 검색어
다른 검색어	건물	1	0.1682	52	27	새로운 검색어
다른 검색어	인권	1	0.2671	64	42	새로운 검색어
다른 검색어	사랑	1	0.1742	58	21	새로운 검색어
다른 검색어	졸업	1	0.1846	54	24	새로운 검색어

[8-4] 분석

[8-4-1] 색인 속도 테스트 분석

단일 노드 환경과 두 가지 클러스터 환경(클러스터(1): Master/Data/Ingest 통합형, 클러스터(2): Master-Data 역할 분리형)을 대상으로 총 17,345건의 문서 색인을 3회 반복 수행하여 처리량과 구조적 효율성을 비교하였다.

단일 노드 환경에서는 네트워크 통신이 존재하지 않아 로컬 처리의 이점이 극대화되어 가장 우수한 색인 성능을 보였다. 회차별 처리량은 232~242 docs/sec 범위를 기록하였으며, 이는 데이터 분산이나 노드 간 복제 과정 없이 CPU와 디스크 I/O를 온전히 색인 작업에만 집중할 수 있었기 때문이다. 반복 수행시에도 처리 속도가 매우 일정하게 유지되며, 소규모 데이터 환경에서는 단일 노드가 가장 효율적이고 안정적인 색인 성능을 제공함을 확인하였다.

클러스터(1)은 단일 노드와 달리 데이터 분산 처리가 필요한 구조에서 테스트를 진행하였다. 초기 색인 속도가 다소 느렸으나, 반복 수행에 따라 점차 개선되어 148~187 docs/sec 범위를 기록하였다. 단일 노드 대비 처리량이 낮은 이유는 인덱싱 요청을 각 샤드가 위치한 노드로 분배하고 복제본을 동기화하는 과정에서 네트워크 및 I/O 오버헤드가 발생했기 때문이다. 그러나 회차가 거듭될수록 JVM 워밍업 및 캐싱 효과가 나타나며 성능이 안정화되는 패턴을 보였다.

클러스터(2)는 역할 분리에 따른 구조적 특징이 색인 성능에 직접적인 영향을 미쳤다. 테스트 결과 처리량은 140~170 docs/sec 범위로 측정되어 클러스터(1)보다 소폭 낮은 결과를 보였다. 이는 node-1이 Master 전용 노드로 설정되어 실제 색인 작업에서 제외됨에 따라 가용 리소스가 축소되었고, Ingest 및 Data 역할이 나머지 노드에 집중되면서 부하가 가중되었기 때문이다. 또한, Master 노드의 조정 절차와 노드 간 데이터 이동 경로가 구조적 오버헤드 역시 성능 저하의 원인으로 분석된다.

결론적으로 단일 노드는 네트워크 비용 없이 소규모 데이터의 빠른 색인에 가장 적합하다. 클러스터(1)은 초기 속도는 느리지만 리소스를 효율적으로 사용하여 반복 작업 시 준수한 성능을 보여준다. 반면 클러스터(2)는 당장의 색인 처리량은 낮지만, Master 노드를 데이터 처리 부하로부터 격리함으로써 시스템 전체의 안정성과 장애 허용 능력을 확보하는 데 최적화된 아키텍처임을 확인하였다.

[8-4-2] 검색 속도 테스트 분석

단일 노드 환경과 두 가지 클러스터 환경(클러스터(1): Master/Data/Ingest 통합형, 클러스터(2): Master-Data 역할 분리형)을 대상으로 Cold Start, 동일 검색어 반복, 다른 검색어 반복이라는 세 가지 시나리오를 수행하여 검색 지연(Latency)과 안정성(Cache 적중 효과)을 비교하였다.

단일 노드 환경에서는 네트워크 통신이 존재하지 않아 초기 응답 성능이 가장 우수했다. Cold Start에서 1.74초의 전체 소요 시간을 기록하였고, 이는 모델 초기 로딩과 디스크 I/O가 포함된 결과임에도 불구하고 분산 오버헤드가 없는 구조적 장점 덕분이었다. 이후 반복 검색 시에는 데이터와 모델이 메모리에 적재된 상태로 실행되어 0.14~0.21초 수준으로 대폭 단축되었으며, 다른 검색어를 입력한 경우에도 0.16~0.34초로 빠른 응답 속도를 유지하였다. 이를 통해 단일 노드는 소규모 데이터 환경에서 매우 효율적인 검색 성능을 제공함을 확인하였다.

클러스터(1)은 초기 Cold Start에서 6.47로 가장 느린 결과를 보였는데, 이는 모델 로딩 외에도 클러스터 메타데이터 동기화, 분산 쿼리 브로드캐스트 등 초기 오버헤드가 누적된 결과였다. 그러나 반복 검색 단계에서는 0.13~0.20초로 빠르게 안정화되어 단일 노드와 대등하거나 일부 구간에서는 더 빠른 결과를 보였다. 또한 다양한 검색어 반복 시에도 0.13~0.17초 범위에서 매우 안정적인 응답 속도를 유지하여, 클러스터 구조가 데이터 분산과 JVM 워밍업 이후의 처리 효율에서 강점을 가진다는 점을 확인할 수 있었다.

클러스터(2)는 역할 분리에 따른 특징이 보다 명확히 드러났다. Cold Start는 5.54초로 측정되어 클러스터(1)보다 약 0.9초 빠르게 시작되었는데, 이는 Master 노드가 데이터 I/O 부하에서 분리되어 클러스터 관리 기능에 집중할 수 있었기 때문이다. 다만 ES Took는 1.75초로 가장 높았던 것은 Master에서 Data 노드로 쿼리를 전달하고 다시 취합하는 과정의 비용이 반영된 것으로 볼 수 있다. 반복 검색의 경우에는 Master → Data 노드 간의 네트워크 홉과 라우팅 오버헤드가 존재해 0.17~0.23초로 단일 노드 및 클러스터(1) 대비 소폭 느린 결과를 보였지만, 다양한 검색어에서도 0.17~0.27초로 안정적인 범위를 유지하였다.

결론적으로 단일 노드는 응답 속도가 가장 중요한 소규모 데이터 환경에 적합하다. 클러스터(1)는 초기 지연은 크지만 반복 검색과 다양한 검색 패턴에서 가장 높은 효율을 보이는 구조이다. 반면 클러스터(2)는 개별 검색 요청은 소폭 느리지만 Master와 Data 역할이 명확히 분리되어 있어 대규모 트래픽이나 확장성이 필요한 운영 환경에서 가장 유리한 아키텍처이다.

[8-5] 최종 선택

Elasticsearch의 단일 노드 환경과 클러스터 환경을 대상으로 색인 성능과 검색 속도를 비교한 결과, 두 환경은 명확한 특성과 장단점을 보였다. 단일 노드는 소규모 문서 처리 환경에서 Cold Start 상황에서 가장 빠른 응답 시간을 제공하며, 전반적으로 안정적인 성능을 유지하였다. 이에 비해 클러스터 환경은 초기 색인 속도에서 다소 지연이 있었으나, 반복 작업이 누적될수록 처리량이 점차 안정화되는 양상을 보였다.

이 프로젝트에서는 22대 국회 회의록 PDF 1,000건 이상을 색인하는 실제 운영 환경을 가정하며, 테스트에 사용된 50건의 PDF(총 17.345건 데이터)는 실제 예상 데이터 규모 대비 매우 작은 수준이다. 향후 데이터가 수십만~백만 단위로 확장될 가능성이 높기 때문에, 색인 과정에서 성능 저하와 장애 위험이 존재하는 단일 노드보다는 노드 간 분산 구조를 갖춘

클러스터 환경이 더 적합하다. 특히 Elasticsearch가 제공하는 Zero-Downtime Reindexing 구조는 단일 노드보다 클러스터 구성에서 더 높은 안정성을 확보할 수 있으며, 서비스 중단 없이 색인 교체를 수행할 수 있다는 점 역시 핵심 고려 요소였다.

비용 측면에서는 단일 노드와 클러스터의 차이는 명확했다. 테스트에 사용된 VM 단가 기준으로 단일 노드는 약 25.46달러로 운영이 가능하지만, 클러스터 환경은 VM 3대를 필요로 하므로 약 76.38달러로 증가한다. 여기에 Cloud SQL 비용과 관리용 VM까지 포함하면 전체 비용은 단일 노드 대비 약 2.5~3배 수준으로 상승한다. 비용만 고려한다면 단일 노드가 훨씬 유리하지만, 본 시스템은 단순 테스트용이 아니라 실제 서비스 운영을 전제로 한다. 검색 서비스 특성상 특정 시점에 트래픽이 집중될 수 있으며, 서비스 중단은 사용자 경험과 신뢰에 큰 영향을 미치기 때문에 고가용성과 안정성 확보가 필수적이다.

이와 같은 요소를 종합적으로 검토한 결과, 장기적인 서비스 품질과 확장성을 고려할 때 단일 노드보다 클러스터 환경이 훨씬 적합하다고 판단하였다. 특히 고가용성, 동시 요청 처리 능력, 안정적인 검색 품질, 대규모 색인 처리 등 운영 환경에서 필요한 요구사항을 충족하는 점을 기준으로 최종적으로 Elasticsearch 클러스터(1) 구성을 선택하였다.

[9] Elasticsearch 클러스터 기반 Full-Stack GCP 설계

[9-1] 시스템 구성

구성 요소	호스트명	주요 역할	역할
Web server	VM5	Reverse Proxy & Static Web Server	Nginx 기반 HTML 서빙 및 API 라우팅
WAS	VM1	Backend API Server	FastAPI 구동, Cloud SQL Proxy 연결
Elasticsearch	VM2	Main Node (Master/Data/Ingest)	클러스터 관리 및 요청 코디네이팅
Elasticsearch	VM3	Data Node(Master Eligible)	데이터 분산 저장 및 백업 마스터
Elasticsearch	VM4	Data Node(Master Eligible)	데이터 분산 저장 및 백업 마스터
DB	Cloud SQL	RDBMS(MySQL)	원본 회의록 및 정형 메타데이터 저장

[9-2] 웹 서버 및 프론트엔드 설계(VM5)

사용자의 접근성을 보장하고 백엔드 서버를 보호하기 위해, VM5에 Docker 기반의 Nginx를 배치하여 웹 서버 및 리버스 프록시 역할을 수행하도록 설계하였다.

[1] 정적 웹 호스팅

- 경량화된 프론트엔드: 복잡한 CSR(Client Side Rendering) 프레임워크 대신, 순수 HTML5/CSS/JavaScript 기반의 정적 페이지를 채택하여 초기 렌더링 속도를 극대화하였다.
- Docker Volume 마운트: 호스트의 HTML 파일을 컨테이너 내부로 마운트하여, 소스 코드 수정 시 이미지를 다시 빌드하지 않고도 즉시 반영될 수 있도록 구성하였다.
- Nginx 서빙: 사용자가 `http://<VM5_Public_IP>/`로 접속 시, Nginx가 즉시 정적 파일을 반환하여 WAS(VM1)의 부하를 줄인다.

[2] 리버스 프록시 및 API 라우팅

Nginx를 프론트엔드와 백엔드 사이의 단일 진입점으로 구성하여 보안 및 CORS 문제를 해결하였다.

- URL 기반 라우팅

- 보안 아키텍처: API 서버(VM1)는 외부망(Public IP)에 직접 포트를 개방하지 않고, 오직 내부망 IP로만 Nginx(VM5)와 통신한다. 이를 통해 백엔드 서버의 직접적인 노출을 차단하여 보안성을 강화하였다.

- 헤더 포워딩: X-Real-IP, X-Forwarded-For 헤더 설정을 통해, 프록시를 거치더라도 FastAPI가 실제 클라이언트의 IP를 식별하여 로그를 기록할 수 있도록 설계하였다.

```
nginx.conf
1  server {
2      listen 80;
3      server_name _;
4
5      # 1. API 요청 프록시 (FastAPI)
6      location /api/ {
7          proxy_pass http://10.128.0.5:8000/; # FastAPI 내부 IP
8          proxy_set_header Host $host;
9          proxy_set_header X-Real-IP $remote_addr;
10         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
11         proxy_set_header X-Forwarded-Proto $scheme;
12     }
13
14     # 2. 정적 HTML
15     location / {
16         root /usr/share/nginx/html;
17         index index.html;
18         try_files $uri $uri/ =404;
19     }
20 }
```

[그림-38]

[9-3] 로드밸런싱 및 오토스케일링 아키텍처 설계

시스템의 트래픽 증가에 유연하게 대응하고, 단일 장애점을 제거하기 위해 GCP Cloud Load Balancing과 Managed Instance Group(MIG)을 결합한고가용성 아키텍처를 설계하였다.

[1] 부하 분산 전략(Global HTTP Load Balancing)

들어오는 모든 사용자 트래픽은 Layer7 기반의 전역 애플리케이션 로드 밸런서를 통해 처리된다.

우선 단일 진입점을 제공하여 전 세계 어디서든 동일한 Anycast IP로 접속할 수 있도록 구성하며, URL 경로 기반 라우팅을 통해 요청을 적절한 백엔드 서비스로 분기한다.

/ 요청은 정적 리소스를 제공하는 Frontend(Nginx) 그룹으로, /api/* 요청은 비즈니스 로직을 처리하는 Backend(FastAPI) 그룹으로 라우팅된다.

[2] 오토스케일링(Autoscaling) 설계

검색 요청이 급증하는 상황에서도 안정적인 성능을 확보하기 위해 백엔드(FastAPI)에 자동 확장 정책을 적용하였다. 우선 VM1(FastAPI)의 상태를 스냅샷으로 저장하여 사용자 지정 이미지를 생성하고, 이를 기반으로 인스턴스 템플릿과 관리형 인스턴스 그룹(MIG)을 구성한다. 확장 정책은 CPU 사용률이 60%를 초과하면 자동으로 인스턴스를 추가하도록 설정하고, 최소 2대의 인스턴스를 상시 유지하여 기본적인 가용성을 확보하며, 트래픽 폭주 시 최대 10대까지 확장될 수 있도록 제한을 설정하였다. 이 프로젝트가 22대 국회 회의록 PDF 1,000건 이상을 정규적으로 색인하고 실시간 검색을 제공하는 운영 환경을 전제로 하기 때문이다. 검색 서비스는 검색어의 벡터화, 결과 재정렬 등에서 CPU-bound 특성을 가지므로, CPU 사용률이 100%에 근접한 후 확장을 시도하면 이미 처리 중인 요청이 지연되거나 타임아웃이 발생할 위험이 크다. 따라서 CPU 60%를 임계치로 설정해 과부하 이전에 선제적으로 확장을 수행하도록 하였으며, 단일 인스턴스 장애로 전체 서비스가 중단되는 단일 장애점을 방지하기 위해 최소 2대를 유지하여 한 대가 장애가 발생하더라도 즉시 트래픽을 이어받을 수 있도록고가용성을 확보하였다.

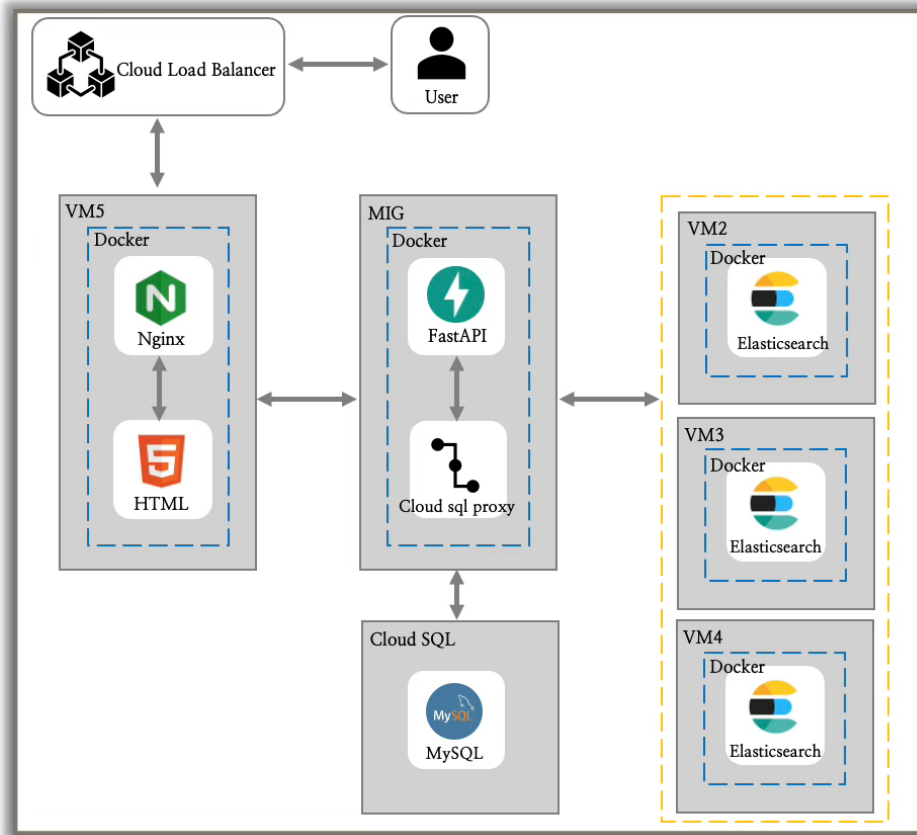
[3] 장애 감지 및 복구(Health Check & Fault Tolerance)

서비스 중단 없는 운영을 위해 로드 밸런서 수준에서 지속적으로 인스턴스 상태를 확인한다. Health Check은 Backend의 /health(Port 8000)와 Frontend의 Port 80을 주기적으로 호출하며, 응답이 없거나 오류가 발생하는 인스턴스는 즉시 트래픽 대상에서 제거된다. 이후 MIG가 자동 치유를 수행하여 문제 있는 인스턴스를 정상 상태의 새 인스턴스로 자동 교체한다.

[4] 네트워크 보안 설계

서버 보안을 강화하기 위해 인스턴스의 직접 노출을 최소화하고, Backend 인스턴스는 로드 밸런서를 경유한 트래픽만 허용하는 Private 통신 원칙을 따른다. 또한 방화벽 규칙을 통해 GCP 로드 밸런서 상태 확인 프록시 IP 대역인 130.211.0.0/22 와 25.191.0.0/16 만 접근을 허용하며, 이 외의 외부 직접 접근은 모두 차단하여 보안성을 강화하였다.

[9-4] 아키텍처



[그림-39]

[10] Elasticsearch 클러스터 기반 Full-Stack GCP 구현

[10-1] Web Server(Nginx) 구축 및 Reverse Proxy 연결

앞서 4-2 에서 설계한 정적 웹 호스팅 및 리버스 프록시 보안 아키텍처를 구현하기 위해, VM5 인스턴스에 Docker 기반 Nginx 웹 서버를 배포하고 내부 네트워크를 통해 WAS(VM1)와 연결하였다.

VM 인스턴스

인스턴스 만들기

VM 가져오기

새로고침

인스턴스

관측 가능성

인스턴스 일정

VM 인스턴스

필터 속성 이름 또는 값 입력

☐ 상태	이름 ↑	영역	권장사항	다음에서 사용 중:	내부 IP	외부 IP	연결
☐	elasticsearch	us-central1-a			es-internal-ip (10.128.0.6) (nic0)		SSH ▾ ⋮
☐	node-1	us-central1-a			10.128.0.7 (nic0)	34.66.137.193 (nic0)	SSH ▾ ⋮
☐	node-2	us-central1-a			10.128.0.8 (nic0)	104.154.187.37 (nic0)	SSH ▾ ⋮
☐	node-3	us-central1-a			10.128.0.9 (nic0)	34.61.255.121 (nic0)	SSH ▾ ⋮
☐	single-node	us-central1-a			10.128.0.5 (nic0)	34.30.245.171 (nic0)	SSH ▾ ⋮
☐	webserver	us-central1-a			10.128.0.16 (nic0)	34.29.254.187 (nic0)	SSH ▾ ⋮

[그림-40]

[1] VM 네트워크 구성 및 방화벽 설정

웹 서버(VM5)가 외부 사용자의 HTTP 요청을 수신할 수 있도록 GCP VPC 방화벽 규칙을 구성하였다.

- 대상 인스턴스: VM5(Web Server)
- 허용 포트: TCP 80(HTTP)

VM5는 시스템의 유일한 공개 진입점 이므로 80번 포트만 개방하였으며, 백엔드 WAS(VM1)는 외부 접근을 차단하고 내부망을 통한 요청만 허용하여 보안성을 강화하였다.

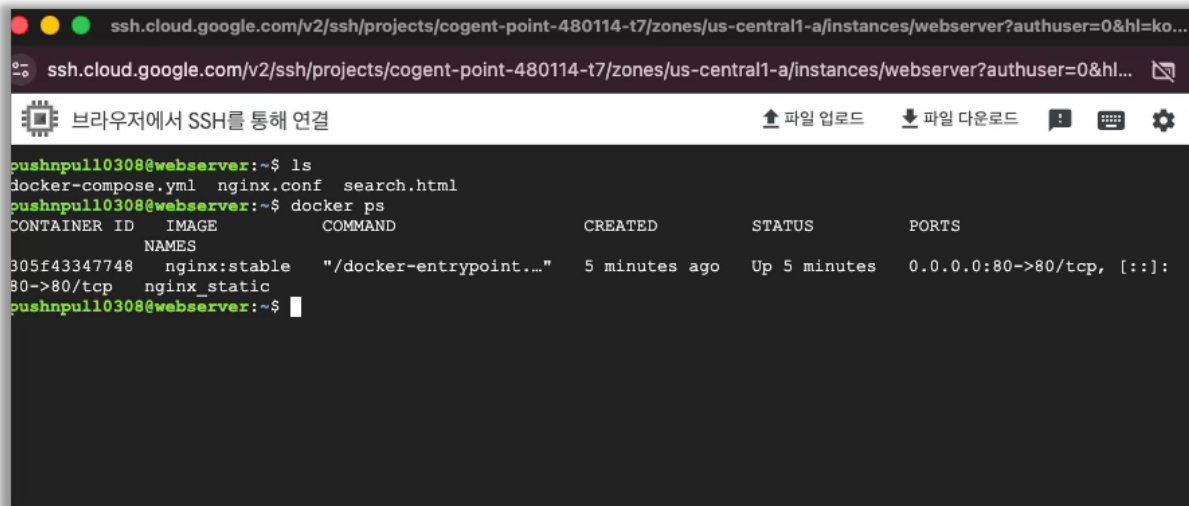
[2] Docker 기반 서비스 배포 환경 구성

유지보수 편의성과 배포 속도를 높이기 위해 Docker Compose 기반으로 Nginx 서버 환경을 구축하였다.

[3] Nginx Reverse Proxy 및 라우팅 구현

nginx.conf 파일을 통해 URL 경로 기반의 Layer7 라우팅을 적용하여 정적 리소스와 API 요청을 분리 처리하였다.

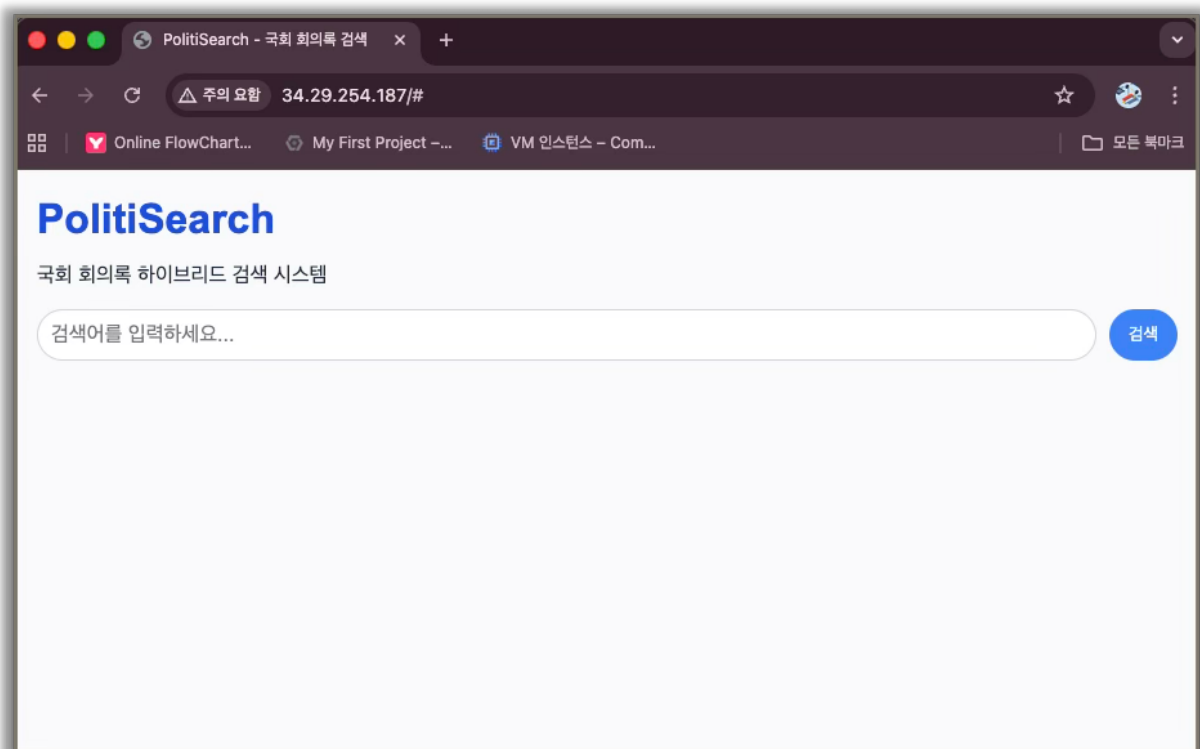
- API 요청 프록시 /api/
- 정적 파일 서빙 /



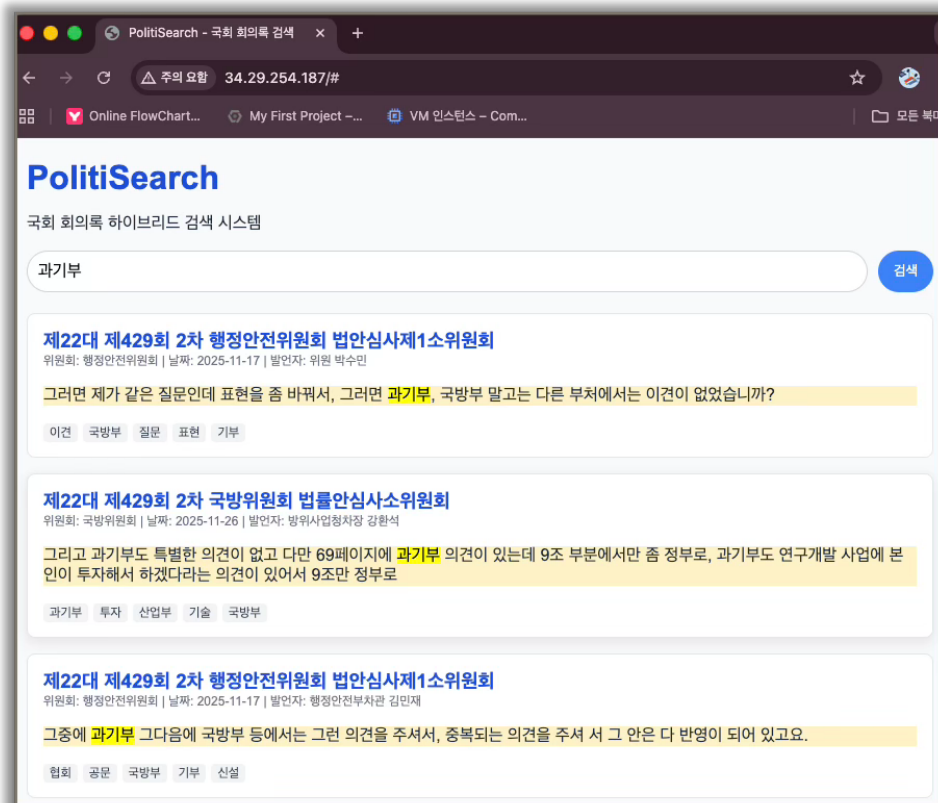
[그림-41]

[4] 구현 결과

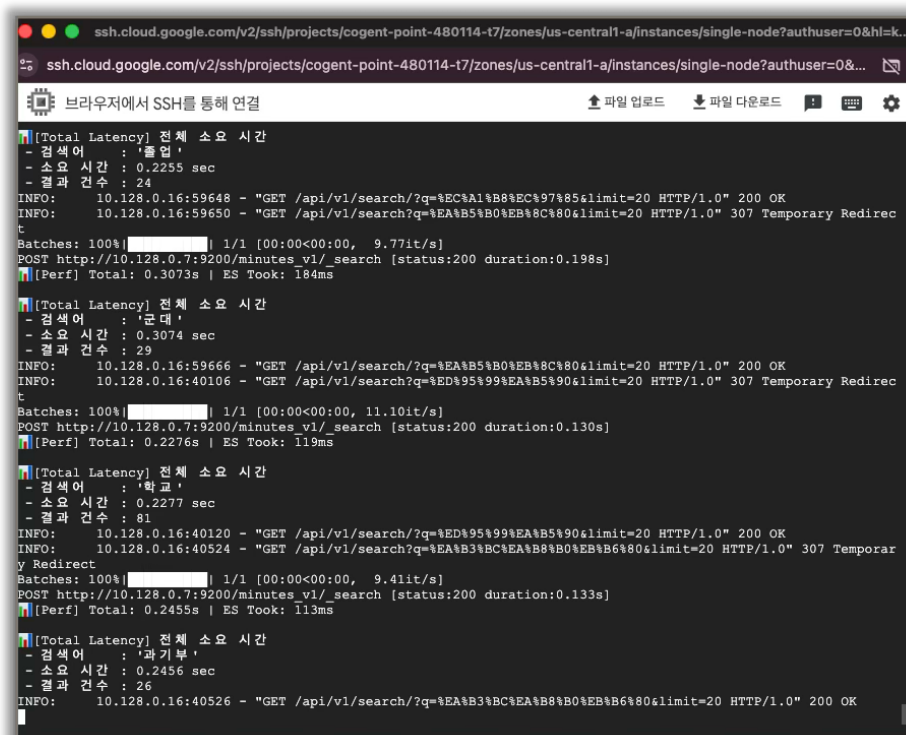
- 단일 진입점 확보: 사용자는 VM5의 공인 IP 하나로 정적 웹 페이지 접속과 API 기반 검색 기능을 모두 이용할 수 있게 되었다.
- 보안성 강화: 브라우저 개발자 도구에서 API 요청 경로를 확인한 결과, 백엔드(VM1)의 내부 IP는 외부에 노출되지 않고 VM5(Nginx)가 응답하는 구조가 정상적으로 동작하여 IP 은닉 효과가 검증되었다.



[그림-42]



[그림-43]



[그림-44]

[10-2] 로드밸런스

[10-2-1] FastAPI 백엔드 VM 을 사용자 지정 이미지로 생성

관리형 인스턴스 그룹(MIG) 기반 자동 확장을 위해서는, 신규 인스턴스가 동일한 실행 환경을 즉시 갖출 수 있도록 사전 구성된 사용자 지정 이미지가 필요하다. 매번 인스턴스 생성 시 Docker 및 런타임 환경을 재설치할 경우 초기 부팅 시간이 길어지고, 확장 시점에서 자연이 발생할 수 있으므로 현재 설정이 완료된 VM1을 이미지로 생성하였다.

이미지 생성 절차

1. GCP 콘솔 -> Compute Engine -> 스토리지 -> 이미지 메뉴 이동
2. [이미지 만들기] 클릭
3. 다음 항목을 설정하여 이미지 생성
 - 이름: fastapi-backend-image-v1
 - 소스 유형: 디스크
 - 소스 디스크: FastAPI 백엔드가 정상 동작 중인 VM1의 부트 디스크
4. 만들기 버튼 클릭하여 이미지 생성

VM1 을 실행 중인 상태에서도 이미지 생성이 가능하지만, 데이터 정합성과 안정성을 위해 일시적으로 중지한 상태에서 수행하는 것이 권장된다.



[그림-45]

[10-2-2] 인스턴스 템플릿 생성

인스턴스 템플릿은 앞서 생성한 사용자 지정 이미지를 기반으로, 새로운 VM 인스턴스가 어떤 사양과 설정으로 가동될지 정의하는 단계이다. 이 템플릿은 이후 관리형 인스턴스 그룹(MIG)에서 자동으로 인스턴스를 생성·확장하는 기준으로 사용된다.

생성 절차

1. GCP 콘솔 -> Compute Engine -> 인스턴스 템플릿 이동
2. [인스턴스 템플릿 만들기] 클릭
3. 다음 항목을 설정하여 템플릿을 구성
 - 이름: fastapi-template
 - 머신 유형: e2-medium
(기존 FastAPI 서버 VM1과 동일한 사양)
 - 부트 디스크 변경
 - [맞춤 이미지] 탭 선택
 - fastapi-backend-image-v1 이미지 지정
 - 고급 옵션 -> 네트워킹
 - 고급 옵션 -> 관리 -> 시작 스크립트
 - 인스턴스 가동 시 FastAPI 서비스를 자동 실행하기 위한 명령어 설정
 - 이미지에 필요한 파일과 환경이 이미 포함되어 있으므로, cd <경로> && docker compose up -d --build 수준의 명령어만으로 서비스 실행이 가능하다.
4. [만들기] 버튼 클릭하여 인스턴스 템플릿 생성



[그림-46]

[10-2-3] 관리형 인스턴스 그룹(MIG) 생성

관리형 인스턴스 그룹(Managed Instance Group, MIG)은 동일한 환경의 VM 인스턴스를 하나의 논리적 그룹으로 묶어 자동 확장, 장애 복구, 부하 분산을 수행하기 위한 구성 요소이다.

본 단계에서는 실제 사용자 트래픽을 처리할 FastAPI 백엔드 서버 집합을 생성한다.

생성 절차

1. GCP 콘솔 -> Compute Engine -> 인스턴스 그룹 이동
2. [인스턴스 그룹 만들기] 클릭
3. 새 관리형 인스턴스 그룹(Stateless) 선택
4. 다음 항목을 설정하여 그룹을 구성한다.

- 이름: fastapi-mig
- 인스턴스 템플릿: fastapi-template

(2)에서 생성한 템플릿

- 자동 확장(Autoscaling) 설정
 - 모드: 활성화
 - 최소 인스턴스 수: 2
 - 최대 인스턴스 수: 10
 - 확장 신호: CPU 사용률
 - 목표 값: 60 %

- 검색 서비스의 CPU 집약적 특성을 고려하여 과부하 이전에 선제적으로 확장하도록 설정

- 상태 확인(Health Check) 설정
 - [상태 확인 생성] 클릭
 - 이름: api-health-check
 - 프로토콜: HTTP
 - 요청 경로: /health

- FastAPI 서버가 정상 동작 시 200 OK를 반환하도록 구현되어 있어야 한다.

5. 설정 완료 후 [만들기] 클릭하여 관리형 인스턴스 그룹 생성

참고

정적 웹 서버(Nginx)가 단일 VM 으로 구성되어있으므로 비관리형 인스턴스 그룹을 생성하여 VM5 를 포함한 nginx-group 을 구성한다. 해당 그룹은 이후 로드 밸런서의 프론트엔드 백엔드 서비스로 연결된다.

← fastapi-mig

Edit Restart/replace VMs Delete Group

상용하는 코드

개요 세부정보 모니터링 오류

상태별 인스턴스

인스턴스 2개

2

상태별 인스턴스

100% 정상

상태 점검 [api-health-check](#)

자동 확장

사용 (min 2, max 10)

Based on 측정값의 1개 and 일정 0개

Status Ready

Creation Time 12월 11, 2025, 5:18:07 PM UTC+09:00

Description

Target running size 2

Template [fastapi-template \(리전\)](#)

Location us-central1 (3/4)

크기 조절 유형 None

VM 인스턴스

정지 중지 시작/재개 그룹에서 삭제 삭제

필터 속성 이름 또는 값 입력

상태	이름 ↑	생성 시간	템플릿	영역	인스턴스별 구성	내부 IP	외부 IP	상태 확인 상황	연결
☐	fastapi-mig-9xvs	12월 11, 2025, 5:19:02 PM UTC+09:00	fastapi-template (리전)	us-central1-c		10.128.0.18 (nec0)	34.71.57.69 02	정상	SSH
☐	fastapi-mig-zv1p	12월 11, 2025, 5:19:01 PM UTC+09:00	fastapi-template (리전)	us-central1-f		10.128.0.17 (nec0)	35.202.227.147 02	정상	SSH

[그림-47]

인스턴스 그룹

인스턴스 그룹 만들기 새로고침 삭제

인스턴스 그룹은 부하 분산 및 자동 서비스(예: 자동 확장 및 자동 복구)를 사용하는 VM 인스턴스의 모음입니다. [자세히 알아보기](#)

필터 속성 이름 또는 값 입력

상태	이름 ↑	인스턴스	템플릿	그룹 유형	생성 시간	추천	자동 확장	위치	상태 확인을 사용하는 리소스
☐	fastapi-mig	2	fastapi-template (리전)	관리형	12월 11, 2025, 5:18:07 PM UTC+09:00		사용 중: 대상 CPU 사용률 60%	us-central1 (3/4)	
☐	nginx-group	1	-	비관리형	12월 11, 2025, 5:49:48 PM UTC+09:00			us-central1-a	

[그림-48]

88

[10-2-4] 로드밸런서(Load Balancer) 생성

외부 사용자 트래픽을 수신하기 위해 애플리케이션 로드 밸런서를 구성한다.

생성 절차

1. 네트워크 서비스 -> 부하 분산(Load Balancing) 메뉴로 이동
2. [로드 밸런서 만들기]를 클릭
3. 유형 선택
 - 애플리케이션 부하 분산기(HTTP/HTTPS) 선택
 - Public(인터넷 연결) 선택
 - Global(전역) 선택
 - 로드밸런서 이름: politisearch-lb
4. 로드 밸런서 구성
 - 프론트엔드 구성
 - 이름: frontend-http
 - 프로토콜: HTTP
 - IP 버전: IPv4
 - 포트: 80
 - 설정 완료
 - 백엔드 구성(백엔드 서비스 및 버킷 만들기 선택)
 - (A) FastAPI 백엔드 서비스
 - 이름: backend-fastapi
 - 백엔드 유형: 인스턴스 그룹
 - 프로토콜: HTTP
 - 백엔드: fastapi-mig 선택
 - 포트 번호: 8000
 - 상태 확인: api-health-check 선택
 - 생성
 - (B) Nginx 백엔드 서비스
 - 이름: backend-nginx
 - 백엔드 유형: nginx-group 선택
 - 포트 번호: 80
 - 상태 확인: HTTP 포트 80용 상태 확인 생성 후 연결
 - 생성
 - 호스트 및 경로 규칙 설정
 - 단순 호스트 및 경로 규칙 선택
 - 호스트: *
 - 경로 /api/* -> backend-fastapi 로 라우팅
 - 기본 서비스: backend-nginx
5. 검토 및 완를 클릭하여 로드 밸런서 생성



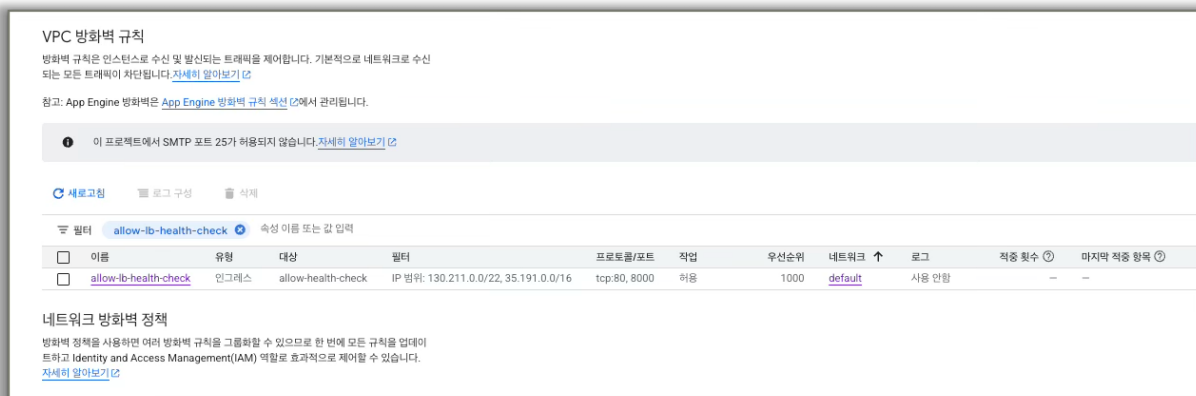
[그림-49]

[10-2-5] 방화벽 규칙 설정

로드밸런서가 백엔드 인스턴스의 상태를 확인하고 정상적으로 트래픽을 전달할 수 있도록 방화벽 규칙을 설정한다. 외부 사용자가 백엔드 포트(8000)에 직접 접근하는 것을 허용하지 않고, 로드 밸런서를 통한 접근만 허용하는 것이 목적이다.

생성 절차

1. VPC 네트워크 -> 방화벽 메뉴로 이동
2. [방화벽 규칙 만들기]를 클릭
 - 이름: allow-lb-health-check
 - 대상 태그: allow_health-check
(인스턴스 템플릿 생성 시 설정한 네트워크 태그)
 - 소스 IPv4 범위: 130.211.0.0/22, 35.191.0.0/16 (Google Cloud Load Balancer의 상태 확인전용 IP 대역)
 - 허용 프로토콜 및 포트: TCP: 8000, 80
3. 만들기를 클릭하여 방화벽 규칙 생성



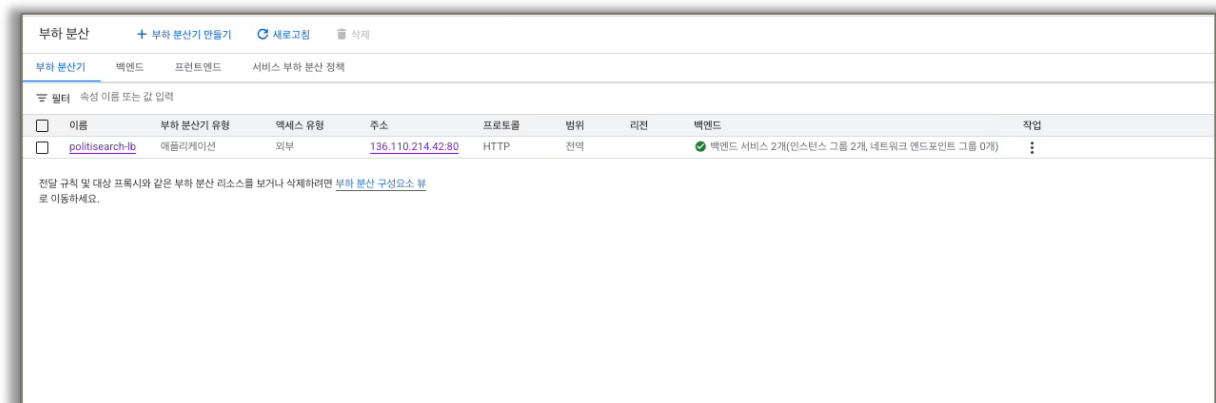
[그림-50]

[10-2-6] 테스트

구성한 로드 밸런서와 백엔드 서비스가 정상적으로 동작하는지 확인하기 위해 접속 테스트를 수행한다.

테스트 절차

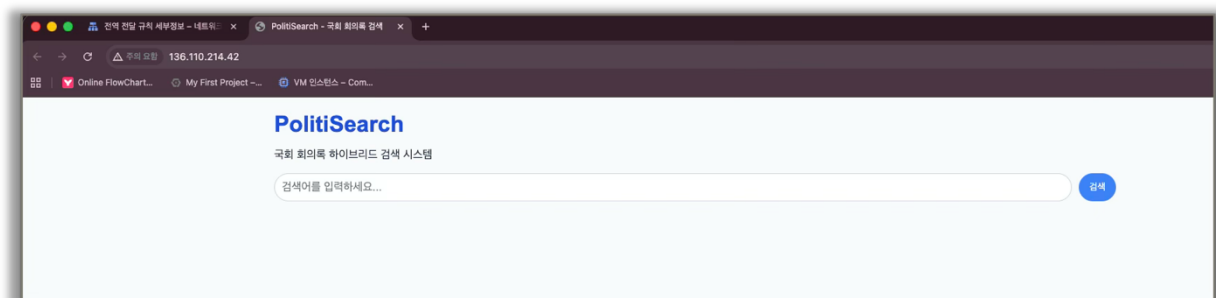
1. 네트워크 서비스 -> 부하 분산 메뉴에서 생성한 politisearch-lb를 선택
2. IP: Port 항목에 표시된 로드 밸런서의 외부 IP 주소를 복사 (136.110.214.42:80) 또는 클릭하여 접속



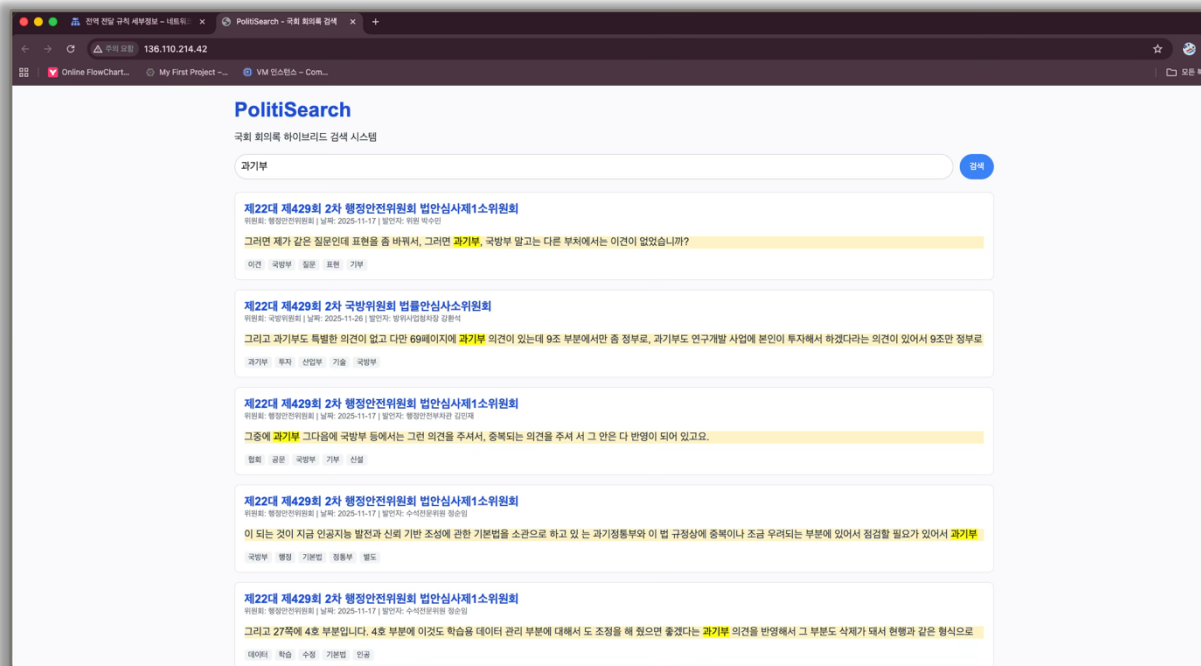
[그림-51]

[10-3] 결과 화면

로드 밸런서를 통해 단일 IP로 접속 시 정적 웹 페이지와 검색 API가 정상적으로 동작하는 것을 확인하였다.



[그림-52]



[그림-53]

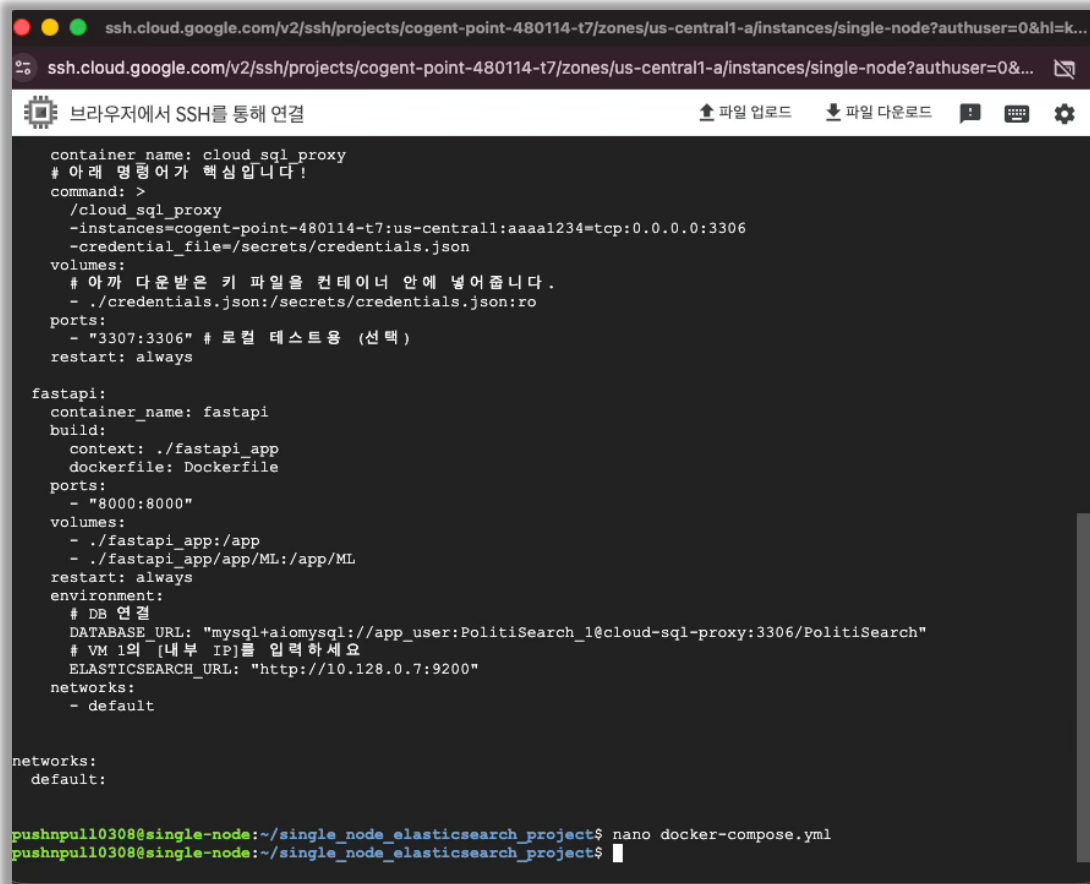
VM 인스턴스									
필터 속성 이름 또는 값 입력									
☐ 상태	이름 ↑	영역	권장사항	다음에서 사용 중:	내부 IP	외부 IP	연결		
☐	elasticsearch	us-central1-a			10.128.0.6 (nic0)		SSH	▼	⋮
☐	fastapi-mig-9xvs	us-central1-c		fastapi-mig	10.128.0.18 (nic0)	34.71.57.69 (nic0)	SSH	▼	⋮
☐	fastapi-mig-zv1p	us-central1-f		fastapi-mig	10.128.0.17 (nic0)	35.202.227.147 (nic0)	SSH	▼	⋮
☐	node-1	us-central1-a			10.128.0.7 (nic0)	34.66.137.193 (nic0)	SSH	▼	⋮
☐	node-2	us-central1-a			10.128.0.8 (nic0)	104.154.187.37 (nic0)	SSH	▼	⋮
☐	node-3	us-central1-a			10.128.0.9 (nic0)	34.61.255.121 (nic0)	SSH	▼	⋮
☐	single-node	us-central1-a			10.128.0.5 (nic0)	34.30.245.171 (nic0)	SSH	▼	⋮
☐	webserver	us-central1-a		nginx-group	10.128.0.16 (nic0)	136.119.173.184 (nic0)	SSH	▼	⋮

[그림-54]

[10-4] 수정 및 업데이트 방법

[10-4-1] 원본(VM1) 수정

1. 중지해 두었던 VM1 인스턴스를 다시 시작
2. SSH로 접속하여 수정할 부분을 수정
3. 파일을 저장한 뒤 종료
4. VM1 을 다시 중지



```
ssh.cloud.google.com/v2/ssh/projects/cogent-point-480114-t7/zones/us-central1-a/instances/single-node?authuser=0&hl=k...
ssh.cloud.google.com/v2/ssh/projects/cogent-point-480114-t7/zones/us-central1-a/instances/single-node?authuser=0&...
브라우저에서 SSH를 통해 연결
파일 업로드
파일 다운로드
container name: cloud_sql_proxy
# 아래 명령어가 핵심입니다!
command: >
  /cloud_sql_proxy
  -instances=cogent-point-480114-t7:us-central1:aaaa1234=tcp:0.0.0.0:3306
  -credential_file=/secrets/credentials.json
volumes:
  # 아까 다운받은 키 파일을 컨테이너 안에 넣어줍니다.
  - ./credentials.json:/secrets/credentials.json:ro
ports:
  - "3307:3306" # 로컬 테스트용 (선택)
restart: always

fastapi:
  container_name: fastapi
  build:
    context: ./fastapi_app
    dockerfile: Dockerfile
  ports:
    - "8000:8000"
  volumes:
    - ./fastapi_app/app
    - ./fastapi_app/app/ML:/app/ML
  restart: always
  environment:
    # DB 연결
    DATABASE_URL: "mysql+aiomysql://app_user:PolitiSearch_1@cloud-sql-proxy:3306/PolitiSearch"
    # VM 1의 [내부 IP]를 입력하세요
    ELASTICSEARCH_URL: "http://10.128.0.7:9200"
  networks:
    - default

networks:
  default:

pushnpull10308@single-node:~/single_node_elasticsearch_project$ nano docker-compose.yml
pushnpull10308@single-node:~/single_node_elasticsearch_project$
```

[그림-55]

[10-4-2] 새로운 이미지 생성(v2)

변경 사항을 모든 백엔드 인스턴스에 동일하게 반영하기 위해, 수정이 완료된 VM1을 기반으로 새로운 커스텀 이미지를 생성한다.

1. Compute Engine -> 스토리지 -> 이미지 메뉴로 이동
2. [이미지 만들기] 클릭
 - 이름: fastapi-backend-image-v2
 - 소스: 디스크
 - 소스 디스크: 설정을 수정한 VM1의 부트 디스크 선택
3. [만들기] 클릭하여 이미지 생성

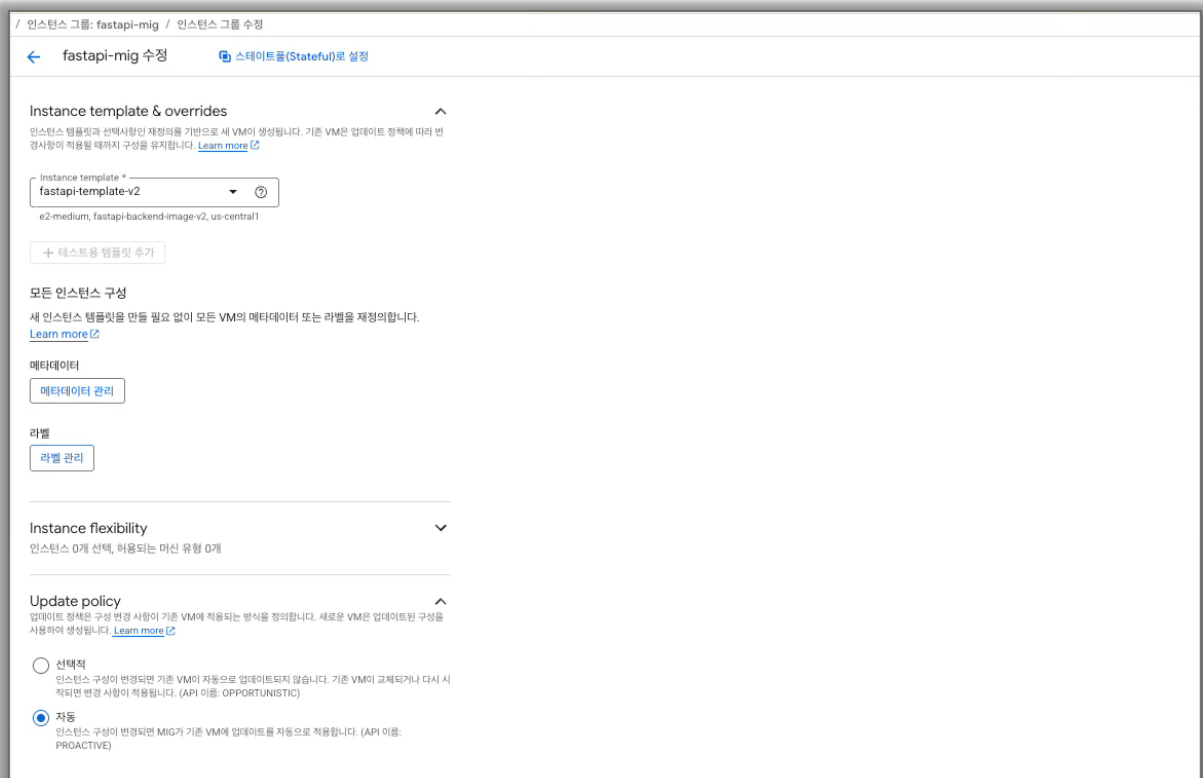


[그림-56]

[10-4-3] 인스턴스 템플릿 만들기

새롭게 생성한 커스텀 이미지를 기반으로, 자동 확장 시 사용될 인스턴스 템플릿을 다시 정의한다.

1. Compute Engine -> 인스턴스 템플릿 메뉴로 이동
2. [인스턴스 템플릿 만들기]를 클릭
 - 이름: fastapi-template-v2
 - 머신 유형: e2-medium (기존 템플릿과 동일한 사양 유지)
 - 부트 디스크 설정:
 - [맞춤 이미지] 탭 선택
 - fastapi-backend-image-v2 이미지 지정
 - 고급 옵션 -> 네트워킹:
 - 네트워크 태그: allow-health-check (로드 밸런서 상태 확인 트래픽 허용 목적)
 - 고급 옵션 -> 관리 -> 시작 스크립트
 - 기존과 동일하게 Docker 컨테이너를 기동하는 명령어 사용
3. [만들기]를 클릭하여 템플릿을 생성



[그림-57]



[그림-58]

[10-4-4] 운영 서버 무중단 배포(Rolling Update)

기존 운영 중인 관리형 인스턴스 그룹(fastapi-mig)에 새 버전(v2) 템플릿을 적용하기 위해 롤링 업데이트를 수행하였다. 이를 통해 서비스 중단 없이 백엔드 서버를 순차적으로 교체할 수 있다.

1. Compute Engine -> 인스턴스 그룹 메뉴로 이동
2. fastapi-mig를 클릭하여 상세 화면으로 진입
3. 상단의 [VM 업데이트(Update VMs)] 버튼 클릭
4. 업데이트 설정을 다음과 같이 구성
 - 새 템플릿: fastapi-template-v2 선택
 - 업데이트 모드: 자동(Proactive)
 (인스턴스를 하나씩 순차 교체하여 무중단 배포 수행)
 - 최대 서지(Max Surge): 1
 (업데이트 중 임시 인스턴스를 추가 생성하여 안정성 확보)
 - 최대 사용할 수 없음(Max Unavailable): 0 또는 1
 (0으로 설정 시 서비스 중단 없이 배포 가능)
5. [업데이트] 버튼을 클릭하여 롤링 업데이트를 시작

The screenshot shows the Google Cloud Platform console for the 'fastapi-mig' instance group. The top section displays the group's status as 'Updating' and provides details like creation time, description, target running size (2), template (fastapi-template-v2), location (us-central1), and in-use by (backend-fastapi). Below this, a table lists the VM instances, showing their status, name, creation time, template, region, instance type, internal IP, external IP, and current state.

상태	이름	생성 시간	템플릿	영역	인스턴스별 구성	내부 IP	외부 IP	상태 확인 상황	연결
정상	fastapi-mig-9xys	12월 11, 2025, 5:19:02 PM UTC+09:00	fastapi-template (리전)	us-central1-c		10.128.0.18 (nic0)	34.71.57.69	정상	SSH
제외 시간	fastapi-mig-hn16	12월 12, 2025, 2:14:18 AM UTC+09:00	fastapi-template-v2 (리전)	us-central1-c		10.128.0.19 (nic0)	136.116.70.161	제외 시간	SSH
정상	fastapi-mig-zv1p	12월 11, 2025, 5:19:01 PM UTC+09:00	fastapi-template (리전)	us-central1-f		10.128.0.17 (nic0)	35.202.227.147	정상	SSH
정상	fastapi-mig-zzn3	12월 12, 2025, 2:14:17 AM UTC+09:00	fastapi-template-v2 (리전)	us-central1-f		10.128.0.20 (nic0)	35.202.103.183	정상	SSH

[그림-59]

인스턴스 그룹: fastapi-mig

← fastapi-mig

Edit

Restart/replace VMs

Delete Group

개요

세부정보

모니터링

오류

상태별 인스턴스

인스턴스 4개

2

상태별 인스턴스

50% 정상

상태 점검 api-health-check

지동 확장

사용 (min 2, max 10)

Based on 확장할 때 1개 and 일일 0개

Status

Updating

Creation Time

12월 11, 2025, 5:18:07 PM UTC+09:00

Description

Target running size

2

Template

fastapi-template (리전), fastapi-template-v2 (리전)

Location

us-central1 (3/4)

크기 조절 요청

None

In use by

backend-fastapi

VM 인스턴스

정지

중지

시작/재개

그룹에서 삭제

삭제

필터

속성 이름 또는 값 입력

상태	이름	생성 시간	템플릿	영역	인스턴스별 구성	내부 IP	외부 IP	상태 확인 상황	연결
☐	fastapi-mig-9xvs	12월 11, 2025, 5:19:02 PM UTC+09:00	fastapi-template (리전)	us-central1-c		10.128.0.18 (nic0)	34.71.57.69	정상	SSH
☐	fastapi-mig-hn16	12월 12, 2025, 2:14:18 AM UTC+09:00	fastapi-template-v2 (리전)	us-central1-c		10.128.0.19 (nic0)	136.116.70.161	알 수 없음	SSH
☐	fastapi-mig-zv1p	12월 11, 2025, 5:19:01 PM UTC+09:00	fastapi-template (리전)	us-central1-f		10.128.0.17 (nic0)	35.202.227.147	정상	SSH
☐	fastapi-mig-zzn3	12월 12, 2025, 2:14:17 AM UTC+09:00	fastapi-template-v2 (리전)	us-central1-f		10.128.0.20 (nic0)	35.202.103.183	알 수 없음	SSH

[그림-60]

인스턴스 그룹: fastapi-mig

← fastapi-mig

Edit

Restart/replace VMs

Delete Group

개요

세부정보

모니터링

오류

상태별 인스턴스

인스턴스 4개

2

상태별 인스턴스

100% 정상

상태 점검 api-health-check

지동 확장

사용 (min 2, max 10)

Based on 확장할 때 1개 and 일일 0개

Status

Updating

Creation Time

12월 11, 2025, 5:18:07 PM UTC+09:00

Description

Target running size

2

Template

fastapi-template-v2 (리전)

Location

us-central1 (3/4)

크기 조절 요청

None

In use by

backend-fastapi

VM 인스턴스

정지

중지

시작/재개

그룹에서 삭제

삭제

필터

속성 이름 또는 값 입력

상태	이름	생성 시간	템플릿	영역	인스턴스별 구성	내부 IP	외부 IP	상태 확인 상황	연결
☐	fastapi-mig-9xvs	12월 11, 2025, 5:19:02 PM UTC+09:00	-	us-central1-c		10.128.0.18 (nic0)	34.71.57.69	정상	SSH
☒	fastapi-mig-hn16	12월 12, 2025, 2:14:18 AM UTC+09:00	fastapi-template-v2 (리전)	us-central1-c		10.128.0.19 (nic0)	136.116.70.161	정상	SSH
☐	fastapi-mig-zv1p	12월 11, 2025, 5:19:01 PM UTC+09:00	-	us-central1-f		10.128.0.17 (nic0)	35.202.227.147	정상	SSH
☒	fastapi-mig-zzn3	12월 12, 2025, 2:14:17 AM UTC+09:00	fastapi-template-v2 (리전)	us-central1-f		10.128.0.20 (nic0)	35.202.103.183	정상	SSH

[그림-61]